

Data Structures

Technical Notes

Rodrigo Kang

Table of contents

1	Introduction	11
2	A First Comparison	14
3	Data Structures — Lists	15
3.1	Creating Lists	16
3.2	Length of a List	18
3.3	Accessing List Elements	19
3.3.1	Negative Indices	20
3.4	List Slicing	22
3.4.1	Omitting Slice Boundaries	23
3.4.2	Slice Steps	24
3.4.3	Slicing and Out-of-Range Boundaries	25
3.5	Lists Are Mutable	25
3.5.1	Slice Assignment Can Change the Length	26
3.6	Membership	27
3.7	Equality of Lists	28
3.8	Modifying Lists	29
3.9	Adding Elements	29
3.9.1	The append() Method	29
3.9.2	The extend() Method	30
3.9.3	The insert() Method	32
3.10	Removing Elements	32
3.10.1	The remove() Method	32
3.10.2	The pop() Method	33
3.10.3	The del Statement	34
3.10.4	The clear() Method	35
3.11	Concatenation	36
3.12	Repetition	37
3.13	Searching and Counting	37
3.13.1	The index() Method	37
3.13.2	The count() Method	38

3.14	Reversing Lists	38
3.15	Sorting Lists	39
3.15.1	sort() versus sorted()	40
3.15.2	Sorting with a Key	42
3.16	Operations That Modify In Place	42
3.17	Variables and Object References	43
3.18	Equality versus Identity	44
3.19	Aliasing and Mutation	45
3.20	Copying Lists	46
3.21	Shallow Copies	47
3.22	Deep Copies	49
3.23	Nested Lists	50
3.24	A Common Nested-List Pitfall	52
3.25	Iterating over Nested Lists	53
3.26	Structural Mutation during Iteration	55
3.27	List Comprehensions	56
3.28	Filtering with List Comprehensions	57
3.29	Conditional Expressions inside Comprehensions	58
3.30	Nested List Comprehensions	60
3.31	Comprehensions versus Explicit Loops	61
3.32	Building Lists through Iteration	62
3.32.1	Transformation	62
3.32.2	Filtering	63
3.32.3	Transformation and Filtering	63
3.33	Lists as Dynamic Collections	64
3.34	Some Computational Considerations	64
3.34.1	Direct Indexing	65
3.34.2	Membership and Search	65
3.34.3	Appending versus Inserting	66
3.35	Lists and Numerical Computing	66
3.36	Choosing Appropriate List Operations	67
4	Problems — Lists	68
4.1	Problem 1 — Indexing	69
4.2	Problem 2 — Index Bounds	69
4.3	Problem 3 — Basic Slicing	69
4.4	Problem 4 — Slice Steps	70
4.5	Problem 5 — Indexing versus Slicing	70
4.6	Problem 6 — Indexed Assignment	71
4.7	Problem 7 — Slice Assignment	71
4.8	Problem 8 — append() versus extend()	71
4.9	Problem 9 — Extending with a String	72
4.10	Problem 10 — insert() versus Assignment	72
4.11	Problem 11 — Removing by Value and Position	73
4.12	Problem 12 — pop()	73
4.13	Problem 13 — del	73
4.14	Problem 14 — sort() versus sorted()	74

4.15	Problem 15 — Sorting by a Derived Criterion	74
4.16	Problem 16 — Equality and Identity	74
4.17	Problem 17 — Aliasing	75
4.18	Problem 18 — Rebinding versus Mutation	75
4.19	Problem 19 — Shallow Copy	75
4.20	Problem 20 — Shallow Copy with Nested Lists	76
4.21	Problem 21 — Replacement versus Inner Mutation	76
4.22	Problem 22 — Deep Copy	77
4.23	Problem 23 — Repeated Nested Lists	77
4.24	Problem 24 — Concatenation and Mutation	77
4.25	Problem 25 — += and Aliasing	78
4.26	Problem 26 — Membership and Counting	78
4.27	Problem 27 — Constructing a List	79
4.28	Problem 28 — Transformation	79
4.29	Problem 29 — Filtering and Transformation	79
4.30	Problem 30 — Conditional Expression in a Comprehension	80
4.31	Problem 31 — Independent Nested Lists	80
4.32	Problem 32 — Nested Traversal	80
4.33	Problem 33 — Flattening a Nested List	81
4.34	Problem 34 — Structural Mutation during Iteration	81
4.35	Problem 35 — List Repetition Is Not Numerical Multiplication	82
4.36	Problem 36 — Building Predictions	82
4.37	Problem 37 — Selecting Valid Measurements	83
4.38	Problem 38 — Top Scores	83
4.39	Problem 39 — Copying a Configuration	83
4.40	Problem 40 — Reasoning about Objects	84
5	Summary	85
6	Data Structures — Dictionaries	86
6.1	Creating Dictionaries	87
6.1.1	Empty Dictionaries	88
6.2	The dict() Constructor	89
6.3	Length of a Dictionary	90
6.4	Accessing Dictionary Values	91
6.5	Missing Keys and KeyError	92
6.6	Membership Tests	93
6.7	Keys Are Unique	94
6.8	Dictionary Keys	95
6.9	Hashability	96
6.9.1	Why Mutability Matters for Keys	97
6.10	Dictionary Values	98
6.11	Dictionaries Are Mutable	99
6.12	Insertion Order	100
6.13	Dictionary Equality	102
6.14	Dictionaries versus Lists	103
6.15	Retrieving Values Safely	104

6.16	The <code>get()</code> Method	104
6.16.1	Distinguishing a Missing Key from a Stored <code>None</code>	106
6.17	Dictionary Views	107
6.18	Views Are Not Lists	108
6.19	Dictionary Views Are Dynamic	109
6.20	Iterating over Dictionaries	110
6.21	Iterating over Values	111
6.22	Iterating over Key-Value Pairs	111
6.23	Adding and Updating Entries	112
6.24	The <code>update()</code> Method	113
6.25	Merging Dictionaries	114
6.25.1	In-Place Merging	115
6.26	Removing Entries	116
6.27	The <code>del</code> Statement	116
6.28	The <code>pop()</code> Method	117
6.29	The <code>popitem()</code> Method	117
6.30	The <code>clear()</code> Method	118
6.31	Common Dictionary Methods	119
6.32	The <code>setdefault()</code> Method	119
6.33	Aliasing with Dictionaries	121
6.34	Equality versus Identity	122
6.35	Copying Dictionaries	123
6.36	Shallow Copying with Nested Mutable Values	123
6.37	Replacing a Value versus Mutating a Shared Value	124
6.38	Deep Copying Nested Dictionaries	126
6.39	Nested Dictionaries	127
6.40	Updating Nested Dictionaries	128
6.41	Iterating over Nested Dictionaries	129
6.42	Merging Nested Dictionaries	130
6.43	Structural Mutation during Iteration	131
6.44	Dictionary Methods and Object Semantics	132
6.45	Constructing Dictionaries from Related Iterables	133
6.46	Dictionary Comprehensions	134
6.47	Transforming Dictionary Values	136
6.48	Filtering Dictionaries	136
6.49	Transforming Keys and Values	138
6.50	Key Collisions during Construction	139
6.51	Frequency Counting	140
6.52	Grouping Values by Key	141
6.53	Lookup Tables	142
6.54	Dictionaries as Records	143
6.55	Dictionaries as Configuration Objects	144
6.56	Dictionaries for Model Metrics	145
6.57	Dictionaries for Model Collections	146
6.58	Computational Characteristics	147
6.59	Keys versus Values for Lookup	148

6.60	Avoiding Redundant Lookups	149
6.61	Choosing between Dictionaries and Other Structures	150
7	Problems - Dictionaries	151
7.1	Problem 1 — Basic Dictionary Access	151
7.2	Problem 2 — Missing Keys	151
7.3	Problem 3 — Membership	152
7.4	Problem 4 — Stored None versus Missing Key	152
7.5	Problem 5 — Duplicate Keys	153
7.6	Problem 6 — Valid and Invalid Keys	153
7.7	Problem 7 — Updating and Inserting	153
7.8	Problem 8 — Dictionary Equality	154
7.9	Problem 9 — Iterating Directly	154
7.10	Problem 10 — keys(), values(), and items()	154
7.11	Problem 11 — Dynamic Views	155
7.12	Problem 12 — Detached List versus View	155
7.13	Problem 13 — Iterating over Items	156
7.14	Problem 14 — update()	156
7.15	Problem 15 — Merging Dictionaries	157
7.16	Problem 16 — versus =	157
7.17	Problem 17 — Removing with pop()	158
7.18	Problem 18 — popitem()	158
7.19	Problem 19 — clear() and Aliasing	158
7.20	Problem 20 — setdefault()	159
7.21	Problem 21 — Aliasing	159
7.22	Problem 22 — Shallow Copy	160
7.23	Problem 23 — Shallow Copy with Mutable Values	160
7.24	Problem 24 — Replacement versus Nested Mutation	161
7.25	Problem 25 — Deep Copy	161
7.26	Problem 26 — Nested Access	162
7.27	Problem 27 — Updating Nested Dictionaries	162
7.28	Problem 28 — Shallow Merge of Nested Dictionaries	162
7.29	Problem 29 — Building from zip()	163
7.30	Problem 30 — Mismatched zip() Inputs	163
7.31	Problem 31 — Basic Dictionary Comprehension	164
7.32	Problem 32 — Transforming Values	164
7.33	Problem 33 — Filtering a Dictionary	165
7.34	Problem 34 — Key Collision	165
7.35	Problem 35 — Frequency Counting	165
7.36	Problem 36 — Grouping	166
7.37	Problem 37 — Lookup Table	166
7.38	Problem 38 — Selecting Model Names	167
7.39	Problem 39 — Structural Mutation during Iteration	168
7.40	Problem 40 — Reasoning about References	168
8	Summary	169

9	Data Structures — Sets	171
9.1	Creating Sets	172
9.2	Creating Sets with <code>set()</code>	173
9.3	The Empty Set	174
9.4	Unique Elements	175
9.5	Sets Are Not Sequences	176
9.6	Set Order	177
9.7	Length of a Set	178
9.8	Membership	178
9.9	Hashability of Set Elements	180
9.10	Sets Are Mutable	181
9.11	Why Mutable Objects Cannot Usually Be Set Elements	182
9.12	Tuples as Set Elements	183
9.13	<code>frozenset</code>	183
9.14	Iterating over Sets	184
9.15	Sets and Duplicate Removal	185
9.16	Sets versus Lists and Dictionaries	186
9.17	Computational Characteristics of Membership	187
9.18	When a Set Is the Natural Representation	188
9.19	Set Relations	189
9.20	Subsets	190
9.21	A Set Is a Subset of Itself	191
9.22	Proper Subsets	192
9.23	Supersets	193
9.24	Set Equality	194
9.25	Disjoint Sets	195
9.26	Set Operations	196
9.27	Union	196
9.28	Union of Multiple Sets	197
9.29	Intersection	198
9.30	Intersection of Multiple Sets	199
9.31	Difference	200
9.32	Finding Missing Elements	201
9.33	Symmetric Difference	202
9.34	Symmetric Difference versus Difference	203
9.35	Summary of Set Operations	204
9.36	Set Identities	205
9.37	Associativity	206
9.38	Distributive Laws	206
9.39	Difference and Intersection	207
9.40	Set Expressions as Logical Conditions	208
9.41	Practical Example — Comparing Feature Sets	209
9.42	Practical Example — Checking Required Features	210
9.43	Practical Example — Detecting Data Leakage between Splits	211
9.44	Operators versus Methods	213
9.45	Set Algebra as a Data-Processing Tool	214

9.46	Modifying Sets	214
9.47	Adding a Single Element	215
9.48	Adding Multiple Elements with <code>update()</code>	216
9.49	Removing Elements with <code>remove()</code>	217
9.50	Removing Elements with <code>discard()</code>	218
9.51	Removing an Arbitrary Element with <code>pop()</code>	218
9.52	Clearing a Set	219
9.53	In-Place Set Operations	219
9.54	Summary of Mutating Methods	221
9.55	Aliasing	221
9.56	Equality versus Identity	222
9.57	Copying Sets	223
9.58	<code>frozenset</code> and Nested Set Structures	224
9.59	Set Comprehensions	224
9.60	Filtering with Set Comprehensions	225
9.61	Extracting Unique Transformed Values	226
9.62	Conversion between Sets and Other Structures	227
9.63	Removing Duplicates while Preserving Order	228
9.64	Sets as Membership Filters	229
9.65	Comparing Categorical Values	230
9.66	Comparing Feature Schemas	230
9.67	Sets and Class Labels	231
9.68	Sets and Computational Complexity	232
9.69	Choosing Sets Deliberately	233
10	Problems - Sets	233
10.1	Problem 1 — Creating a Set	234
10.2	Problem 2 — Empty Set	234
10.3	Problem 3 — String Conversion	234
10.4	Problem 4 — Membership	235
10.5	Problem 5 — Invalid Set Elements	235
10.6	Problem 6 — Set Equality	235
10.7	Problem 7 — Subset versus Proper Subset	236
10.8	Problem 8 — Superset	236
10.9	Problem 9 — Disjointness	236
10.10	Problem 10 — Union	237
10.11	Problem 11 — Intersection	237
10.12	Problem 12 — Difference Is Directional	237
10.13	Problem 13 — Symmetric Difference	238
10.14	Problem 14 — Missing Features	238
10.15	Problem 15 — Unexpected Features	239
10.16	Problem 16 — Schema Difference	239
10.17	Problem 17 — Train/Test Overlap	239
10.18	Problem 18 — <code>add()</code>	240
10.19	Problem 19 — <code>update()</code>	240
10.20	Problem 20 — <code>remove()</code> versus <code>discard()</code>	241
10.21	Problem 21 — <code>pop()</code>	241

10.22	Problem 22 — <code>clear()</code> and Aliasing	241
10.23	Problem 23 — In-Place Intersection	242
10.24	Problem 24 — In-Place Difference	242
10.25	Problem 25 — In-Place Symmetric Difference	243
10.26	Problem 26 — Aliasing	243
10.27	Problem 27 — Copying	243
10.28	Problem 28 — <code>frozenset</code>	244
10.29	Problem 29 — Basic Set Comprehension	244
10.30	Problem 30 — Duplicate Results in a Set Comprehension	244
10.31	Problem 31 — Filtering	244
10.32	Problem 32 — Unique Normalised Labels	245
10.33	Problem 33 — Unique Letters in a Phrase	245
10.34	Problem 34 — Common Letters between Words	245
10.35	Problem 35 — Symmetric Difference of Integer Intervals	245
10.36	Problem 36 — Maximum without <code>max()</code>	246
10.37	Problem 37 — Sieve of Eratosthenes	246
10.38	Problem 38 — Removing Duplicates while Preserving Order	246
10.39	Problem 39 — Class-Set Comparison	247
10.40	Problem 40 — Reasoning about Mutation and References	247
11	Summary	248
12	Data Structures — Tuples	250
12.1	Tuple Literals	251
12.2	The Comma Creates the Tuple	252
12.3	Tuple Packing	253
12.4	Creating Tuples with <code>tuple()</code>	253
12.5	The Empty Tuple	254
12.6	Single-Element Tuples	255
12.7	Length of a Tuple	256
12.8	Indexing	257
12.9	Negative Indexing	258
12.10	Slicing	259
12.11	Omitting Slice Bounds	260
12.12	Slice Steps	260
12.13	Membership	261
12.14	Tuples Preserve Duplicates	263
12.15	Tuples Are Immutable	263
12.16	Immutability Does Not Mean That Every Contained Object Is Immutable	264
12.17	Tuple Immutability and Hashability	265
12.18	Not Every Tuple Is Hashable	266
12.19	Immutability versus Rebinding	267
12.20	“Modifying” a Tuple by Reconstruction	268
12.21	When Immutability Is Useful	269
12.22	Lists versus Tuples	269
12.23	Tuples as Records	270
12.24	Tuples as Composite Keys	271

12.25	Choosing a Tuple Deliberately	272
12.26	Tuple Unpacking	273
12.27	Packing and Unpacking	273
12.28	Unpacking Requires Compatible Structure	275
12.29	Extended Unpacking with *	276
12.30	Starred Targets in Different Positions	277
12.31	Ignoring Values with _	278
12.32	Combining _ and Starred Unpacking	279
12.33	Swapping Variables	280
12.34	Multiple Assignment	280
12.35	Nested Unpacking	281
12.36	Returning Multiple Values from Functions	282
12.37	Tuple Concatenation	283
12.38	Concatenation Does Not Mutate a Tuple	284
12.39	Tuple Repetition	285
12.40	Repetition and Object References	286
12.41	Tuple Methods	287
12.42	count()	287
12.43	index()	288
12.44	Iterating over Tuples	289
12.45	Iterating with enumerate()	289
12.46	Unpacking Tuples in Loops	290
12.47	Unpacking More Complex Records in Loops	291
12.48	Tuples and Other Data Structures	291
12.49	Tuples inside Lists	292
12.50	Tuples inside Dictionaries	293
12.51	Tuples inside Sets	294
12.52	Converting Iterables to Tuples	294
12.53	The zip() Function	295
12.54	Understanding zip() Positionally	297
12.55	Iterating Directly over zip()	297
12.56	zip() with More Than Two Iterables	298
12.57	zip() Stops at the Shortest Iterable	299
12.58	Strict zip()	300
12.59	zip() Objects Are Iterators	301
12.60	Converting zip() to a Tuple	302
12.61	Converting zip() to a Dictionary	303
12.62	Unzipping with zip(*)	304
12.63	zip() as a Structural Operation	305
12.64	When Unpacking Improves Readability	306
12.65	Tuples as Structural Sequences	307
12.66	Practical Tuple Patterns	308
12.67	Fixed-Size Records	309
12.68	Tuples versus Dictionaries as Records	309
12.69	Tuples versus Lists	311
12.70	Tuples versus Sets	312

12.71	Tuples versus Dictionaries and Sets as Hashable Objects	313
12.72	Composite Identifiers	314
12.73	Structural Validation through Unpacking	315
12.74	Swapping and State Updates	316
12.75	Functions Returning Structured Results	317
12.76	Tuples and Function Arguments	317
12.77	Combining enumerate() and Tuples	318
12.78	Dictionary items() and Tuples	319
12.79	Comprehensions Producing Tuples	320
12.80	Sorting Collections of Tuples	321
12.81	Lexicographic Comparison	322
12.82	Computational Characteristics	323
12.83	Choosing among the Main Data Structures	324
13	Problems - Tuples	325
13.1	Problem 1 — Basic Tuple Creation	325
13.2	Problem 2 — The Single-Element Tuple	326
13.3	Problem 3 — Parentheses versus Commas	326
13.4	Problem 4 — Converting an Iterable	326
13.5	Problem 5 — Indexing	327
13.6	Problem 6 — Slicing	327
13.7	Problem 7 — Membership	328
13.8	Problem 8 — Duplicate Values	328
13.9	Problem 9 — Immutability	329
13.10	Problem 10 — Rebinding Is Not Mutation	329
13.11	Problem 11 — Mutable Object inside a Tuple	329
13.12	Problem 12 — Replacing a Mutable Element	330
13.13	Problem 13 — Tuple Hashability	330
13.14	Problem 14 — Tuple as a Dictionary Key	331
13.15	Problem 15 — Tuple as a Set Element	331
13.16	Problem 16 — Basic Packing	331
13.17	Problem 17 — Basic Unpacking	332
13.18	Problem 18 — Unpacking Error	332
13.19	Problem 19 — Starred Unpacking	332
13.20	Problem 20 — Ignoring Values	333
13.21	Problem 21 — Swapping Variables	333
13.22	Problem 22 — Nested Unpacking	334
13.23	Problem 23 — Function Returning a Tuple	334
13.24	Problem 24 — Expanding Function Arguments	334
13.25	Problem 25 — Concatenation	335
13.26	Problem 26 — Repetition	335
13.27	Problem 27 — Repeated Mutable Reference	336
13.28	Problem 28 — Iterating with Unpacking	336
13.29	Problem 29 — enumerate()	336
13.30	Problem 30 — zip()	337
13.31	Problem 31 — zip() Iterator Consumption	337
13.32	Problem 32 — Unequal Lengths with zip()	338

13.33	Problem 33 — Unzipping	338
13.34	Problem 34 — Classification by Sign	339
13.35	Problem 35 — Grade Classification	340
13.36	Problem 36 — Character Counts as Tuple Records	340
13.37	Problem 37 — Average by Subject	341
13.38	Problem 38 — Points and Quadrants	341
13.39	Problem 39 — Words and Initial Letters	342
13.40	Problem 40 — Reasoning about Tuples and References	342
14	Summary	343

1 Introduction

The basic types introduced earlier allow individual values to be represented in Python. In practical programs, however, information rarely consists of isolated values. Data usually needs to be collected, organised, associated, searched, or processed as a group.

Python provides several built-in data structures for this purpose. Among the most important are:

- list
- tuple
- dict
- set

Although all four can be used to organise multiple objects, they represent different relationships between those objects. Choosing among them is therefore not simply a matter of syntax. The appropriate structure depends on the properties of the information being represented and on the operations that need to be performed.

A **list** represents an ordered sequence whose contents can be modified:

```

1 measurements = [
2     10.2,
3     10.7,
4     11.1,
5 ]

```

A **tuple** also represents an ordered sequence, but its structure is immutable:

```
1 coordinate = (  
2     3.5,  
3     7.2,  
4 )
```

A **dictionary** represents associations between keys and values:

```
1 customer = {  
2     "id": "C001",  
3     "age": 42,  
4     "country": "New Zealand",  
5 }
```

A **set** represents a collection of unique elements and is particularly useful when membership and set operations are central to the problem:

```
1 features = {  
2     "age",  
3     "income",  
4     "balance",  
5 }
```

These structures therefore correspond to different conceptual models:

```
1 list  -> mutable sequence  
2 tuple -> fixed sequence  
3 dict  -> key-value mapping  
4 set   -> unique membership
```

This distinction is more important than the literal notation used to create them.

For example, consider the following objects:

```
1 ["age", "income", "balance"]  
2  
3 ("age", "income", "balance")  
4  
5 {"age", "income", "balance"}  
6  
7 {
```

```

8  "age": 42,
9  "income": 85000,
10 "balance": 1200,
11 }

```

They contain similar kinds of values, but they do not represent the same information.

In the list, position and order are preserved and the sequence can be modified. In the tuple, position and order are also preserved, but the sequence itself is fixed. In the set, the relevant concept is whether an element belongs to the collection rather than where it occurs. In the dictionary, each value is associated with a key that provides a way to identify and retrieve it.

Understanding Python data structures therefore involves more than learning their methods. It requires understanding the **data model** represented by each structure.

Several questions are particularly useful when choosing a representation:

- Does the order of the elements matter?
- Does position have meaning?
- Can the collection change after it is created?
- Are duplicate values meaningful?
- Does each value need to be associated with a key?
- Is efficient membership testing important?
- Does the structure need to be hashable or contain hashable elements?

These questions lead naturally to different choices.

A sequence of observations that may grow during a computation is naturally represented by a list:

```

1 scores = [
2     0.81,
3     0.84,
4     0.89,
5 ]

```

A fixed pair of coordinates is naturally represented by a tuple:

```

1 point = (
2     3.5,
3     7.2,
4 )

```

A collection associating model names with evaluation scores is naturally represented by a dictionary:

```
1 scores = {  
2     "linear": 0.81,  
3     "tree": 0.84,  
4     "forest": 0.89,  
5 }
```

A collection whose main purpose is to represent which features are present is naturally represented by a set:

```
1 selected_features = {  
2     "age",  
3     "income",  
4     "balance",  
5 }
```

The choice of data structure can therefore make the intended meaning of a program more explicit.

2 A First Comparison

The main properties of the four built-in structures can be summarised as follows:

Data structure	Main abstraction	Ordered	Mutable	Duplicates	Access model
list	mutable sequence	yes	yes	allowed	position
tuple	fixed sequence	yes	no	allowed	position
dict	mapping	yes*	yes	keys are unique	key
set	unique membership	no**	yes	not allowed	membership

Table 1: First comparison of the principal built-in Python data structures.

* Dictionaries preserve insertion order in modern Python, but their fundamental abstraction is a mapping between keys and values rather than positional access.

** Sets should not be treated as ordered sequences. Their iteration order is not part of the positional semantics of the structure.

This comparison already reveals an important principle:

Data structures should be chosen according to the relationships and operations that define the data, not merely according to which structure can contain the required values.

The following sections develop each structure in detail, beginning with lists and then considering tuples, dictionaries, and sets. Their individual behaviour will also provide a basis for comparing them and understanding how they can be combined in larger programs.

3 Data Structures — Lists

A **list** is one of Python's fundamental built-in data structures. It represents an ordered, mutable collection of objects and is commonly used when a program needs to store, access, modify, or iterate over a sequence of values.

A list is written as a comma-separated sequence of elements enclosed in square brackets:

```
1 values = [3, 7, 2, 9]
2
3 values
```

[3, 7, 2, 9]

Lists have several important properties:

- they are **ordered**: each element occupies a specific position;
- they are **mutable**: their contents can be changed after the list has been created;
- they can contain **duplicate values**;
- they can contain objects of different types;
- they can contain other lists and, more generally, arbitrary Python objects.

For example:

```
1 data = [42, 3.14, "Python", True]
2
3 data
```

```
[42, 3.14, 'Python', True]
```

This list is heterogeneous: its elements have different types.

Python does not require all elements of a list to share a common type. This flexibility distinguishes lists from many specialised numerical data structures, where homogeneous data types are often an important part of the representation.

In data science and machine learning, lists frequently appear as general-purpose containers: collections of feature names, model configurations, file paths, intermediate results, labels, or other Python objects. For numerical computation itself, specialised structures such as NumPy arrays are usually more appropriate, but lists remain a fundamental part of the Python language and its surrounding ecosystem.

3.1 Creating Lists

A list can be created directly using square brackets:

```
1 numbers = [1, 2, 3, 4, 5]
2 names = ["Alice", "Bob", "Charlie"]
3
4 numbers, names
```

```
([1, 2, 3, 4, 5], ['Alice', 'Bob', 'Charlie'])
```

An empty list contains no elements:

```
1 empty = []
2
3 empty
```

```
[]
```

Its length is zero:

```
1 len(empty)
```

```
0
```


The built-in `list()` constructor provides another way to create a list.

For example:

```
1 list()
```

```
[]
```

also creates an empty list.

More importantly, `list()` can construct a list from an iterable:

```
1 list("Python")
```

```
['P', 'y', 't', 'h', 'o', 'n']
```

The string is iterable, so its characters become the elements of the resulting list:

```
1 ['P', 'y', 't', 'h', 'o', 'n']
```

Similarly:

```
1 list(range(5))
```

```
[0, 1, 2, 3, 4]
```

produces

```
1 [0, 1, 2, 3, 4]
```

The distinction is useful:

```
1 [object]
```

creates a list containing `object` as one element, whereas

```
1 list(iterable)
```

iterates over its argument and uses the resulting values as the elements of the new list.

For example:

```
1 ["Python"], list("Python")
```

```
(['Python'], ['P', 'y', 't', 'h', 'o', 'n'])
```

The first expression creates a list of length one containing a string. The second creates a list of length six containing the individual characters of that string.

3.2 Length of a List

The number of elements in a list is obtained using the built-in `len()` function:

```
1 values = [10, 20, 30, 40, 50]
```

```
2
```

```
3 len(values)
```

```
5
```

If a list contains n elements, its valid non-negative indices are

$$0, 1, 2, \dots, n - 1.$$

Consequently, the final element is located at index

$$n - 1.$$

This relationship between length and indexing is fundamental when working with Python sequences.

For an empty list,

```
1 values = []
```

```
2
```

```
3 len(values)
```

```
0
```

returns 0.

An empty list is also false in a Boolean context:

```

1 values = []
2
3 if not values:
4     print("The list is empty")

```

The list is empty

Therefore, when the intention is simply to determine whether a list contains any elements,

```

1 if values:
2     ...

```

or

```

1 if not values:
2     ...

```

is generally more direct than explicitly comparing `len(values)` with zero.

3.3 Accessing List Elements

Because lists are ordered, every element has a positional index.

Consider:

```

1 names = ["Emily", "James", "Sophia", "Daniel", "Emma"]
2
3 names

```

`['Emily', 'James', 'Sophia', 'Daniel', 'Emma']`

The correspondence between positions and elements is

Index	Element
0	"Emily"
1	"James"
2	"Sophia"
3	"Daniel"
4	"Emma"

Index	Element
-------	---------

Table 2: Zero-based indexing of a Python list.

Python uses **zero-based indexing**, so the first element has index 0, not 1.

Individual elements are accessed using square brackets:

```
1 names[0]
```

'Emily'

```
1 names[2]
```

'Sophia'

For a list of length n , accessing index i using a non-negative index requires

$$0 \leq i < n.$$

Attempting to access a position outside the valid range raises an `IndexError`:

```
1 names[5]
```

```
1 IndexError: list index out of range
```

The error occurs because a list of length five has valid non-negative indices only from 0 through 4.

3.3.1 Negative Indices

Python sequences also support negative indexing.

A negative index counts backward from the end of the sequence:

Index	Position
-1	last element

Index	Position
-2	second-to-last element
-3	third-to-last element
$\vdots$	$\vdots$

Table 3: Interpretation of negative list indices.

For example:

```
1 names = ["Emily", "James", "Sophia", "Daniel", "Emma"]
2
3 names[-1]
```

'Emma'

returns the final element, while

```
1 names[-2]
```

'Daniel'

returns the second-to-last element.

For a list of length n , the indices

$$-n, -(n-1), \dots, -2, -1$$

refer to the same elements as

$$0, 1, \dots, n-2, n-1,$$

respectively.

Thus,

```
1 names[0] == names[-len(names)]
```

True

evaluates to True.

Negative indexing is particularly useful when the desired position is naturally described relative to the end of a sequence.

3.4 List Slicing

Indexing retrieves a single element. **Slicing** retrieves a range of elements and produces a new list.

The general syntax is

```
1 sequence[start:stop:step]
```

where

- start is the index at which the slice begins;
- stop is the boundary at which the slice ends;
- step determines the spacing between selected elements.

As with `range()`, the stop position is excluded.

Consider:

```
1 values = [10, 20, 30, 40, 50, 60]
2
3 values[1:4]
```

[20, 30, 40]

The selected indices are

1, 2, 3,

so the result is

```
1 [20, 30, 40]
```

but not 50, because index 4 is the stopping boundary.

This half-open convention can be written as

`[start, stop)`.

It is the same convention encountered previously with `range()`.

3.4.1 Omitting Slice Boundaries

If start is omitted, slicing begins at the start of the list:

```
1 values[:3]
```

`[10, 20, 30]`

which returns the first three elements.

If stop is omitted, slicing continues to the end:

```
1 values[3:]
```

`[40, 50, 60]`

Both boundaries can be omitted:

```
1 values[:]
```

`[10, 20, 30, 40, 50, 60]`

This selects the complete sequence.

An important point is that

```
1 values[:]
```

does not simply provide another way of referring to the same list. It creates a **new list containing references to the same elements**. This distinction will become important when we discuss copying and aliasing.

3.4.2 Slice Steps

The third component of a slice specifies the step:

```
1 values = [10, 20, 30, 40, 50, 60]
2
3 values[::2]
```

[10, 30, 50]

selects every second element:

```
1 [10, 30, 50]
```

Similarly:

```
1 values[1::2]
```

[20, 40, 60]

selects the elements at odd indices:

```
1 [20, 40, 60]
```

A negative step traverses the sequence in reverse order.

For example:

```
1 values[::-1]
```

[60, 50, 40, 30, 20, 10]

returns a reversed copy of the list.

The expression

```
1 values[::-1]
```

is therefore a common slicing idiom, although Python also provides other ways to traverse or reverse a sequence.

3.4.3 Slicing and Out-of-Range Boundaries

Indexing and slicing behave differently when their boundaries exceed the length of a list.

Direct indexing beyond the valid range raises an error:

```
1 values[100]
```

but a slice is clipped to the available range:

```
1 values[2:100]
```

```
[30, 40, 50, 60]
```

This returns all available elements from index 2 onward rather than raising `IndexError`.

Similarly:

```
1 values[:100]
```

```
[10, 20, 30, 40, 50, 60]
```

returns the complete list.

This behaviour makes slicing convenient for requesting a range of elements without having to ensure that every boundary corresponds to an existing position.

3.5 Lists Are Mutable

One of the defining properties of a list is **mutability**.

A mutable object can be modified after it has been created. Individual list elements can therefore be replaced using indexed assignment:

```
1 values = [10, 20, 30, 40]
```

```
2
```

```
3 values[1] = 25
```

```
4
```

```
5 values
```

```
[10, 25, 30, 40]
```

The element at index 1 has changed from 20 to 25.

Multiple elements can also be replaced through slice assignment:

```
1 values = [10, 20, 30, 40, 50]
2
3 values[1:4] = [200, 300, 400]
4
5 values
```

[10, 200, 300, 400, 50]

Unlike ordinary slicing, which retrieves elements, placing a slice on the left-hand side of an assignment modifies the original list.

This distinction can be summarised as

```
1 sublist = values[1:4]
```

which **creates a new list**, versus

```
1 values[1:4] = replacement
```

which **modifies values**.

3.5.1 Slice Assignment Can Change the Length

The number of replacement elements does not have to equal the number of elements selected by the slice.

For example:

```
1 values = [1, 2, 3, 4, 5]
2
3 values[1:4] = [10, 20]
4
5 values
```

[1, 10, 20, 5]

The three elements

```
1 [2, 3, 4]
```

are replaced by two elements

```
1 [10, 20]
```

so the list becomes shorter.

Conversely:

```
1 values = [1, 2, 3]
2
3 values[1:2] = [10, 20, 30]
4
5 values
```

```
[1, 10, 20, 30, 3]
```

replaces one element with three elements, increasing the length of the list.

This behaviour follows from the fact that lists are mutable, dynamically sized sequences rather than fixed-size containers.

3.6 Membership

The operators `in` and `not in` test whether a value occurs in a list.

For example:

```
1 features = ["age", "income", "balance"]
2
3 "income" in features
```

```
True
```

returns `True`, whereas

```
1 "country" in features
```

```
False
```

returns False.

The complementary test is

```
1 "country" not in features
```

True

Membership tests are frequently combined with conditional statements:

```
1 features = ["age", "income", "balance"]
2
3 if "income" in features:
4     print("Income is available")
```

Income is available

The expression

```
1 x in values
```

asks whether the list contains an element equal to `x`; it does not ask for the position of that element. When the position itself is required, other operations such as `index()` may be appropriate.

3.7 Equality of Lists

Two lists are equal when they have the same length and corresponding elements compare equal in the same order.

For example:

```
1 [1, 2, 3] == [1, 2, 3]
```

True

returns True, while

```
1 [1, 2, 3] == [3, 2, 1]
```

False

returns False.

Thus, list equality is **order-sensitive**.

Likewise:

```
1 [1, 2, 2] == [1, 2]
```

False

is false because the lists have different lengths.

This should be distinguished from asking whether two variables refer to the **same list object**. That distinction concerns object identity rather than equality and becomes particularly important because lists are mutable. We will examine it when discussing references, aliasing, and copying.

3.8 Modifying Lists

Because lists are mutable, their structure can be changed after creation. Elements can be added, removed, reordered, or replaced without creating an entirely new list object.

Python provides several list methods for these operations. A key distinction throughout this section is whether an operation

- **modifies the existing list in place, or**
- **creates and returns a new list.**

This distinction becomes particularly important when multiple variables refer to the same list object.

3.9 Adding Elements

3.9.1 The `append()` Method

The `append()` method adds a single object to the end of a list:

```
1 models = ["linear regression", "decision tree"]
2
3 models.append("random forest")
4
5 models
```

```
['linear regression', 'decision tree', 'random forest']
```

The resulting list is

```
1 ['linear regression', 'decision tree', 'random forest']
```

The important point is that `append()` adds its argument as **one element**.

For example:

```
1 values = [1, 2, 3]
2
3 values.append([4, 5])
4
5 values
```

```
[1, 2, 3, [4, 5]]
```

produces

```
1 [1, 2, 3, [4, 5]]
```

The list `[4, 5]` has been added as a single nested element.

This differs from adding its individual elements, which is the role of `extend()`.

3.9.2 The `extend()` Method

The `extend()` method iterates over its argument and appends each resulting element to the list:

```
1 values = [1, 2, 3]
2
3 values.extend([4, 5])
4
5 values
```

```
[1, 2, 3, 4, 5]
```

The result is

```
1 [1, 2, 3, 4, 5]
```

The distinction between `append()` and `extend()` is therefore:

```
1 values.append(iterable)
```

adds the iterable itself as one element, whereas

```
1 values.extend(iterable)
```

adds the elements produced by the iterable individually.

For example:

```
1 a = [1, 2]
2 b = [1, 2]
3
4 a.append("abc")
5 b.extend("abc")
6
7 a, b
```

```
([1, 2, 'abc'], [1, 2, 'a', 'b', 'c'])
```

The first list becomes

```
1 [1, 2, 'abc']
```

while the second becomes

```
1 [1, 2, 'a', 'b', 'c']
```

because strings are iterable.

3.9.3 The insert() Method

The insert() method adds an element at a specified position:

```
1 list.insert(index, object)
```

For example:

```
1 features = ["age", "balance", "income"]
2
3 features.insert(1, "country")
4
5 features
```

```
['age', 'country', 'balance', 'income']
```

The resulting list is

```
1 ['age', 'country', 'balance', 'income']
```

The existing element at index 1 and all subsequent elements are shifted to the right.

Unlike indexed assignment,

```
1 features[1] = "country"
```

which replaces an existing element, insert() increases the length of the list.

3.10 Removing Elements

Python provides several mechanisms for removing list elements. The appropriate operation depends on whether the element is identified by **value**, by **position**, or whether the removed value itself is needed.

3.10.1 The remove() Method

The remove() method deletes the **first occurrence** of a specified value:


```
1 values = [10, 20, 30, 20, 40]
2
3 values.remove(20)
4
5 values
```

[10, 30, 20, 40]

The result is

```
1 [10, 30, 20, 40]
```

Only the first occurrence of 20 is removed.

If the requested value does not exist, `remove()` raises a `ValueError`.

For example:

```
1 values.remove(100)
```

would result in

```
1 ValueError: list.remove(x): x not in list
```

Therefore, when absence is possible and should not be considered an error, membership can be checked first:

```
1 if value in values:
2     values.remove(value)
```

3.10.2 The `pop()` Method

The `pop()` method removes an element by position **and returns the removed object**.

Without an argument,

```
1 values = [10, 20, 30]
2
3 last = values.pop()
4
5 last, values
```

(30, [10, 20])

the final element is removed.

The result is equivalent conceptually to taking the last element and deleting it, but `pop()` performs both operations together.

An index can also be specified:

```
1 values = [10, 20, 30, 40]
2
3 removed = values.pop(1)
4
5 removed, values
```

(20, [10, 30, 40])

which removes and returns the element at index 1.

Thus:

```
1 removed == 20
2 values == [10, 30, 40]
```

`pop()` is useful when the removed value still needs to participate in the computation.

3.10.3 The `del` Statement

The `del` statement removes an item or slice without returning it:

```
1 values = [10, 20, 30, 40]
2
3 del values[1]
4
5 values
```

[10, 30, 40]

It can also remove several elements at once:

```

1 values = [10, 20, 30, 40, 50]
2
3 del values[1:4]
4
5 values

```

[10, 50]

which leaves

```

1 [10, 50]

```

The distinction among the three mechanisms can be summarised as follows:

Operation	Identifies by	Returns removed element?
remove(value)	value	no
pop(index)	index	yes
del list[index]	index or slice	no

Table 4: Common mechanisms for removing list elements.

3.10.4 The clear() Method

The clear() method removes all elements from a list:

```

1 values = [1, 2, 3, 4]
2
3 values.clear()
4
5 values

```

[]

The list object still exists, but it is now empty.

This distinction will become important shortly. Clearing a list is not equivalent, in terms of object identity, to assigning a variable to a newly created empty list.

3.11 Concatenation

Two lists can be concatenated using the + operator:

```
1 a = [1, 2, 3]
2 b = [4, 5, 6]
3
4 c = a + b
5
6 c
```

[1, 2, 3, 4, 5, 6]

The result is

```
1 [1, 2, 3, 4, 5, 6]
```

Unlike methods such as `append()` and `extend()`, list concatenation produces a **new list**.

For example:

```
1 a = [1, 2]
2 b = [3, 4]
3
4 c = a + b
5
6 c is a, c is b
```

(False, False)

Both comparisons are False.

The original lists remain unchanged:

```
1 a, b
```

([1, 2], [3, 4])

The += operator requires more care.

For a list,

```
1 a += b
```

typically extends the existing list in place rather than behaving exactly like

```
1 a = a + b
```

This difference matters when other variables refer to the same list and will become clearer once aliasing is introduced.

3.12 Repetition

Lists can be repeated using the `*` operator:

```
1 pattern = ["A", "B", "C"]
2
3 pattern * 3
```

```
['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C']
```

which produces

```
1 ['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C']
```

The original list is not modified.

For lists containing immutable objects, repetition generally behaves as expected. However, repetition has an important subtlety when the repeated elements are themselves mutable objects. We will return to this when discussing nested lists.

3.13 Searching and Counting

3.13.1 The `index()` Method

The `index()` method returns the index of the first occurrence of a value:

```
1 values = [10, 20, 30, 20]
2
3 values.index(20)
```

1

The result is 1, even though 20 appears again later.

If the value does not exist, `index()` raises a `ValueError`.

Therefore,

```
1 value in values
```

and

```
1 values.index(value)
```

answer different questions:

- membership asks whether the value is present;
- `index()` asks where its first occurrence is located.

3.13.2 The `count()` Method

The `count()` method returns the number of occurrences of a value:

```
1 values = [1, 2, 2, 3, 2, 4]
2
3 values.count(2)
```

3

returns

```
1 3
```

This is useful when equality-based frequency is required for a specific object.

3.14 Reversing Lists

The `reverse()` method reverses the order of a list **in place**:

```

1 values = [1, 2, 3, 4]
2
3 values.reverse()
4
5 values

```

[4, 3, 2, 1]

The original list is modified.

By contrast:

```

1 values = [1, 2, 3, 4]
2
3 reversed_values = values[::-1]
4
5 values, reversed_values

```

([1, 2, 3, 4], [4, 3, 2, 1])

creates a new reversed list and leaves the original unchanged.

Python also provides the built-in `reversed()` function:

```

1 values = [1, 2, 3, 4]
2
3 list(reversed(values))

```

[4, 3, 2, 1]

`reversed()` itself returns an iterator rather than directly constructing a list.

The appropriate choice therefore depends on whether the original list should be modified and whether a concrete reversed list is actually required.

3.15 Sorting Lists

The `sort()` method sorts a list **in place**:

```
1 values = [5, 2, 8, 1, 4]
2
3 values.sort()
4
5 values
```

[1, 2, 4, 5, 8]

The result is

```
1 [1, 2, 4, 5, 8]
```

Descending order can be requested with

```
1 values.sort(reverse=True)
```

For example:

```
1 values = [5, 2, 8, 1, 4]
2
3 values.sort(reverse=True)
4
5 values
```

[8, 5, 4, 2, 1]

produces

```
1 [8, 5, 4, 2, 1]
```

3.15.1 sort() versus sorted()

Python also provides the built-in sorted() function.

Consider:

```
1 values = [5, 2, 8, 1, 4]
2
3 ordered = sorted(values)
4
5 values, ordered
```



```
([5, 2, 8, 1, 4], [1, 2, 4, 5, 8])
```

The original list remains unchanged, while `ordered` refers to a new sorted list.

This gives an important distinction:

```
1 values.sort()
```

modifies values in place, whereas

```
1 sorted(values)
```

returns a new sorted list.

Another important consequence is that `sort()` does **not** return the sorted list.

For example:

```
1 values = [3, 1, 2]
2
3 result = values.sort()
4
5 result
```

The value of `result` is `None`.

The sorting operation succeeded:

```
1 values
```

```
[1, 2, 3]
```

but its result is represented by the modified state of `values`, not by a return value.

This behaviour is common among methods whose primary purpose is to modify a mutable object in place.

3.15.2 Sorting with a Key

Sorting does not need to use the objects themselves as the ordering criterion.

The key argument specifies a function used to compute a sorting key for each element.

For example:

```
1 models = ["tree", "linear regression", "svm", "random forest"]
2
3 sorted(models, key=len)
```

```
['svm', 'tree', 'random forest', 'linear regression']
```

orders the strings according to their lengths.

Similarly:

```
1 records = [
2     ["model_a", 0.81],
3     ["model_b", 0.76],
4     ["model_c", 0.89],
5 ]
6
7 sorted(records, key=lambda record: record[1], reverse=True)
```

```
[['model_c', 0.89], ['model_a', 0.81], ['model_b', 0.76]]
```

sorts the records by their second element.

Anonymous functions such as lambda will be discussed separately. Here, the important idea is simply that sorting may be based on a derived criterion rather than on the complete object itself.

3.16 Operations That Modify In Place

It is useful to distinguish methods that modify the list from operations that return new objects.

For example:

Operation	Modifies original list?	Main return value
append()	yes	None
extend()	yes	None
insert()	yes	None
remove()	yes	None
pop()	yes	removed element
clear()	yes	None
reverse()	yes	None
sort()	yes	None
a + b	no	new list
a * n	no	new list
sorted(a)	no	new list
a[::-1]	no	new list

Table 5: In-place operations and operations that create new lists.

This distinction is not merely stylistic. Because variables refer to objects, modifying a list in place can be visible through every reference to that same list.

3.17 Variables and Object References

A Python variable should not be thought of as a container that physically contains an object. More precisely, a variable name is **bound to an object**.

Consider:

```
1 a = [1, 2, 3]
2 b = a
```

The assignment

```
1 b = a
```

does not create a second list.

Instead, both names refer to the same list object.

This can be observed by modifying the list through one of the names:

```

1 a = [1, 2, 3]
2 b = a
3
4 b.append(4)
5
6 a, b

```

`([1, 2, 3, 4], [1, 2, 3, 4])`

Both variables now appear to contain

```

1 [1, 2, 3, 4]

```

because only one list exists.

A conceptual representation is

```

1 a └─┐
2   └─┴─> [1, 2, 3, 4]
3 b └─┐

```

This situation is known as **aliasing**: multiple names refer to the same object.

3.18 Equality versus Identity

The distinction between equality and identity can now be made precise.

Consider:

```

1 a = [1, 2, 3]
2 b = [1, 2, 3]
3
4 a == b

```

True

This evaluates to True because the lists contain equal elements in the same order.

However:

```
1 a is b
```

False

evaluates to False.

The lists are equal in value but they are distinct objects.

Now consider:

```
1 a = [1, 2, 3]
2 b = a
3
4 a == b, a is b
```

(True, True)

Both expressions are True.

Therefore:

- == compares **values**;
- is compares **object identity**.

For ordinary value comparisons, == is the appropriate operation. Identity tests are useful when the question genuinely concerns whether two references denote the same object.

A particularly important identity comparison is

```
1 x is None
```

rather than

```
1 x == None
```

because None is a singleton object and the intended question is identity.

3.19 Aliasing and Mutation

Aliasing becomes important because lists are mutable.

Consider:

```

1 training_features = ["age", "income", "balance"]
2 selected_features = training_features
3
4 selected_features.remove("income")
5
6 training_features

```

['age', 'balance']

The result is

```

1 ['age', 'balance']

```

Even though the modification was expressed using `selected_features`, `training_features` refers to the same object and therefore observes the same change.

This behaviour is not specific to lists; it follows from Python's object model. It becomes particularly noticeable with mutable objects because their internal state can change without rebinding the variable names.

3.20 Copying Lists

If an independent list is required, a copy must be created.

Several common techniques create a shallow copy:

```

1 copy_1 = original.copy()
2 copy_2 = original[:]
3 copy_3 = list(original)

```

For example:

```

1 original = [1, 2, 3]
2 copied = original.copy()
3
4 copied.append(4)
5
6 original, copied

```

([1, 2, 3], [1, 2, 3, 4])

The result is

```
1 original == [1, 2, 3]
2 copied == [1, 2, 3, 4]
```

and

```
1 original is copied
```

False

is False.

At first sight, this might suggest that copying a list completely separates the two structures. That is only partly true.

The operations above create **shallow copies**.

3.21 Shallow Copies

A Python list does not contain arbitrary objects directly in the conceptual sense relevant here; it contains references to its elements.

A shallow copy creates a new outer list, but the references to its elements are copied.

This distinction matters when those elements are themselves mutable.

Consider:

```
1 original = [[1, 2], [3, 4]]
2 copied = original.copy()
3
4 original is copied
```

False

The outer lists are different objects.

However:

```
1 original[0] is copied[0]
```

True

evaluates to True.

The first inner list is shared.

A conceptual representation is

```
1 original —> [ • , • ]
2   |   |
3   |   └──> [3, 4]
4   └──> [1, 2]
5   ↑
6 copied  —> [ • , • ]
```

The two outer lists are independent, but they contain references to the same inner lists.

Consequently:

```
1 original = [[1, 2], [3, 4]]
2 copied = original.copy()
3
4 copied[0].append(99)
5
6 original, copied
```

([[1, 2, 99], [3, 4]], [[1, 2, 99], [3, 4]])

produces

```
1 ([[1, 2, 99], [3, 4]],
2  [[1, 2, 99], [3, 4]])
```

The mutation affects the shared inner list.

By contrast, replacing an entire element in one outer list does not modify the other outer list:

```
1 original = [[1, 2], [3, 4]]
2 copied = original.copy()
3
4 copied[0] = ["new"]
5
6 original, copied
```



```
(([1, 2], [3, 4]), [['new'], [3, 4]])
```

Now the outer lists contain different references at index 0.

This distinction between

```
1 copied[0].append(...)
```

and

```
1 copied[0] = ...
```

is fundamental:

- the first mutates an object shared by both lists;
- the second changes which object the copied outer list refers to at that position.

3.22 Deep Copies

When a nested mutable structure must be copied recursively, Python provides `copy.deepcopy()` from the `copy` module.

For example:

```
1 import copy
2
3 original = [[1, 2], [3, 4]]
4 copied = copy.deepcopy(original)
5
6 copied[0].append(99)
7
8 original, copied
```

```
(([1, 2], [3, 4]), [[1, 2, 99], [3, 4]])
```

The nested mutable objects are duplicated as well, so modifying the copied inner list does not modify the original structure.

The distinction can be summarised as:

```
1 assignment -> same outer object
2 shallow copy -> new outer object, shared nested objects
3 deep copy -> recursively copied object structure
```

Deep copying should not be used automatically. It may be unnecessary, expensive, or semantically incorrect when some contained objects are intentionally meant to be shared.

The important point is to understand what level of independence the program actually requires.

3.23 Nested Lists

A list may contain other lists as elements.

For example:

```
1 records = [
2     ["model_a", 0.82],
3     ["model_b", 0.87],
4     ["model_c", 0.79],
5 ]
6
7 records
```

```
[['model_a', 0.82], ['model_b', 0.87], ['model_c', 0.79]]
```

The outer list contains three elements, each of which is itself a list.

Nested indexing can be used to access inner elements:

```
1 records[1]
```

```
['model_b', 0.87]
```

returns

```
1 ['model_b', 0.87]
```

while

```
1 records[1][0]
```

'model_b'

returns

```
1 'model_b'
```

and

```
1 records[1][1]
```

0.87

returns

```
1 0.87
```

The first index selects an element of the outer list; the second index selects an element from that inner list.

Nested lists can represent hierarchical or table-like information, but they remain ordinary Python lists. They do not automatically acquire mathematical matrix semantics.

For example:

```
1 data = [  
2     [1, 2, 3],  
3     [4, 5, 6],  
4 ]
```

can be interpreted as two rows of data, but operations such as

```
1 data * 2
```

perform **list repetition**, not scalar multiplication of a mathematical matrix.

Likewise:

```
1 data + data
```

concatenates the outer lists rather than performing matrix addition.

For numerical linear algebra, specialised structures such as NumPy arrays are designed to provide the appropriate semantics and computational behaviour.

3.24 A Common Nested-List Pitfall

List repetition becomes particularly important when constructing nested mutable structures.

Consider:

```
1 rows = [[0, 0, 0]] * 3
2
3 rows
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

At first sight, this appears to create three independent rows.

However:

```
1 rows[0] is rows[1]
```

```
True
```

evaluates to True.

The expression

```
1 [[0, 0, 0]] * 3
```

repeats the **reference to the same inner list** three times.

Therefore:

```
1 rows = [[0, 0, 0]] * 3
2
3 rows[0][1] = 5
4
5 rows
```

```
[[0, 5, 0], [0, 5, 0], [0, 5, 0]]
```

produces

```
1 [[0, 5, 0],  
2  [0, 5, 0],  
3  [0, 5, 0]]
```

All three apparent rows changed because they are aliases for the same inner list.

A correct construction of independent rows is, for example:

```
1 rows = []  
2  
3 for _ in range(3):  
4     rows.append([0, 0, 0])  
5  
6 rows[0][1] = 5  
7  
8 rows
```

```
[[0, 5, 0], [0, 0, 0], [0, 0, 0]]
```

which produces

```
1 [[0, 5, 0],  
2  [0, 0, 0],  
3  [0, 0, 0]]
```

Later, list comprehensions will provide a more concise way to express the same construction:

```
1 rows = [[0, 0, 0] for _ in range(3)]
```

The crucial difference is that the inner list expression is evaluated separately for each iteration, producing distinct list objects.

3.25 Iterating over Nested Lists

Nested structures can be traversed using nested loops:

```

1 data = [
2     [1, 2, 3],
3     [4, 5, 6],
4 ]
5
6 for row in data:
7     for value in row:
8         print(value)

```

```

1
2
3
4
5
6

```

The outer loop traverses the rows, while the inner loop traverses the elements within each row.

When the position is required, `enumerate()` can be used at either level:

```

1 data = [
2     [1, 2, 3],
3     [4, 5, 6],
4 ]
5
6 for i, row in enumerate(data):
7     for j, value in enumerate(row):
8         print(i, j, value)

```

```

0 0 1
0 1 2
0 2 3
1 0 4
1 1 5
1 2 6

```

The pair `(i, j)` identifies the nested position of each value.

Again, this resembles matrix indexing conceptually, but it remains ordinary nested-list indexing rather than specialised multidimensional array indexing.

3.26 Structural Mutation during Iteration

Because lists are mutable, it is possible to add or remove elements while iterating over them. This should be done carefully.

Consider:

```
1 values = [1, 2, 3, 4, 5, 6]
2
3 for value in values:
4     if value % 2 == 0:
5         values.remove(value)
6
7 values
```

[1, 3, 5]

This particular example may appear to work, but modifying the structure of the iterable while traversing it can change the positions of elements that Python has not yet visited.

A more problematic example is:

```
1 values = [1, 2, 2, 2, 3]
2
3 for value in values:
4     if value == 2:
5         values.remove(value)
6
7 values
```

[1, 2, 3]

Because removal shifts later elements to the left, some values may be skipped.

A safer approach is often to construct a new list:

```
1 values = [1, 2, 2, 2, 3]
2
3 filtered = []
4
5 for value in values:
6     if value != 2:
```

```
7     filtered.append(value)
8
9 filtered
```

```
[1, 3]
```

List comprehensions will later provide a concise representation of this filtering pattern.

3.27 List Comprehensions

Many list-processing tasks follow a simple pattern:

1. iterate over an iterable;
2. compute a value for each element;
3. append the result to a new list.

For example:

```
1 values = [1, 2, 3, 4, 5]
2
3 squares = []
4
5 for value in values:
6     squares.append(value ** 2)
7
8 squares
```

```
[1, 4, 9, 16, 25]
```

Python provides **list comprehensions** as a concise syntax for this pattern.

The previous loop can be written as

```
1 values = [1, 2, 3, 4, 5]
2
3 squares = [value ** 2 for value in values]
4
5 squares
```

```
[1, 4, 9, 16, 25]
```


The general form is

```
1 [expression for variable in iterable]
```

Conceptually, Python evaluates expression once for every value produced by the iterable and collects the results into a new list.

For the previous example,

$$x \mapsto x^2,$$

so

$$[1, 2, 3, 4, 5] \mapsto [1, 4, 9, 16, 25].$$

A list comprehension creates a **new list**; it does not modify the original iterable.

3.28 Filtering with List Comprehensions

A condition may be added to select which input elements contribute to the resulting list.

The general form is

```
1 [expression for variable in iterable if condition]
```

For example:

```
1 values = [-3, 5, -1, 7, 0, 4]
2
3 non_negative = [value for value in values if value >= 0]
4
5 non_negative
```

[5, 7, 0, 4]

This is equivalent to

```

1 non_negative = []
2
3 for value in values:
4     if value >= 0:
5         non_negative.append(value)

```

The comprehension contains two conceptually separate operations:

- if `value >= 0` determines **which elements are retained**;
- `value` determines **what is stored in the output list**.

These two roles can differ.

For example:

```

1 values = [-3, 5, -1, 7, 0, 4]
2
3 squares = [value ** 2 for value in values if value >= 0]
4
5 squares

```

```
[25, 49, 0, 16]
```

first filters the non-negative values and then squares those that remain.

Mathematically, if

$$X = \{x_1, \dots, x_n\},$$

the comprehension represents a transformation applied only to elements satisfying a predicate P :

$$[f(x) \mid x \in X, P(x)].$$

3.29 Conditional Expressions inside Comprehensions

A conditional expression can also appear in the expression part of a list comprehension:

```

1 values = [-2, 4, -1, 6]
2
3 labels = ["negative" if value < 0 else "non-negative" for value in values]
4
5 labels

```

['negative', 'non-negative', 'negative', 'non-negative']

This is different from filtering.

Compare

```

1 [value for value in values if value >= 0]

```

with

```

1 [value if value >= 0 else 0 for value in values]

```

The first expression **removes** negative values:

```

1 values = [-2, 4, -1, 6]
2
3 [value for value in values if value >= 0]

```

[4, 6]

The second preserves every position but **transforms** negative values:

```

1 [value if value >= 0 else 0 for value in values]

```

[0, 4, 0, 6]

This distinction is important:

```

1 ... for x in iterable if condition

```

is a **filter**, whereas

```
1 value_if_true if condition else value_if_false
```

is a **conditional expression producing a value**.

3.30 Nested List Comprehensions

List comprehensions can be nested.

For example, a list containing three independent rows can be created as

```
1 rows = [[0, 0, 0] for _ in range(3)]
2
3 rows
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

This construction avoids the aliasing problem caused by

```
1 [[0, 0, 0]] * 3
```

because the inner list expression is evaluated separately at every iteration.

A nested comprehension may also represent a nested loop.

For example:

```
1 pairs = [(i, j) for i in range(3) for j in range(2)]
2
3 pairs
```

```
[(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

is equivalent to

```
1 pairs = []
2
3 for i in range(3):
4     for j in range(2):
5         pairs.append((i, j))
```

The order of the for clauses in the comprehension follows the same order as the corresponding nested loops.

However, nested comprehensions can quickly become difficult to read. For example, a compact expression containing several nested loops and conditions may save lines of code while making the computational structure harder to understand.

A list comprehension is most useful when the transformation is simple and the intent remains immediately visible.

3.31 Comprehensions versus Explicit Loops

List comprehensions are particularly appropriate for simple **map-like** and **filter-like** transformations.

For example:

```
1 temperatures = [12.4, 15.1, 18.7, 21.3]
2
3 rounded = [round(value) for value in temperatures]
4
5 rounded
```

[12, 15, 19, 21]

or

```
1 scores = [0.43, 0.78, 0.91, 0.54, 0.88]
2
3 high_scores = [score for score in scores if score >= 0.8]
4
5 high_scores
```

[0.91, 0.88]

An explicit loop is often preferable when the body

- performs several operations;
- contains multiple branches;
- modifies external state;
- uses break or continue;
- needs detailed logging or error handling;

- would make the comprehension difficult to understand.

For example:

```
1 results = []
2
3 for value in values:
4     if value is None:
5         continue
6
7     transformed = preprocess(value)
8     validated = validate(transformed)
9
10    if validated:
11        results.append(transformed)
```

should generally not be compressed merely to reduce its number of lines.

Conciseness is useful when it preserves clarity, not when it hides the control flow.

3.32 Building Lists through Iteration

Before comprehensions are used automatically, it is useful to recognise several common explicit-loop patterns.

3.32.1 Transformation

A transformation creates one output value for each input value:

```
1 values = [1, 2, 3, 4]
2
3 transformed = []
4
5 for value in values:
6     transformed.append(2 * value)
7
8 transformed
```

[2, 4, 6, 8]

which can be expressed as

```
1 [2 * value for value in values]
```

[2, 4, 6, 8]

3.32.2 Filtering

A filtering operation retains only values satisfying a condition:

```
1 values = [-2, 4, -1, 7, 0]
2
3 filtered = []
4
5 for value in values:
6     if value >= 0:
7         filtered.append(value)
8
9 filtered
```

[4, 7, 0]

equivalent to

```
1 [value for value in values if value >= 0]
```

[4, 7, 0]

3.32.3 Transformation and Filtering

Both operations can be combined:

```
1 values = [-2, 4, -1, 7, 0]
2
3 squares = []
4
5 for value in values:
6     if value >= 0:
7         squares.append(value ** 2)
8
9 squares
```

```
[16, 49, 0]
```

which becomes

```
1 [value ** 2 for value in values if value >= 0]
```

```
[16, 49, 0]
```

Recognising these patterns makes it easier to decide whether an explicit loop or comprehension best represents the computation.

3.33 Lists as Dynamic Collections

Python lists can grow and shrink dynamically. This makes them useful when the number of elements is not known before execution.

For example:

```
1 measurements = []
2
3 for value in [0.41, 0.63, 0.72, 0.88]:
4     if value >= 0.5:
5         measurements.append(value)
6
7 measurements
```

```
[0.63, 0.72, 0.88]
```

The final size of measurements depends on the data rather than being fixed in advance.

This dynamic behaviour is convenient for general-purpose programming. It differs from many numerical array structures, where shape and data type play a more central role in the representation and computational model.

3.34 Some Computational Considerations

Lists provide convenient general-purpose sequence operations, but not all list operations have the same computational characteristics.

A detailed study of algorithmic complexity belongs elsewhere, but several qualitative distinctions are useful.

3.34.1 Direct Indexing

Retrieving an element by position,

```
1 values[i]
```

uses a known index and does not require searching through preceding values conceptually from the user's point of view.

This makes positional access fundamentally different from searching for a value.

3.34.2 Membership and Search

An expression such as

```
1 target in values
```

may need to inspect elements sequentially until

- the target is found, or
- the list is exhausted.

Similarly,

```
1 values.index(target)
```

searches for the first matching value.

For a list containing n elements, a search may therefore require examining a substantial fraction of those n elements.

This becomes important when the same membership test is performed repeatedly on large collections. Other data structures, such as sets and dictionaries, are designed for different access patterns and will be introduced separately.

3.34.3 Appending versus Inserting

Appending an element to the end of a list is a natural operation:

```
1 values.append(x)
```

By contrast, inserting near the beginning

```
1 values.insert(0, x)
```

requires existing elements to change their positions.

Likewise, removing an element from the middle of a list changes the positions of subsequent elements.

This is one reason lists are particularly natural for sequential collections and accumulation at the end, but are not equally well suited to every possible insertion or lookup pattern.

3.35 Lists and Numerical Computing

A list can contain numerical values:

```
1 values = [1.2, 3.4, 5.6]
2
3 values
```

```
[1.2, 3.4, 5.6]
```

but Python lists do not provide element-wise numerical semantics.

For example:

```
1 values * 2
```

```
[1.2, 3.4, 5.6, 1.2, 3.4, 5.6]
```

repeats the list rather than multiplying every element by two.

The result is

```
1 [1.2, 3.4, 5.6, 1.2, 3.4, 5.6]
```

Similarly:

```
1 [1, 2] + [3, 4]
```

produces

```
1 [1, 2, 3, 4]
```

rather than

```
1 [4, 6]
```

These operations follow **sequence semantics**, not vector or matrix semantics.

Element-wise numerical processing can be written explicitly:

```
1 values = [1.2, 3.4, 5.6]
2
3 scaled = [2 * value for value in values]
4
5 scaled
```

```
[2.4, 6.8, 11.2]
```

but numerical Python libraries provide specialised data structures and operations designed for this purpose.

Therefore, lists should be understood first as general-purpose Python sequences. Their ability to store numbers does not make them numerical arrays.

3.36 Choosing Appropriate List Operations

Many list tasks can be expressed in several ways. The preferred form should make the intention of the computation clear.

For example, when adding one object:

```
1 values.append(x)
```

is clearer than

```
1 values.extend([x])
```

When adding the elements of another iterable:

```
1 values.extend(other)
```

communicates the intention better than repeatedly calling `append()`.

When a removed element is needed:

```
1 value = values.pop()
```

is preferable to separately retrieving and deleting it.

When the original ordering must remain unchanged:

```
1 ordered = sorted(values)
```

is preferable to sorting the original list and then trying to reconstruct its previous state.

Similarly, when a simple transformation naturally produces a new list,

```
1 [x ** 2 for x in values]
```

may express the computation more directly than an explicit accumulator loop.

The appropriate operation should reflect the intended semantics, particularly whether the original list is meant to change.

4 Problems — Lists

The following problems focus on list semantics rather than on memorising individual methods. In particular, pay attention to mutability, object identity, copying, slicing, structural changes, and the distinction between operations that modify a list and operations that create a new one.

Where possible, predict the result before executing the code.

4.1 Problem 1 — Indexing

Given

```
1 values = [4, 7, 1, 9, 3]
```

determine the result of

```
1 values[0]  
2 values[3]  
3 values[-1]  
4 values[-3]
```

without executing the code.

For each expression, state the corresponding non-negative index.

4.2 Problem 2 — Index Bounds

Given

```
1 values = [10, 20, 30]
```

what happens when evaluating

```
1 values[3]
```

and

```
1 values[-4]
```

?

Explain why both accesses are invalid.

4.3 Problem 3 — Basic Slicing

Given

```
1 values = [10, 20, 30, 40, 50, 60]
```

predict:

```
1 values[1:4]
2 values[:3]
3 values[3:]
4 values[:]
```

Explain why the element at the stopping index is excluded.

4.4 Problem 4 — Slice Steps

Predict the result of:

```
1 values = list(range(10))
2
3 values[::2]
4 values[1::2]
5 values[2:8:2]
6 values[::-1]
7 values[8:2:-2]
```

For each expression, identify the selected indices.

4.5 Problem 5 — Indexing versus Slicing

Given

```
1 values = [1, 2, 3, 4]
```

compare the behaviour of

```
1 values[20]
```

with

```
1 values[1:20]
```

Why does one operation raise an exception while the other does not?

4.6 Problem 6 — Indexed Assignment

Starting from

```
1 values = [10, 20, 30, 40]
```

change the list to

```
1 [10, 25, 30, 50]
```

using indexed assignment only.

Do not create a new list.

4.7 Problem 7 — Slice Assignment

Predict the final list:

```
1 values = [1, 2, 3, 4, 5]
2
3 values[1:4] = [10, 20]
```

Explain why the length changes.

Then predict:

```
1 values[1:2] = [7, 8, 9]
```

using the result of the previous operation.

4.8 Problem 8 — `append()` versus `extend()`

Without executing the code, determine the final values of a and b:

```
1 a = [1, 2]
2 b = [1, 2]
3
4 a.append([3, 4])
5 b.extend([3, 4])
```

Explain the structural difference between the two resulting lists.

4.9 Problem 9 — Extending with a String

Predict:

```
1 values = [1, 2]
2
3 values.extend("abc")
```

Why are three elements added rather than one?

How would the result differ if `append()` were used instead?

4.10 Problem 10 — `insert()` versus Assignment

Given

```
1 values = ["a", "b", "c"]
```

predict the difference between

```
1 values[1] = "x"
```

and, starting again from the original list,

```
1 values.insert(1, "x")
```

Compare the resulting list lengths.

4.11 Problem 11 — Removing by Value and Position

Given

```
1 values = [4, 2, 7, 2, 9]
```

determine the result of

```
1 values.remove(2)
```

Which occurrence is removed?

Then explain how you would remove the element currently located at index 2, regardless of its value.

4.12 Problem 12 — pop()

Predict the values of removed and values:

```
1 values = [10, 20, 30, 40]
2
3 removed = values.pop(1)
```

Then determine what would happen with

```
1 values.pop()
```

after the first operation.

4.13 Problem 13 — del

Starting from

```
1 values = [0, 1, 2, 3, 4, 5, 6]
```

use del to remove

1. the element at index 2;
2. the slice containing the elements from the new indices 2 through 4.

Predict the final list.

4.14 Problem 14 — `sort()` versus `sorted()`

Predict the final state:

```
1 values = [4, 1, 3, 2]
2
3 result = values.sort()
```

What are the values of `values` and `result`?

Then rewrite the operation so that the original list remains unchanged and a sorted list is stored in `result`.

4.15 Problem 15 — Sorting by a Derived Criterion

Given

```
1 models = [
2     ["linear", 0.82],
3     ["forest", 0.91],
4     ["tree", 0.78],
5 ]
```

sort the records from highest score to lowest score using the second element of each inner list as the sorting key.

Do not manually rearrange the elements.

4.16 Problem 16 — Equality and Identity

Predict:

```
1 a = [1, 2, 3]
2 b = [1, 2, 3]
3 c = a
4
5 a == b
6 a is b
7 a == c
8 a is c
```

Explain every result.

4.17 Problem 17 — Aliasing

Without executing the code, determine the final values of a and b:

```
1 a = [1, 2, 3]
2 b = a
3
4 b.append(4)
5 a[0] = 100
```

How many list objects are involved?

4.18 Problem 18 — Rebinding versus Mutation

Consider:

```
1 a = [1, 2, 3]
2 b = a
3
4 b = [4, 5, 6]
```

What are the final values of a and b?

Compare this with:

```
1 a = [1, 2, 3]
2 b = a
3
4 b[:] = [4, 5, 6]
```

Explain why the results differ.

4.19 Problem 19 — Shallow Copy

Predict:

```
1 a = [1, 2, 3]
2 b = a.copy()
3
4 b.append(4)
```

What are a and b?

What are the values of

```
1 a == b
2 a is b
```

after the modification?

4.20 Problem 20 — Shallow Copy with Nested Lists

Consider:

```
1 a = [[1, 2], [3, 4]]
2 b = a.copy()
3
4 b[0].append(5)
```

Predict both lists.

Then predict the values of

```
1 a is b
2 a[0] is b[0]
```

Explain the apparent contradiction between the two identity tests.

4.21 Problem 21 — Replacement versus Inner Mutation

Starting independently from

```
1 a = [[1, 2], [3, 4]]
2 b = a.copy()
```

compare the effects of

```
1 b[0].append(99)
```

and

```
1 b[0] = [99]
```

on a.

Explain the difference in terms of references.

4.22 Problem 22 — Deep Copy

Use `copy.deepcopy()` to create an independent copy of

```
1 original = [[1, 2], [3, 4]]
```

Modify one of the inner lists of the copy and verify that the original remains unchanged.

Then compare

```
1 original[0] is copied[0]
```

with the corresponding result from a shallow copy.

4.23 Problem 23 — Repeated Nested Lists

Predict the output:

```
1 rows = [[0, 0]] * 3
2
3 rows[0][1] = 5
4
5 print(rows)
```

Explain why more than one apparent row changes.

Then construct three independent rows correctly.

4.24 Problem 24 — Concatenation and Mutation

Predict the result:

```
1 a = [1, 2]
2 b = a
3 c = a + [3]
4
5 c.append(4)
```

What are the final values of a, b, and c?

Which variables refer to the same object?

4.25 Problem 25 — += and Aliasing

Predict:

```
1 a = [1, 2]
2 b = a
3
4 a += [3, 4]
```

What are a and b?

Then compare this with:

```
1 a = [1, 2]
2 b = a
3
4 a = a + [3, 4]
```

Explain why these two forms can behave differently when aliases exist.

4.26 Problem 26 — Membership and Counting

Given

```
1 features = ["age", "income", "balance", "age", "country"]
```

determine

```
1 "age" in features
2 "gender" in features
3 features.count("age")
4 features.index("age")
```

Explain why `count()` and `index()` answer different questions.

4.27 Problem 27 — Constructing a List

Given

```
1 values = [3, -1, 5, -4, 7, 0]
```

construct a new list containing only the non-negative values using

1. an explicit for loop;
2. a list comprehension.

The original list must remain unchanged.

4.28 Problem 28 — Transformation

Using an explicit loop, construct a list containing the squares of

```
1 values = [1, 2, 3, 4, 5]
```

Then rewrite the solution using a list comprehension.

Verify that both approaches produce equal lists.

4.29 Problem 29 — Filtering and Transformation

Given

```
1 values = [-4, -2, 0, 3, 5]
```

create

```
1 [0, 9, 25]
```

using a single list comprehension.

Explain separately

- the filtering condition;
- the transformation expression.

4.30 Problem 30 — Conditional Expression in a Comprehension

Given

```
1 scores = [0.32, 0.61, 0.48, 0.84, 0.73]
2 threshold = 0.5
```

construct

```
1 [0, 1, 0, 1, 1]
```

using a list comprehension with a conditional expression.

Explain why this is not a filtering comprehension.

4.31 Problem 31 — Independent Nested Lists

Create a 4×3 nested list initially filled with zeros so that all rows are independent objects.

Do **not** use

```
1 [[0, 0, 0]] * 4
```

Then modify one element in the first row and verify that no other row changes.

4.32 Problem 32 — Nested Traversal

Given


```
1 data = [  
2     [2, 4, 6],  
3     [1, 3, 5],  
4     [7, 8, 9],  
5 ]
```

use nested loops to

1. print every element;
2. calculate the total sum;
3. calculate the sum of each row.

Do not use `sum()`.

4.33 Problem 33 — Flattening a Nested List

Given

```
1 data = [  
2     [1, 2],  
3     [3, 4],  
4     [5, 6],  
5 ]
```

construct

```
1 [1, 2, 3, 4, 5, 6]
```

first using nested loops and then using a nested list comprehension.

4.34 Problem 34 — Structural Mutation during Iteration

Predict the result:

```
1 values = [1, 2, 2, 2, 3]  
2  
3 for value in values:  
4     if value == 2:  
5         values.remove(value)
```

Why might the final list still contain a 2?

Rewrite the operation so that every 2 is removed without structurally modifying the list being traversed.

4.35 Problem 35 — List Repetition Is Not Numerical Multiplication

Given

```
1 values = [1, 2, 3]
```

predict:

```
1 values * 2
```

Then construct

```
1 [2, 4, 6]
```

using a list comprehension.

Explain the difference between sequence repetition and element-wise numerical multiplication.

4.36 Problem 36 — Building Predictions

Suppose a model produces

```
1 probabilities = [0.18, 0.72, 0.51, 0.43, 0.89]
2 threshold = 0.5
```

construct the binary predictions

$$\hat{y}_i = \begin{cases} 1, & p_i \geq 0.5, \\ 0, & p_i < 0.5 \end{cases}$$

using

1. an explicit loop;
2. a list comprehension.

The two results should compare equal.

4.37 Problem 37 — Selecting Valid Measurements

Given

```
1 values = [0.42, None, -0.1, 0.71, None, 0.88, -0.3]
```

construct a new list containing only non-None, non-negative values.

First solve the problem with an explicit loop and then with a list comprehension.

Be careful about the order of the conditions.

4.38 Problem 38 — Top Scores

Given

```
1 results = [  
2     ["model_a", 0.82],  
3     ["model_b", 0.91],  
4     ["model_c", 0.77],  
5     ["model_d", 0.88],  
6 ]
```

create a new list containing only the model names whose scores are at least 0.85.

The expected result is

```
1 ["model_b", "model_d"]
```

Use a list comprehension.

4.39 Problem 39 — Copying a Configuration

Suppose

```
1 config = [  
2     ["learning_rate", 0.01],  
3     ["epochs", 100],  
4 ]
```

and you want to create a second configuration that can be modified independently.

Explain why

```
1 config_b = config
```

is not sufficient.

Then explain why

```
1 config_b = config.copy()
```

may still be insufficient if the inner lists are going to be modified.

Create an appropriately independent copy and demonstrate the difference.

4.40 Problem 40 — Reasoning about Objects

Without executing the code, determine the exact output:

```
1 a = [[1, 2], [3, 4]]
2 b = a
3 c = a.copy()
4 d = a[:]
5
6 b[0].append(5)
7 c.append([6, 7])
8 d[1][0] = 99
9
10 print(a)
11 print(b)
12 print(c)
13 print(d)
14
15 print(a is b)
16 print(a is c)
17 print(a[0] is c[0])
18 print(a[1] is d[1])
```

Trace the object relationships carefully before predicting the result.

5 Summary

A Python list is an ordered, mutable sequence of object references. This combination of ordering and mutability makes lists flexible general-purpose containers, but it also means that their behaviour cannot be understood solely in terms of the values displayed on the screen.

Indexing retrieves individual elements, while slicing selects ranges and normally creates new outer lists. Slice assignment, indexed assignment, and methods such as `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `reverse()`, and `sort()` modify list state in different ways.

An important distinction is whether an operation acts **in place** or creates a **new object**. Concatenation, slicing, `sorted()`, and list comprehensions generally create new lists, whereas many list methods modify the existing list.

Because variables are references to objects, assignment does not copy a list:

```
1 b = a
```

creates an alias. Both names then refer to the same mutable object. Shallow copies create independent outer lists but continue to share references to nested objects. Deep copying can recursively duplicate nested mutable structures when full independence is genuinely required.

Nested lists provide hierarchical structure but remain ordinary Python lists. In particular, a list of lists may resemble a mathematical matrix without acquiring matrix arithmetic semantics. Repetition, concatenation, indexing, and mutation still follow Python sequence rules.

List comprehensions provide a concise way to construct new lists from transformations and filters:

```
1 [f(x) for x in iterable]
```

and

```
1 [f(x) for x in iterable if condition]
```

They are particularly effective when the operation is simple and declarative. Explicit loops remain preferable when the control flow itself is substantial.

For data science and machine learning work, lists are frequently used as containers for metadata, configuration values, labels, feature names, intermediate objects, and dynamically collected results. Numerical operations over large homogeneous data are usually better represented by specialised structures such as NumPy arrays.

The central conceptual point is therefore not merely how to create and modify a list, but how **mutability, references, copying, and sequence semantics interact**. Those ideas recur throughout Python and become increasingly important as data structures and program state become more complex.

The general principle is that structural modification of a list during iteration should be deliberate and justified, rather than assumed to behave like iteration over a fixed snapshot.

6 Data Structures — Dictionaries

A **dictionary** is one of Python's fundamental built-in data structures. It represents a collection of associations between **keys** and **values**.

A dictionary can therefore be viewed conceptually as a mapping

$$k \mapsto v,$$

where each key k identifies a corresponding value v .

For example:

```
1 model = {  
2     "name": "linear regression",  
3     "features": 12,  
4     "score": 0.87,  
5 }  
6  
7 model
```

```
{'name': 'linear regression', 'features': 12, 'score': 0.87}
```

The dictionary contains three key-value associations:

```
1 "name"  -> "linear regression"  
2 "features" -> 12  
3 "score"  -> 0.87
```

Unlike a list, where elements are primarily identified by their positional indices, dictionary values are accessed through **keys**.

This difference reflects two distinct data models:

```
1 list    position -> value
2 dictionary key    -> value
```

Dictionaries are particularly useful when values have meaningful identifiers. In data science and machine-learning code, they frequently represent configurations, parameter sets, metadata, records, mappings between labels and values, evaluation metrics, grouped results, and lookup tables.

Python dictionaries have several important properties:

- they store **key-value pairs**;
- each key is **unique** within a dictionary;
- values can be arbitrary Python objects;
- keys must satisfy specific requirements related to **hashability**;
- dictionaries are **mutable**;
- modern Python dictionaries preserve **insertion order**.

The central idea is that a dictionary is a **mapping**, not a positional sequence.

6.1 Creating Dictionaries

The most common dictionary syntax uses curly braces `{}` containing comma-separated key-value pairs:

```
1 {
2     key_1: value_1,
3     key_2: value_2,
4     ...
5 }
```

For example:

```
1 person = {
2     "name": "Emma",
3     "age": 31,
4     "height": 1.68,
5 }
6
7 person
```

```
{'name': 'Emma', 'age': 31, 'height': 1.68}
```

The colon : separates each key from its corresponding value.

The dictionary contains three entries:

Key	Value
"name"	"Emma"
"age"	31
"height"	1.68

Table 6: Example of key-value associations in a Python dictionary.

The type of the object is dict:

```
1 type(person)
```

```
dict
```

6.1.1 Empty Dictionaries

An empty dictionary can be created using

```
1 empty = {}
```

```
2
```

```
3 empty
```

```
{}
```

or with the dict() constructor:

```
1 empty = dict()
```

```
2
```

```
3 empty
```

```
{}
```

Its length is zero:


```
1 len(empty)
```

```
0
```

Like other empty containers, an empty dictionary is false in a Boolean context:

```
1 parameters = {}  
2  
3 if not parameters:  
4     print("No parameters are defined")
```

```
No parameters are defined
```

This allows emptiness to be tested directly without explicitly comparing `len(parameters)` with zero.

6.2 The `dict()` Constructor

The built-in `dict()` constructor provides several additional ways to create dictionaries.

For example, when keys are valid Python identifiers, keyword arguments can be used:

```
1 model = dict(  
2     name="linear regression",  
3     features=12,  
4     score=0.87,  
5 )  
6  
7 model
```

```
{'name': 'linear regression', 'features': 12, 'score': 0.87}
```

This is equivalent in value to

```
1 {  
2     "name": "linear regression",  
3     "features": 12,  
4     "score": 0.87,  
5 }
```

The `dict()` constructor can also create a dictionary from an iterable of key-value pairs:

```
1 pairs = [  
2     ("name", "linear regression"),  
3     ("features", 12),  
4     ("score", 0.87),  
5 ]  
6  
7 model = dict(pairs)  
8  
9 model
```

```
{'name': 'linear regression', 'features': 12, 'score': 0.87}
```

Each element of `pairs` contains exactly two components:

$$(k_i, v_i),$$

which become a key-value association in the resulting dictionary.

This representation is useful when key-value pairs are generated dynamically or obtained from another computation.

6.3 Length of a Dictionary

The built-in `len()` function returns the number of key-value pairs in a dictionary:

```
1 model = {  
2     "name": "linear regression",  
3     "features": 12,  
4     "score": 0.87,  
5 }  
6  
7 len(model)
```

```
3
```

The result is 3.

It is important to interpret dictionary length correctly. It counts the number of **entries**, not the total number of Python objects contained within the values.

For example:

```
1 data = {  
2     "features": ["age", "income", "balance"],  
3     "target": "default",  
4 }  
5  
6 len(data)
```

2

returns 2, even though the value associated with "features" is itself a list containing three elements.

The dictionary has only two top-level key-value associations.

6.4 Accessing Dictionary Values

Dictionary values can be retrieved using square brackets with a key:

```
1 dictionary[key]
```

For example:

```
1 model = {  
2     "name": "linear regression",  
3     "features": 12,  
4     "score": 0.87,  
5 }  
6  
7 model["name"]
```

'linear regression'

returns

```
1 'linear regression'
```

and

```
1 model["score"]
```

0.87

returns

```
1 0.87
```

The expression inside the brackets is therefore not a positional index. It is a **key lookup**.

This is fundamentally different from list indexing.

For a list:

```
1 values[0]
```

means

retrieve the value stored at position 0.

For a dictionary:

```
1 values["score"]
```

means

retrieve the value associated with the key "score".

6.5 Missing Keys and KeyError

Accessing a key that does not exist raises a `KeyError`.

For example:

```
1 model["algorithm"]
```

when "algorithm" is not a key in model produces an exception similar to

```
1 KeyError: 'algorithm'
```

This behaviour is useful when the absence of the requested key represents an invalid program state.

However, dictionaries also provide other forms of lookup when missing keys are expected and should be handled explicitly. The `get()` method will be introduced later for this purpose.

6.6 Membership Tests

The `in` and `not in` operators test **dictionary keys**.

Consider:

```
1 model = {  
2     "name": "linear regression",  
3     "features": 12,  
4     "score": 0.87,  
5 }  
6  
7 "name" in model
```

True

returns True.

By contrast:

```
1 "linear regression" in model
```

False

returns False.

Although "linear regression" is present as a **value**, it is not a key.

Thus,

```
1 x in dictionary
```

is conceptually equivalent to asking

is `x` one of the dictionary's keys?

This distinction is important because dictionaries are organised around key-based lookup.

Membership is often used before direct access:

```

1 model = {
2     "name": "linear regression",
3     "score": 0.87,
4 }
5
6 if "score" in model:
7     print(model["score"])

```

0.87

Whether this is preferable to other lookup patterns depends on what should happen when the key is absent.

6.7 Keys Are Unique

A dictionary cannot contain multiple independent entries with the same key.

Consider:

```

1 data = {
2     "score": 0.81,
3     "score": 0.89,
4 }
5
6 data

```

{'score': 0.89}

The resulting dictionary contains only one "score" entry:

```

1 {'score': 0.89}

```

When the same key occurs more than once during dictionary construction, the later association replaces the earlier value.

Conceptually, a mapping associates each key with at most one value:

$$k \mapsto v.$$

It cannot simultaneously contain

$$k \mapsto v_1$$

and

$$k \mapsto v_2$$

as separate entries for the same key.

This does not prevent different keys from sharing equal values:

```
1 scores = {  
2     "model_a": 0.87,  
3     "model_b": 0.87,  
4 }  
5  
6 scores
```

```
{'model_a': 0.87, 'model_b': 0.87}
```

Values need not be unique.

6.8 Dictionary Keys

Dictionary keys do not need to be strings.

For example:

```
1 mapping = {  
2     1: "one",  
3     2: "two",  
4     3: "three",  
5 }  
6  
7 mapping[2]
```

```
'two'
```

returns "two".

Different immutable types may also be used as keys:

```
1 example = {  
2     "name": "model",  
3     1: "first",  
4     (2, 3): "pair",  
5 }  
6  
7 example
```

```
{'name': 'model', 1: 'first', (2, 3): 'pair'}
```

However, not every Python object can be used as a dictionary key.

The key requirement is **hashability**.

6.9 Hashability

Dictionary lookup is based on hashing. A dictionary key must therefore be a **hashable object**.

At a high level, a hashable object has

1. a hash value that remains stable during its lifetime;
2. an equality relation compatible with that hash value.

The built-in `hash()` function illustrates this concept:

```
1 hash("model")
```

```
4475305162039075220
```

```
1 hash(42)
```

```
42
```

Strings and integers are hashable, so they can be used as dictionary keys.

Tuples can also be hashable when all of their elements are hashable:


```
1 coordinates = {  
2     (0, 0): "origin",  
3     (1, 0): "point A",  
4     (0, 1): "point B",  
5 }  
6  
7 coordinates[(0, 0)]
```

'origin'

By contrast, lists are mutable and are not hashable:

```
1 hash([1, 2, 3])
```

raises

```
1 TypeError: unhashable type: 'list'
```

Consequently, a list cannot be used as a dictionary key:

```
1 {  
2     [1, 2]: "value"  
3 }
```

also raises a `TypeError`.

6.9.1 Why Mutability Matters for Keys

The restriction is closely related to dictionary lookup.

Suppose, conceptually, that a mutable object could be used as a key and that its contents were later changed. If the change altered the object's hash or equality behaviour, the dictionary would no longer have a stable way to locate the entry associated with that key.

Dictionary keys therefore require stable lookup semantics.

This explains why many common immutable objects are suitable keys, while mutable structures such as lists and dictionaries are not.

It is more precise to say

dictionary keys must be hashable
than simply

dictionary keys must be immutable.

Immutability is strongly related to hashability, but hashability is the actual requirement imposed by dictionaries.

6.10 Dictionary Values

Dictionary values have no corresponding hashability restriction.

They may be numbers:

```
1 metrics = {  
2     "accuracy": 0.91,  
3     "precision": 0.87,  
4     "recall": 0.84,  
5 }  
6  
7 metrics
```

```
{'accuracy': 0.91, 'precision': 0.87, 'recall': 0.84}
```

lists:

```
1 dataset = {  
2     "features": ["age", "income", "balance"],  
3     "target": "default",  
4 }  
5  
6 dataset
```

```
{'features': ['age', 'income', 'balance'], 'target': 'default'}
```

other dictionaries:

```

1 experiment = {
2     "model": {
3         "type": "random forest",
4         "trees": 200,
5     },
6     "metrics": {
7         "accuracy": 0.91,
8         "recall": 0.84,
9     },
10 }
11
12 experiment

```

```

{'model': {'type': 'random forest', 'trees': 200},
 'metrics': {'accuracy': 0.91, 'recall': 0.84}}

```

or arbitrary other Python objects.

This flexibility allows dictionaries to represent structured records and hierarchical configurations.

6.11 Dictionaries Are Mutable

Dictionaries are mutable. Existing key-value associations can be changed after creation.

For example:

```

1 model = {
2     "name": "linear regression",
3     "score": 0.82,
4 }
5
6 model["score"] = 0.87
7
8 model

```

```

{'name': 'linear regression', 'score': 0.87}

```

The value associated with "score" has been replaced.

A new key-value association can be added using the same syntax:

```
1 model["features"] = 12
2
3 model
```

```
{'name': 'linear regression', 'score': 0.87, 'features': 12}
```

The same expression therefore has two possible effects:

```
1 dictionary[key] = value
```

- if key already exists, its associated value is replaced;
- if key does not exist, a new entry is inserted.

For example:

```
1 model = {
2     "name": "linear regression",
3     "score": 0.82,
4 }
5
6 model["score"] = 0.87
7 model["features"] = 12
8
9 model
```

```
{'name': 'linear regression', 'score': 0.87, 'features': 12}
```

The operation on "score" performs an **update**, while the operation on "features" performs an **insertion**.

6.12 Insertion Order

Python dictionaries preserve the order in which keys are inserted.

For example:

```
1 record = {}
2
3 record["name"] = "model_a"
4 record["score"] = 0.91
5 record["features"] = 12
6
7 record
```

```
{'name': 'model_a', 'score': 0.91, 'features': 12}
```

iterating over this dictionary will encounter the keys in the order

```
1 "name"
2 "score"
3 "features"
```

unless the structure is subsequently modified in a way that changes which entries exist.

This property should not be confused with positional indexing.

A dictionary may preserve insertion order, but it remains a mapping. Expressions such as

```
1 record[0]
```

do **not** mean “the first dictionary element” unless 0 is actually a key.

Thus:

```
1 ordered
```

does not imply

```
1 positionally indexed
```

for dictionaries.

The ordering property is useful for predictable iteration and representation, while key-based lookup remains the defining access mechanism.

6.13 Dictionary Equality

Two dictionaries compare equal when they contain equal key-value associations, regardless of the order in which those entries were inserted.

For example:

```
1 a = {  
2     "name": "model",  
3     "score": 0.91,  
4 }  
5  
6 b = {  
7     "score": 0.91,  
8     "name": "model",  
9 }  
10  
11 a == b
```

True

evaluates to True.

This differs from lists, where order is part of equality:

```
1 [1, 2] == [2, 1]
```

False

is False.

For dictionaries, equality concerns the mapping between keys and values, not insertion order.

The dictionaries

$$\{k_1 : v_1, k_2 : v_2\}$$

and

$$\{k_2 : v_2, k_1 : v_1\}$$

represent the same mapping.

6.14 Dictionaries versus Lists

Lists and dictionaries are both mutable containers, but they model different kinds of relationships.

Property	List	Dictionary
Main abstraction	sequence	mapping
Access	positional index	key
Ordering	positional order	insertion order
Duplicate identifiers	indices are inherently distinct	keys must be unique
Membership with in	tests values	tests keys
Typical purpose	ordered collection	association / lookup

Table 7: Conceptual comparison between lists and dictionaries.

Consider:

```
1 record_list = ["model_a", 0.91, 12]
2
3 record_dict = {
4     "name": "model_a",
5     "score": 0.91,
6     "features": 12,
7 }
```

The list requires the programmer to know that

```
1 position 0 -> model name
2 position 1 -> score
3 position 2 -> number of features
```

The dictionary makes those meanings explicit:

```
1 record_dict["name"]
2 record_dict["score"]
3 record_dict["features"]
```

This does not mean dictionaries are universally preferable. The appropriate structure depends on the data model:

- use a sequence when **position and ordering** define the structure;
- use a mapping when values are naturally identified by **keys**.

Choosing between data structures is therefore a semantic decision rather than merely a syntactic preference.

6.15 Retrieving Values Safely

Direct key access using square brackets is appropriate when the requested key is expected to exist:

```
1 metrics = {  
2     "accuracy": 0.91,  
3     "precision": 0.87,  
4 }  
5  
6 metrics["accuracy"]
```

0.91

If the key is missing, however, Python raises a `KeyError`.

For example:

```
1 metrics["recall"]
```

would fail when "recall" is absent.

Sometimes this is desirable because a missing key represents an invalid program state. In other situations, missing keys are expected and should be handled without raising an exception.

The `get()` method provides this second form of lookup.

6.16 The `get()` Method

The general form is

```
1 dictionary.get(key)
```

If the key exists, the corresponding value is returned:


```
1 metrics = {  
2     "accuracy": 0.91,  
3     "precision": 0.87,  
4 }  
5  
6 metrics.get("accuracy")
```

0.91

If the key does not exist, `get()` returns `None` by default:

```
1 metrics.get("recall")
```

A different default value can be specified:

```
1 metrics.get("recall", 0.0)
```

0.0

which returns 0.0 when "recall" is absent.

The distinction is therefore:

```
1 dictionary[key]
```

means

retrieve this key; its absence is an error,

whereas

```
1 dictionary.get(key, default)
```

means

retrieve this key if present; otherwise use a fallback value.

The appropriate choice depends on the semantics of the data rather than on which syntax is shorter.

For example, if every trained model is required to contain an "accuracy" entry, direct access may be preferable because a missing key should reveal a problem:

```
1 accuracy = metrics["accuracy"]
```

If "description" is genuinely optional, then

```
1 description = metadata.get("description")
```

may better represent the intended logic.

6.16.1 Distinguishing a Missing Key from a Stored None

A subtle issue arises when None is itself a legitimate dictionary value.

Consider:

```
1 record = {  
2     "name": "model_a",  
3     "comment": None,  
4 }  
5  
6 record.get("comment")
```

and

```
1 record.get("owner")
```

Both expressions return None.

Therefore, the return value of get() alone cannot distinguish

- a key that exists and is associated with None;
- a key that does not exist.

When that distinction matters, membership should be tested explicitly:

```
1 if "comment" in record:  
2     print("The key exists")
```

The key exists

This illustrates why lookup behaviour should be chosen according to the information the program needs to preserve.

6.17 Dictionary Views

Python dictionaries provide three fundamental methods for inspecting their contents:

```
1 dictionary.keys()
2 dictionary.values()
3 dictionary.items()
```

These correspond to three views of the mapping:

```
1 keys    -> identifiers
2 values  -> associated objects
3 items   -> key-value pairs
```

For example:

```
1 metrics = {
2     "accuracy": 0.91,
3     "precision": 0.87,
4     "recall": 0.84,
5 }
6
7 metrics.keys()
```

```
dict_keys(['accuracy', 'precision', 'recall'])
```

```
1 metrics.values()
```

```
dict_values([0.91, 0.87, 0.84])
```

```
1 metrics.items()
```

```
dict_items([('accuracy', 0.91), ('precision', 0.87), ('recall', 0.84)])
```

Conceptually,

```
1 metrics.keys()
```

represents

```
1 "accuracy", "precision", "recall"
```

while

```
1 metrics.values()
```

represents

```
1 0.91, 0.87, 0.84
```

and

```
1 metrics.items()
```

represents the pairs

```
1 ("accuracy", 0.91)
2 ("precision", 0.87)
3 ("recall", 0.84)
```

6.18 Views Are Not Lists

The objects returned by `keys()`, `values()`, and `items()` are **dictionary view objects**, not lists.

For example:

```
1 metrics = {
2     "accuracy": 0.91,
3     "precision": 0.87,
4 }
5
6 type(metrics.keys())
```

`dict_keys`

returns a dictionary-view type rather than list.

A concrete list can be created explicitly when necessary:

```
1 list(metrics.keys())
```

```
['accuracy', 'precision']
```

Similarly:

```
1 list(metrics.values())
```

```
[0.91, 0.87]
```

and

```
1 list(metrics.items())
```

```
[('accuracy', 0.91), ('precision', 0.87)]
```

produce ordinary lists.

In many cases, conversion is unnecessary because dictionary views can be iterated over directly.

6.19 Dictionary Views Are Dynamic

Dictionary views reflect changes to the underlying dictionary.

Consider:

```
1 metrics = {  
2     "accuracy": 0.91,  
3     "precision": 0.87,  
4 }  
5  
6 keys = metrics.keys()  
7  
8 list(keys)
```

```
['accuracy', 'precision']
```

Now modify the dictionary:

```
1 metrics["recall"] = 0.84
2
3 list(keys)
```

```
['accuracy', 'precision', 'recall']
```

The existing view now includes "recall".

This happens because `keys` is not a detached copy of the dictionary's keys. It is a dynamic view of the underlying mapping.

This differs from

```
1 keys = list(metrics.keys())
```

which constructs an independent list containing the keys available at that moment.

6.20 Iterating over Dictionaries

Iterating directly over a dictionary traverses its **keys**.

For example:

```
1 metrics = {
2     "accuracy": 0.91,
3     "precision": 0.87,
4     "recall": 0.84,
5 }
6
7 for metric in metrics:
8     print(metric)
```

```
accuracy
precision
recall
```

is equivalent in effect to

```
1 for metric in metrics.keys():
2     print(metric)
```

This behaviour follows naturally from the dictionary's key-based mapping semantics.

6.21 Iterating over Values

If only the values are required, use `values()`:

```
1 metrics = {  
2     "accuracy": 0.91,  
3     "precision": 0.87,  
4     "recall": 0.84,  
5 }  
6  
7 for value in metrics.values():  
8     print(value)
```

```
0.91  
0.87  
0.84
```

The iteration variable successively receives

0.91, 0.87, 0.84.

6.22 Iterating over Key-Value Pairs

Frequently, both the key and the corresponding value are needed.

The `items()` method provides each association as a pair:

```
1 metrics = {  
2     "accuracy": 0.91,  
3     "precision": 0.87,  
4     "recall": 0.84,  
5 }  
6  
7 for metric, value in metrics.items():  
8     print(metric, value)
```

```
accuracy 0.91  
precision 0.87  
recall 0.84
```

The pair is unpacked directly into two variables.

Conceptually, the loop traverses

$$(k_1, v_1), \quad (k_2, v_2), \quad \dots$$

rather than iterating over keys and then performing a second lookup.

Compare:

```
1 for key in metrics:
2     value = metrics[key]
3     print(key, value)
```

with

```
1 for key, value in metrics.items():
2     print(key, value)
```

Both are valid, but the second form more directly expresses that the computation requires both components of each mapping entry.

6.23 Adding and Updating Entries

As introduced earlier, indexed assignment can both insert and update entries:

```
1 dictionary[key] = value
```

For example:

```
1 config = {
2     "learning_rate": 0.01,
3     "epochs": 100,
4 }
5
6 config["batch_size"] = 64
7 config["epochs"] = 150
8
9 config
```



```
{'learning_rate': 0.01, 'epochs': 150, 'batch_size': 64}
```

The first assignment inserts a new key. The second modifies the value associated with an existing key.

This behaviour makes dictionaries convenient for incrementally constructed records and configurations.

6.24 The `update()` Method

The `update()` method applies multiple key-value associations to an existing dictionary.

For example:

```
1 config = {  
2     "learning_rate": 0.01,  
3     "epochs": 100,  
4 }  
5  
6 config.update({  
7     "epochs": 150,  
8     "batch_size": 64,  
9 })  
10  
11 config
```

```
{'learning_rate': 0.01, 'epochs': 150, 'batch_size': 64}
```

The result is

```
1 {  
2     'learning_rate': 0.01,  
3     'epochs': 150,  
4     'batch_size': 64  
5 }
```

Existing keys are updated, while new keys are inserted.

The method therefore follows the same key-resolution rule as individual assignment:

existing key $\longrightarrow$ replace value

new key → insert entry.

`update()` can also receive keyword arguments when the keys are valid identifiers:

```
1 config = {  
2     "learning_rate": 0.01,  
3 }  
4  
5 config.update(  
6     epochs=100,  
7     batch_size=64,  
8 )  
9  
10 config
```

```
{'learning_rate': 0.01, 'epochs': 100, 'batch_size': 64}
```

6.25 Merging Dictionaries

Python also provides operators for combining mappings.

Consider:

```
1 defaults = {  
2     "learning_rate": 0.01,  
3     "epochs": 100,  
4 }  
5  
6 custom = {  
7     "epochs": 200,  
8     "batch_size": 64,  
9 }
```

The merge operator `|` creates a new dictionary:

```
1 config = defaults | custom  
2  
3 config
```

```
{'learning_rate': 0.01, 'epochs': 200, 'batch_size': 64}
```

The result is

```
1 {  
2   'learning_rate': 0.01,  
3   'epochs': 200,  
4   'batch_size': 64  
5 }
```

When the same key appears in both operands, the value from the **right-hand dictionary** takes precedence.

Thus,

left | right

can be interpreted as

start with the left mapping and override or extend it using the right mapping.

The original dictionaries remain unchanged:

```
1 defaults, custom
```

```
({'learning_rate': 0.01, 'epochs': 100}, {'epochs': 200, 'batch_size': 64})
```

6.25.1 In-Place Merging

The |= operator modifies the left dictionary in place:

```
1 defaults = {  
2   "learning_rate": 0.01,  
3   "epochs": 100,  
4 }  
5  
6 custom = {  
7   "epochs": 200,  
8   "batch_size": 64,  
9 }
```

```
10
11 defaults |= custom
12
13 defaults
```

```
{'learning_rate': 0.01, 'epochs': 200, 'batch_size': 64}
```

This distinction parallels other mutable structures:

```
1 a | b  -> new dictionary
2 a |= b -> update a in place
```

Understanding whether an operation creates a new object or modifies an existing one becomes important when multiple references to the same dictionary exist.

6.26 Removing Entries

Several mechanisms can remove entries from a dictionary.

6.27 The del Statement

The del statement removes an entry identified by its key:

```
1 config = {
2     "learning_rate": 0.01,
3     "epochs": 100,
4     "batch_size": 64,
5 }
6
7 del config["batch_size"]
8
9 config
```

```
{'learning_rate': 0.01, 'epochs': 100}
```

If the key does not exist, del raises a `KeyError`.

6.28 The pop() Method

The pop() method removes a key-value association and returns the corresponding value:

```
1 config = {  
2     "learning_rate": 0.01,  
3     "epochs": 100,  
4 }  
5  
6 epochs = config.pop("epochs")  
7  
8 epochs, config
```

(100, {'learning_rate': 0.01})

The removed value is 100, and the "epochs" entry no longer exists.

A default can be provided:

```
1 config.pop("batch_size", None)
```

If "batch_size" is absent, None is returned rather than raising KeyError.

Therefore:

```
1 dictionary.pop(key)
```

means

remove this key; absence is an error,

while

```
1 dictionary.pop(key, default)
```

allows missing keys to be handled through a fallback value.

6.29 The popitem() Method

The popitem() method removes and returns the most recently inserted key-value pair:

```
1 config = {  
2     "learning_rate": 0.01,  
3     "epochs": 100,  
4     "batch_size": 64,  
5 }  
6  
7 config.popitem()
```

('batch_size', 64)

returns a pair equivalent to

```
1 ('batch_size', 64)
```

and removes that entry.

Because dictionaries preserve insertion order, `popitem()` follows a last-in, first-out removal pattern.

If the dictionary is empty, `popitem()` raises a `KeyError`.

This method is useful when the order of insertion itself is relevant to the removal process, but it should not be used merely as a substitute for removing a known key.

6.30 The `clear()` Method

The `clear()` method removes all entries while preserving the dictionary object itself:

```
1 config = {  
2     "learning_rate": 0.01,  
3     "epochs": 100,  
4 }  
5  
6 config.clear()  
7  
8 config
```

{}

The result is an empty dictionary:

```
1 {}
```

As with lists, clearing an object is different from rebinding a variable to a newly created empty object. This distinction matters when aliases exist.

6.31 Common Dictionary Methods

The principal methods introduced so far can be summarised as follows:

Method	Purpose
<code>get()</code>	retrieve a value with an optional fallback
<code>keys()</code>	return a dynamic view of keys
<code>values()</code>	return a dynamic view of values
<code>items()</code>	return a dynamic view of key-value pairs
<code>update()</code>	insert or update multiple entries
<code>pop()</code>	remove a key and return its value
<code>popitem()</code>	remove and return the most recently inserted pair
<code>clear()</code>	remove all entries

Table 8: Common dictionary methods.

The purpose of these methods is easier to remember when viewed through the mapping model:

```
1 lookup    -> get()
2 inspect   -> keys(), values(), items()
3 modify    -> update()
4 remove    -> pop(), popitem(), clear()
```

6.32 The `setdefault()` Method

The `setdefault()` method combines lookup with conditional insertion.

Its general form is

```
1 dictionary.setdefault(key, default)
```

If key already exists, its current value is returned and the dictionary is unchanged.

If the key does not exist, the key is inserted with default as its value, and that value is returned.

For example:

```
1 config = {  
2     "learning_rate": 0.01,  
3 }  
4  
5 config.setdefault("learning_rate", 0.1)
```

0.01

returns 0.01, and the existing value is preserved.

By contrast:

```
1 config.setdefault("epochs", 100)
```

100

inserts

```
1 "epochs": 100
```

into the dictionary.

The final dictionary is

```
1 config
```

```
{'learning_rate': 0.01, 'epochs': 100}
```

`setdefault()` can be useful when a value must exist before it can be updated.

For example:

```
1 groups = {}  
2  
3 groups.setdefault("A", []).append("sample_1")  
4 groups.setdefault("A", []).append("sample_2")  
5  
6 groups
```



```
{'A': ['sample_1', 'sample_2']}
```

The result is

```
1 {'A': ['sample_1', 'sample_2']}
```

On the first call, the missing key "A" is associated with a new empty list. On the second call, the existing list is returned.

This pattern is useful, although specialised structures such as `collections.defaultdict` can express repeated grouping more directly in larger workflows. That structure can be introduced separately rather than treating `setdefault()` as the default solution to every grouping problem.

6.33 Aliasing with Dictionaries

Like lists, dictionaries are mutable objects.

Assignment therefore creates another reference rather than an independent copy.

Consider:

```
1 a = {  
2     "score": 0.81,  
3 }  
4  
5 b = a  
6  
7 b["score"] = 0.91  
8  
9 a, b
```

```
({'score': 0.91}, {'score': 0.91})
```

Both variables now display

```
1 {'score': 0.91}
```

because `a` and `b` refer to the same dictionary object.

A conceptual representation is

```

1 a ─┐
2   └─> {"score": 0.91}
3 b ─┐

```

This is the same aliasing principle previously encountered with lists.

6.34 Equality versus Identity

Two dictionaries may contain the same mapping while being distinct objects.

For example:

```

1 a = {"x": 1, "y": 2}
2 b = {"y": 2, "x": 1}
3
4 a == b

```

True

evaluates to True.

However:

```

1 a is b

```

False

evaluates to False.

The dictionaries represent equal mappings, but they were created independently.

By contrast:

```

1 a = {"x": 1}
2 b = a
3
4 a is b

```

True

is True.

As before:

- `==` tests value equality;
- `is` tests object identity.

6.35 Copying Dictionaries

The `copy()` method creates a shallow copy:

```
1 original = {  
2     "accuracy": 0.91,  
3     "recall": 0.84,  
4 }  
5  
6 copied = original.copy()  
7  
8 copied["accuracy"] = 0.95  
9  
10 original, copied
```

```
({'accuracy': 0.91, 'recall': 0.84}, {'accuracy': 0.95, 'recall': 0.84})
```

The original dictionary remains unchanged because the two outer dictionaries are distinct.

Equivalent shallow-copy constructions include

```
1 copied = dict(original)
```

and, for simple mappings,

```
1 copied = original.copy()
```

The important point is that these are **shallow copies**.

6.36 Shallow Copying with Nested Mutable Values

Consider:

```
1 original = {  
2     "features": ["age", "income"],  
3     "metrics": {  
4         "accuracy": 0.91,  
5     },  
6 }  
7  
8 copied = original.copy()
```

The outer dictionaries are independent:

```
1 original is copied
```

False

returns False.

However, their nested mutable values remain shared:

```
1 original["features"] is copied["features"]
```

True

returns True.

Likewise:

```
1 original["metrics"] is copied["metrics"]
```

True

is also True.

Consequently:

```
1 copied["features"].append("balance")
2
3 original
```

```
{'features': ['age', 'income', 'balance'], 'metrics': {'accuracy': 0.91}}
```

shows that the original dictionary's list has changed as well.

The shallow copy duplicated the mapping structure at the outer level, but copied references to the values.

6.37 Replacing a Value versus Mutating a Shared Value

The distinction becomes clearer by comparing two operations.

First:

```

1 original = {
2     "features": ["age", "income"],
3 }
4
5 copied = original.copy()
6
7 copied["features"].append("balance")
8
9 original, copied

```

```

({'features': ['age', 'income', 'balance']},
 {'features': ['age', 'income', 'balance']})

```

Both dictionaries observe the changed list because that list is shared.

Now compare:

```

1 original = {
2     "features": ["age", "income"],
3 }
4
5 copied = original.copy()
6
7 copied["features"] = ["balance"]
8
9 original, copied

```

```

({'features': ['age', 'income']}, {'features': ['balance']})

```

This time, only copied changes.

The assignment

```

1 copied["features"] = ...

```

changes which object the copied dictionary associates with the key "features".

It does not mutate the original list.

Thus:

```
1 copied[key].append(...)
```

may mutate a shared value,

while

```
1 copied[key] = ...
```

replaces the reference stored in the copied mapping.

6.38 Deep Copying Nested Dictionaries

When nested mutable values must also be independent, `copy.deepcopy()` can be used:

```
1 import copy
2
3 original = {
4     "features": ["age", "income"],
5     "metrics": {
6         "accuracy": 0.91,
7     },
8 }
9
10 copied = copy.deepcopy(original)
11
12 copied["features"].append("balance")
13 copied["metrics"]["accuracy"] = 0.95
14
15 original, copied
```

```
({'features': ['age', 'income'], 'metrics': {'accuracy': 0.91}},
 {'features': ['age', 'income', 'balance'], 'metrics': {'accuracy': 0.95}})
```

The original nested objects are unaffected.

As with lists, deep copying should be used when recursive independence is actually required rather than automatically applied to every structure.

6.39 Nested Dictionaries

Dictionary values may themselves be dictionaries.

For example:

```
1 experiment = {  
2     "model": {  
3         "name": "random forest",  
4         "trees": 200,  
5         "max_depth": 10,  
6     },  
7     "metrics": {  
8         "accuracy": 0.91,  
9         "precision": 0.87,  
10        "recall": 0.84,  
11    },  
12 }  
13  
14 experiment
```

```
{'model': {'name': 'random forest', 'trees': 200, 'max_depth': 10},  
'metrics': {'accuracy': 0.91, 'precision': 0.87, 'recall': 0.84}}
```

The outer dictionary has two keys:

```
1 "model"  
2 "metrics"
```

and each is associated with another dictionary.

Nested access follows the hierarchy of keys:

```
1 experiment["model"]["name"]
```

'random forest'

returns

```
1 'random forest'
```

while

```
1 experiment["metrics"]["recall"]
```

0.84

returns

```
1 0.84
```

Conceptually,

```
1 experiment["metrics"]["recall"]
```

means

1. retrieve the dictionary associated with "metrics";
2. from that dictionary, retrieve the value associated with "recall".

6.40 Updating Nested Dictionaries

Nested entries can be modified directly:

```
1 experiment = {  
2     "model": {  
3         "name": "random forest",  
4         "trees": 200,  
5     },  
6     "metrics": {  
7         "accuracy": 0.91,  
8     },  
9 }  
10  
11 experiment["model"]["trees"] = 300  
12 experiment["metrics"]["recall"] = 0.84  
13  
14 experiment
```

```
{'model': {'name': 'random forest', 'trees': 300},  
'metrics': {'accuracy': 0.91, 'recall': 0.84}}
```


The first operation updates an existing nested key. The second inserts a new key into the nested "metrics" dictionary.

Nested structures therefore preserve the same mapping semantics at every level.

6.41 Iterating over Nested Dictionaries

Nested dictionaries can be traversed using nested loops.

For example:

```
1 experiment = {
2     "model": {
3         "name": "random forest",
4         "trees": 200,
5     },
6     "metrics": {
7         "accuracy": 0.91,
8         "recall": 0.84,
9     },
10 }
11
12 for section, values in experiment.items():
13     print(section)
14
15     for key, value in values.items():
16         print(" ", key, value)
```

```
model
  name random forest
  trees 200
metrics
  accuracy 0.91
  recall 0.84
```

The outer loop traverses the top-level sections, while the inner loop traverses the key-value pairs within each nested dictionary.

Nested dictionaries are particularly natural when the data itself has hierarchical labels.

However, excessive nesting can make access patterns cumbersome:

```
1 data["experiment"]["model"]["parameters"]["regularization"]["value"]
```

At that point, the structure may still be technically valid, but it becomes worth asking whether a dedicated object model, table, configuration class, or specialised data representation would make the data easier to manage.

6.42 Merging Nested Dictionaries

Dictionary merging operates at the level of the keys being merged. It does **not** recursively merge nested dictionaries by default.

Consider:

```
1 defaults = {
2     "model": {
3         "trees": 100,
4         "max_depth": 10,
5     },
6     "verbose": False,
7 }
8
9 custom = {
10     "model": {
11         "trees": 300,
12     },
13 }
14
15 merged = defaults | custom
16
17 merged
```

```
{'model': {'trees': 300}, 'verbose': False}
```

The result associates "model" entirely with

```
1 {"trees": 300}
```

The nested "max_depth" entry from defaults is not preserved.

The merge is **shallow** with respect to nested mappings.

Conceptually:

```
1 top-level key collision
2   ↓
3 right-hand value replaces left-hand value
```

even when those values happen to be dictionaries.

Recursive merging requires additional logic and should not be assumed from the ordinary dictionary merge operators.

6.43 Structural Mutation during Iteration

As with lists, care is required when changing the size of a dictionary while iterating over it.

For example:

```
1 scores = {
2     "a": 0.91,
3     "b": 0.42,
4     "c": 0.87,
5 }
6
7 for key in scores:
8     if scores[key] < 0.5:
9         del scores[key]
```

modifies the dictionary's size during active iteration and raises a runtime error.

A safer approach is to iterate over a detached collection of keys:

```
1 scores = {
2     "a": 0.91,
3     "b": 0.42,
4     "c": 0.87,
5 }
6
7 for key in list(scores):
8     if scores[key] < 0.5:
9         del scores[key]
10
11 scores
```

```
{'a': 0.91, 'c': 0.87}
```

or, more naturally for a filtering operation, construct a new dictionary. Dictionary comprehensions will provide a concise way to express that pattern in the next section.

The general principle remains the same:

structural mutation of a container while traversing that same structure should be deliberate rather than assumed to be safe.

6.44 Dictionary Methods and Object Semantics

Several dictionary operations can now be classified according to whether they modify the existing object or produce another object.

Operation	Modifies original dictionary?	Result
<code>d[key] = value</code>	yes	insertion or update
<code>update()</code>	yes	None
<code>pop()</code>	yes	removed value
<code>popitem()</code>	yes	removed key-value pair
<code>clear()</code>	yes	None
<code>setdefault()</code>	possibly	existing or inserted value
<code>copy()</code>	no	new shallow dictionary
<code>a b</code>	no	new merged dictionary
<code>a = b</code>	yes	modified left dictionary

Table 9: Mutation and return behaviour of common dictionary operations.

The same distinction that mattered for lists matters here:

```
1 mutate existing object
```

and

```
1 construct a new object
```

are not interchangeable when aliases exist.

For mutable data structures, reasoning about code therefore requires tracking both

1. the values represented by an object;
2. the references that point to that object.

6.45 Constructing Dictionaries from Related Iterables

A common situation is to have one iterable containing keys and another containing the corresponding values.

For example:

```
1 models = ["linear", "tree", "forest"]
2 scores = [0.82, 0.78, 0.91]
```

These two sequences can be combined using `zip()`:

```
1 pairs = zip(models, scores)
2
3 list(pairs)
```

```
[('linear', 0.82), ('tree', 0.78), ('forest', 0.91)]
```

Conceptually, the resulting pairs are

("linear", 0.82), ("tree", 0.78), ("forest", 0.91).

Because `dict()` can construct a dictionary from an iterable of key-value pairs, the complete mapping can be created directly:

```
1 models = ["linear", "tree", "forest"]
2 scores = [0.82, 0.78, 0.91]
3
4 model_scores = dict(zip(models, scores))
5
6 model_scores
```

```
{'linear': 0.82, 'tree': 0.78, 'forest': 0.91}
```

The result is

```
1 {
2     'linear': 0.82,
3     'tree': 0.78,
4     'forest': 0.91
5 }
```

This pattern is useful whenever two corresponding sequences represent identifiers and associated values.

As with any use of `zip()`, the lengths of the iterables deserve attention. By default, `zip()` stops when the shortest iterable is exhausted.

For example:

```
1 models = ["linear", "tree", "forest"]
2 scores = [0.82, 0.78]
3
4 dict(zip(models, scores))
```

```
{'linear': 0.82, 'tree': 0.78}
```

silently excludes "forest".

If equal lengths are required by the data model, strict iteration can make that expectation explicit:

```
1 dict(zip(models, scores, strict=True))
```

With mismatched lengths, this raises a `ValueError` rather than silently truncating the data.

6.46 Dictionary Comprehensions

Python provides **dictionary comprehensions** for constructing dictionaries from iterable data.

Their general form is

```
1 {
2     key_expression: value_expression
3     for variable in iterable
4 }
```

For example:

```

1 values = [1, 2, 3, 4]
2
3 squares = {
4     value: value ** 2
5     for value in values
6 }
7
8 squares

```

{1: 1, 2: 4, 3: 9, 4: 16}

produces

```

1 {
2     1: 1,
3     2: 4,
4     3: 9,
5     4: 16
6 }

```

For every input value, the comprehension independently determines

- the key to insert;
- the value to associate with that key.

Mathematically, the construction

$$x \mapsto f(x)$$

can be represented as

```

1 {
2     x: f(x)
3     for x in iterable
4 }

```

when each x serves naturally as a unique key.

6.47 Transforming Dictionary Values

Dictionary comprehensions can operate directly on an existing mapping.

For example:

```
1 scores = {  
2     "model_a": 0.82,  
3     "model_b": 0.91,  
4     "model_c": 0.77,  
5 }  
6  
7 percentages = {  
8     model: 100 * score  
9     for model, score in scores.items()  
10 }  
11  
12 percentages
```

```
{'model_a': 82.0, 'model_b': 91.0, 'model_c': 77.0}
```

The keys are preserved while the values are transformed.

Conceptually,

$$(k, v) \mapsto (k, f(v)).$$

The resulting dictionary is new; the original mapping remains unchanged.

```
1 scores
```

```
{'model_a': 0.82, 'model_b': 0.91, 'model_c': 0.77}
```

6.48 Filtering Dictionaries

A condition can be added to a dictionary comprehension:

```
1 {  
2     key_expression: value_expression  
3     for variable in iterable  
4     if condition  
5 }
```


For example:

```
1 scores = {
2     "model_a": 0.82,
3     "model_b": 0.91,
4     "model_c": 0.77,
5     "model_d": 0.88,
6 }
7
8 selected = {
9     model: score
10    for model, score in scores.items()
11    if score >= 0.85
12 }
13
14 selected
```

```
{'model_b': 0.91, 'model_d': 0.88}
```

The result is

```
1 {
2     'model_b': 0.91,
3     'model_d': 0.88
4 }
```

This pattern is particularly useful when filtering a mapping without structurally modifying the dictionary being iterated over.

Compare:

```
1 for key in scores:
2     if scores[key] < 0.85:
3         del scores[key]
```

which changes the dictionary size during iteration, with

```
1 selected = {
2     key: value
3     for key, value in scores.items()
4     if value >= 0.85
5 }
```

which constructs a new mapping.

6.49 Transforming Keys and Values

Both sides of the key-value association can be transformed.

For example:

```
1 metrics = {  
2     "Accuracy": 0.91,  
3     "Precision": 0.87,  
4     "Recall": 0.84,  
5 }  
6  
7 normalized = {  
8     metric.lower(): round(value, 2)  
9     for metric, value in metrics.items()  
10 }  
11  
12 normalized
```

```
{'accuracy': 0.91, 'precision': 0.87, 'recall': 0.84}
```

produces keys in lowercase and transformed values.

However, key transformations require care because dictionary keys must remain unique.

Consider:

```
1 data = {  
2     "A": 1,  
3     "a": 2,  
4 }
```

The comprehension

```
1 {  
2     key.lower(): value  
3     for key, value in data.items()  
4 }
```

produces only one "a" entry.

Both original keys are transformed into the same resulting key, so the later association replaces the earlier one.

This follows directly from the uniqueness requirement for dictionary keys.

6.50 Key Collisions during Construction

Dictionary construction does not preserve multiple values for duplicate keys unless those values are explicitly collected into another structure.

Consider:

```
1 pairs = [  
2     ("A", 1),  
3     ("B", 2),  
4     ("A", 3),  
5 ]  
6  
7 dict(pairs)
```

{'A': 3, 'B': 2}

The result is

```
1 {'A': 3, 'B': 2}
```

The second association for "A" replaces the first.

This is an important distinction between dictionaries and grouping structures.

If the intended result is instead

```
1 {  
2     "A": [1, 3],  
3     "B": [2],  
4 }
```

the values must be explicitly accumulated.

6.51 Frequency Counting

A classic dictionary pattern is counting how many times each value occurs.

Suppose:

```
1 labels = ["cat", "dog", "cat", "bird", "dog", "cat"]
```

We can build a frequency dictionary using an explicit loop:

```
1 counts = {}
2
3 for label in labels:
4     if label in counts:
5         counts[label] += 1
6     else:
7         counts[label] = 1
8
9 counts
```

```
{'cat': 3, 'dog': 2, 'bird': 1}
```

The result is

```
1 {
2     'cat': 3,
3     'dog': 2,
4     'bird': 1
5 }
```

The dictionary state evolves dynamically:

$$\text{label} \mapsto \text{number of occurrences observed so far.}$$

The `get()` method provides a compact version of the same pattern:

```
1 counts = {}
2
3 for label in labels:
4     counts[label] = counts.get(label, 0) + 1
5
6 counts
```

```
{'cat': 3, 'dog': 2, 'bird': 1}
```

If label is absent, `get(label, 0)` provides the initial count 0. If it already exists, its current count is returned.

This pattern is worth understanding explicitly because frequency counting appears repeatedly in data processing.

For production code, Python also provides specialised tools such as `collections.Counter`, but the explicit dictionary implementation makes the underlying algorithm clear.

6.52 Grouping Values by Key

Another common pattern is collecting multiple values under the same key.

Consider:

```
1 records = [  
2     ("A", 0.82),  
3     ("B", 0.71),  
4     ("A", 0.91),  
5     ("B", 0.76),  
6     ("C", 0.88),  
7 ]
```

Suppose the desired result is

```
1 {  
2     'A': [0.82, 0.91],  
3     'B': [0.71, 0.76],  
4     'C': [0.88]  
5 }
```

An explicit solution is:

```
1 groups = {}  
2  
3 for group, value in records:  
4     if group not in groups:  
5         groups[group] = []  
6
```

```

7 groups[group].append(value)
8
9 groups

```

```
{'A': [0.82, 0.91], 'B': [0.71, 0.76], 'C': [0.88]}
```

This illustrates an important data-model distinction:

- the dictionary key identifies the group;
- the dictionary value is itself a mutable collection containing all observations assigned to that group.

Using `setdefault()` gives a more compact form:

```

1 groups = {}
2
3 for group, value in records:
4     groups.setdefault(group, []).append(value)
5
6 groups

```

```
{'A': [0.82, 0.91], 'B': [0.71, 0.76], 'C': [0.88]}
```

A specialised alternative is `collections.defaultdict`, which can be useful when grouping is pervasive. The explicit dictionary pattern remains valuable because it exposes the underlying logic.

6.53 Lookup Tables

Dictionaries are natural **lookup tables** when a known identifier determines an associated value.

For example:

```

1 class_names = {
2     0: "negative",
3     1: "positive",
4 }

```

A numeric prediction can be mapped to a human-readable label:

```

1 prediction = 1
2
3 class_names[prediction]

```

'positive'

Similarly:

```

1 feature_types = {
2     "age": "numeric",
3     "income": "numeric",
4     "country": "categorical",
5 }

```

allows metadata to be retrieved directly by feature name.

The defining operation is

$$k \mapsto v,$$

which is exactly the abstraction provided by a mapping.

This is generally clearer than encoding the same relationship through positional assumptions.

6.54 Dictionaries as Records

A dictionary can represent a record whose fields have explicit names:

```

1 model = {
2     "name": "random forest",
3     "n_estimators": 200,
4     "max_depth": 10,
5     "accuracy": 0.91,
6 }

```

Compared with

```

1 model = ["random forest", 200, 10, 0.91]

```

the dictionary makes the semantics of each field explicit.

For example:

```
1 model["accuracy"]
```

communicates more information than

```
1 model[3]
```

because the field meaning is encoded in the access itself.

This makes dictionaries useful for lightweight structured records.

However, as records become more complex or require validation, behaviour, type guarantees, or a stable schema, dedicated structures such as classes, dataclasses, typed models, or tabular data structures may become more appropriate.

6.55 Dictionaries as Configuration Objects

Dictionaries are frequently used for model and application configuration.

For example:

```
1 config = {  
2     "model": "random_forest",  
3     "n_estimators": 200,  
4     "max_depth": 12,  
5     "random_state": 42,  
6 }  
7  
8 config
```

```
{'model': 'random_forest',  
'n_estimators': 200,  
'max_depth': 12,  
'random_state': 42}
```

Configurations are often constructed by combining defaults with user-specific overrides:


```

1 defaults = {
2     "n_estimators": 100,
3     "max_depth": None,
4     "random_state": 42,
5 }
6
7 custom = {
8     "n_estimators": 300,
9 }
10
11 config = defaults | custom
12
13 config

```

```
{'n_estimators': 300, 'max_depth': None, 'random_state': 42}
```

The resulting configuration preserves all defaults except those explicitly overridden.

This pattern is convenient, but nested configuration dictionaries require particular care because ordinary dictionary merging is shallow.

6.56 Dictionaries for Model Metrics

A dictionary can naturally associate metric names with measured values:

```

1 metrics = {
2     "accuracy": 0.91,
3     "precision": 0.87,
4     "recall": 0.84,
5     "f1": 0.855,
6 }

```

This representation supports direct lookup:

```
1 metrics["f1"]
```

```
0.855
```

iteration:

```
1 for metric, value in metrics.items():
2     print(metric, value)
```

```
accuracy 0.91
precision 0.87
recall 0.84
f1 0.855
```

and filtering:

```
1 {
2     metric: value
3     for metric, value in metrics.items()
4     if value >= 0.86
5 }
```

```
{'accuracy': 0.91, 'precision': 0.87}
```

The dictionary is useful here because the metric names are meaningful identifiers rather than positional labels.

6.57 Dictionaries for Model Collections

A mapping can also associate identifiers with more complex records:

```
1 results = {
2     "linear": {
3         "accuracy": 0.84,
4         "training_time": 1.2,
5     },
6     "forest": {
7         "accuracy": 0.91,
8         "training_time": 8.7,
9     },
10    "svm": {
11        "accuracy": 0.88,
12        "training_time": 4.1,
13    },
14 }
15
16 results
```

```
{'linear': {'accuracy': 0.84, 'training_time': 1.2},  
'forest': {'accuracy': 0.91, 'training_time': 8.7},  
'svm': {'accuracy': 0.88, 'training_time': 4.1}}
```

Nested access can retrieve a particular metric:

```
1 results["forest"]["accuracy"]
```

0.91

and iteration can compare all models:

```
1 for model, metrics in results.items():  
2     print(model, metrics["accuracy"])
```

```
linear 0.84  
forest 0.91  
svm 0.88
```

Again, nested dictionaries are appropriate when the hierarchy corresponds naturally to meaningful identifiers.

6.58 Computational Characteristics

One of the main practical advantages of dictionaries is efficient key-based lookup.

At a conceptual level, a list search such as

```
1 target in values
```

may need to compare the target with many list elements.

A dictionary instead organises lookup around hashing.

For example:

```
1 dictionary[key]
```

uses the hash of the key to determine where the corresponding entry should be found.

Under ordinary conditions, operations such as

```
1 dictionary[key]
2 key in dictionary
3 dictionary[key] = value
```

have **average-case constant-time behaviour**, conventionally written as

$$O(1).$$

This does not mean that every dictionary operation always takes exactly the same amount of physical time, nor that worst-case behaviour is constant. It means that dictionary lookup is designed so that the expected amount of work does not grow linearly with the number of stored entries under normal hashing behaviour.

By contrast, searching for a particular value in a list generally has linear behaviour:

$$O(n).$$

This difference explains why dictionaries are particularly effective for repeated lookup by identifier.

6.59 Keys versus Values for Lookup

Dictionaries are optimised around keys, not values.

Testing

```
1 key in dictionary
```

uses key lookup.

By contrast:

```
1 value in dictionary.values()
```

generally requires searching through the values.

For example:

```
1 scores = {  
2     "a": 0.81,  
3     "b": 0.91,  
4     "c": 0.87,  
5 }  
6  
7 0.91 in scores.values()
```

True

works, but it does not have the same lookup semantics as searching for a key.

This reinforces an important design principle:

choose dictionary keys according to how the data will need to be retrieved.

If a program repeatedly needs to retrieve values by model name, the model name is a natural key.

6.60 Avoiding Redundant Lookups

Consider:

```
1 if key in data:  
2     value = data[key]
```

This performs a membership check and then a lookup.

Sometimes that is exactly what the logic requires, particularly when the distinction between presence and absence is important.

In other cases, `get()` may express the operation more directly:

```
1 value = data.get(key)
```

However, these forms are not always equivalent because `None` may be a legitimate stored value.

Likewise:

```
1 if key not in counts:
2     counts[key] = 0
3
4 counts[key] += 1
```

can often be written as

```
1 counts[key] = counts.get(key, 0) + 1
```

The preferred structure is the one that preserves the intended semantics clearly.

6.61 Choosing between Dictionaries and Other Structures

A dictionary is appropriate when objects are naturally identified by unique keys.

For example:

```
1 employee ID -> employee record
2 feature name -> feature metadata
3 model name -> model metrics
4 class ID -> class label
5 parameter -> configured value
```

A list is more natural when the structure is fundamentally sequential:

```
1 observation 0
2 observation 1
3 observation 2
4 ...
```

A set is more natural when only unique membership matters and no associated value is required.

A table-like structure such as a DataFrame may be more appropriate when many records share a common schema and operations are primarily columnar or tabular.

The choice of data structure should therefore reflect the operations the program needs to perform.

7 Problems - Dictionaries

The following problems focus on dictionary semantics, lookup behaviour, mutability, iteration, views, copying, and construction patterns.

Where possible, predict the result before executing the code.

7.1 Problem 1 — Basic Dictionary Access

Given

```
1 model = {  
2     "name": "linear",  
3     "score": 0.84,  
4     "features": 12,  
5 }
```

determine the result of

```
1 model["name"]  
2 model["score"]  
3 len(model)
```

What does `len(model)` count?

7.2 Problem 2 — Missing Keys

Given

```
1 metrics = {  
2     "accuracy": 0.91,  
3 }
```

compare

```
1 metrics["recall"]
```

with

```
1 metrics.get("recall")
```

and

```
1 metrics.get("recall", 0.0)
```

Explain the different missing-key semantics.

7.3 Problem 3 — Membership

Given

```
1 record = {  
2     "name": "model_a",  
3     "score": 0.91,  
4 }
```

predict:

```
1 "name" in record  
2 "model_a" in record  
3 0.91 in record
```

Explain what membership tests on a dictionary actually inspect.

7.4 Problem 4 — Stored None versus Missing Key

Consider:

```
1 record = {  
2     "comment": None,  
3 }
```

Compare:

```
1 record.get("comment")  
2 record.get("owner")
```

Why are the return values insufficient to determine whether the key exists?

Write an appropriate test for key presence.

7.5 Problem 5 — Duplicate Keys

Predict:

```
1 data = {  
2     "x": 1,  
3     "y": 2,  
4     "x": 3,  
5 }
```

What is the final value associated with "x"?

Why are there not two separate "x" entries?

7.6 Problem 6 — Valid and Invalid Keys

Determine which of the following objects can be dictionary keys:

```
1 42  
2 3.14  
3 "model"  
4 (1, 2)  
5 [1, 2]  
6 {"x": 1}
```

Explain the criterion using **hashability** rather than merely listing allowed types.

7.7 Problem 7 — Updating and Inserting

Starting from

```
1 config = {  
2     "epochs": 100,  
3     "learning_rate": 0.01,  
4 }
```

use indexed assignment to

1. change epochs to 200;
2. insert "batch_size": 64.

Explain why the same syntax performs both operations.

7.8 Problem 8 — Dictionary Equality

Predict:

```
1 a = {"x": 1, "y": 2}
2 b = {"y": 2, "x": 1}
3
4 a == b
```

Then compare this with:

```
1 [1, 2] == [2, 1]
```

Explain the difference between mapping equality and sequence equality.

7.9 Problem 9 — Iterating Directly

Given

```
1 metrics = {
2     "accuracy": 0.91,
3     "precision": 0.87,
4     "recall": 0.84,
5 }
```

what values does

```
1 for x in metrics:
2     print(x)
```

produce?

Rewrite the loop explicitly using the equivalent dictionary view.

7.10 Problem 10 — keys(), values(), and items()

For the dictionary in Problem 9, explain what is produced conceptually by

```
1 metrics.keys()
2 metrics.values()
3 metrics.items()
```

Which method is most appropriate when both the metric name and value are required?

7.11 Problem 11 — Dynamic Views

Predict:

```
1 data = {
2     "a": 1,
3     "b": 2,
4 }
5
6 keys = data.keys()
7
8 data["c"] = 3
9
10 print(list(keys))
```

Why does keys contain "c" even though it was created before "c" was inserted?

7.12 Problem 12 — Detached List versus View

Compare:

```
1 data = {"a": 1, "b": 2}
2
3 view = data.keys()
4 snapshot = list(data.keys())
5
6 data["c"] = 3
```

Predict

```
1 list(view)
2 snapshot
```

Explain the semantic difference.

7.13 Problem 13 — Iterating over Items

Given

```
1 scores = {  
2     "linear": 0.82,  
3     "tree": 0.78,  
4     "forest": 0.91,  
5 }
```

use `items()` to print

```
1 linear 0.82  
2 tree 0.78  
3 forest 0.91
```

without performing a second dictionary lookup inside the loop.

7.14 Problem 14 — `update()`

Starting from

```
1 config = {  
2     "epochs": 100,  
3     "learning_rate": 0.01,  
4 }
```

apply

```
1 {  
2     "epochs": 200,  
3     "batch_size": 64,  
4 }
```

using `update()`.

Which entry is replaced and which is inserted?

7.15 Problem 15 — Merging Dictionaries

Predict:

```
1 defaults = {  
2     "epochs": 100,  
3     "learning_rate": 0.01,  
4 }  
5  
6 custom = {  
7     "epochs": 300,  
8     "batch_size": 64,  
9 }  
10  
11 config = defaults | custom
```

What is config?

Are defaults and custom modified?

Explain the conflict-resolution rule.

7.16 Problem 16 — | versus |=

Starting from

```
1 a = {"x": 1}  
2 b = a
```

compare:

```
1 a = a | {"y": 2}
```

with

```
1 a |= {"y": 2}
```

using independent starting states.

How does aliasing affect the value observed through b?

7.17 Problem 17 — Removing with pop()

Given

```
1 config = {  
2     "epochs": 100,  
3     "batch_size": 64,  
4 }
```

predict:

```
1 removed = config.pop("batch_size")
```

What are removed and config afterwards?

Then explain the role of a default argument to pop().

7.18 Problem 18 — popitem()

Given

```
1 data = {  
2     "a": 1,  
3     "b": 2,  
4     "c": 3,  
5 }
```

predict the pair removed by

```
1 data.popitem()
```

Explain how insertion order determines the result.

7.19 Problem 19 — clear() and Aliasing

Predict:

```
1 a = {"x": 1, "y": 2}
2 b = a
3
4 a.clear()
5
6 print(a)
7 print(b)
```

Then compare with:

```
1 a = {"x": 1, "y": 2}
2 b = a
3
4 a = {}
5
6 print(a)
7 print(b)
```

Explain mutation versus rebinding.

7.20 Problem 20 — setdefault()

Predict:

```
1 config = {
2     "epochs": 100,
3 }
4
5 a = config.setdefault("epochs", 200)
6 b = config.setdefault("batch_size", 64)
```

Determine a, b, and the final dictionary.

7.21 Problem 21 — Aliasing

Without executing the code, determine the final values:

```
1 a = {"score": 0.81}
2 b = a
3
4 b["score"] = 0.91
5 b["name"] = "model_a"
```

What is a is b?

7.22 Problem 22 — Shallow Copy

Predict:

```
1 a = {
2     "score": 0.81,
3 }
4
5 b = a.copy()
6
7 b["score"] = 0.91
```

What are the two dictionaries?

What are

```
1 a == b
2 a is b
```

after the modification?

7.23 Problem 23 — Shallow Copy with Mutable Values

Consider:

```
1 a = {
2     "features": ["age", "income"],
3 }
4
5 b = a.copy()
6
7 b["features"].append("balance")
```


Predict both dictionaries.

Then determine:

```
1 a is b
2 a["features"] is b["features"]
```

7.24 Problem 24 — Replacement versus Nested Mutation

Starting independently from

```
1 a = {
2     "features": ["age", "income"],
3 }
4
5 b = a.copy()
```

compare:

```
1 b["features"].append("balance")
```

with

```
1 b["features"] = ["balance"]
```

Explain why only one of these operations necessarily modifies what is observed through a.

7.25 Problem 25 — Deep Copy

Given

```
1 config = {
2     "model": {
3         "type": "forest",
4         "trees": 100,
5     },
6     "features": ["age", "income"],
7 }
```

create an independent recursive copy using `copy.deepcopy()`.

Modify both nested structures in the copy and verify that the original remains unchanged.

7.26 Problem 26 — Nested Access

Given

```
1 experiment = {  
2     "model": {  
3         "name": "forest",  
4         "trees": 200,  
5     },  
6     "metrics": {  
7         "accuracy": 0.91,  
8         "recall": 0.84,  
9     },  
10 }
```

retrieve

1. "forest";
2. 200;
3. 0.84.

Write each nested-key expression explicitly.

7.27 Problem 27 — Updating Nested Dictionaries

Using the dictionary from Problem 26,

1. change trees to 300;
2. insert "precision": 0.87 into the metrics dictionary.

Do not replace the complete nested dictionaries.

7.28 Problem 28 — Shallow Merge of Nested Dictionaries

Predict:

```
1 defaults = {  
2     "model": {  
3         "trees": 100,  
4         "max_depth": 10,  
5     },
```

```

6     "verbose": False,
7 }
8
9 custom = {
10     "model": {
11         "trees": 300,
12     },
13 }
14
15 merged = defaults | custom

```

Does `merged["model"]` contain `"max_depth"`?

Explain why ordinary dictionary merge is not recursive.

7.29 Problem 29 — Building from `zip()`

Given

```

1 names = ["linear", "tree", "forest"]
2 scores = [0.82, 0.78, 0.91]

```

construct

```

1 {
2     "linear": 0.82,
3     "tree": 0.78,
4     "forest": 0.91,
5 }

```

using `dict()` and `zip()`.

7.30 Problem 30 — Mismatched `zip()` Inputs

Predict:

```

1 names = ["linear", "tree", "forest"]
2 scores = [0.82, 0.78]
3
4 dict(zip(names, scores))

```

Which element is lost?

Why can this be dangerous?

Then explain how `strict=True` changes the behaviour.

7.31 Problem 31 — Basic Dictionary Comprehension

Construct a dictionary mapping the integers from 1 through 5 to their squares.

The expected result is

```
1 {  
2   1: 1,  
3   2: 4,  
4   3: 9,  
5   4: 16,  
6   5: 25,  
7 }
```

Use a dictionary comprehension.

7.32 Problem 32 — Transforming Values

Given

```
1 scores = {  
2   "a": 0.82,  
3   "b": 0.91,  
4   "c": 0.77,  
5 }
```

construct a new dictionary with the same keys and percentage values:

```
1 {  
2   "a": 82.0,  
3   "b": 91.0,  
4   "c": 77.0,  
5 }
```

Do not modify the original.

7.33 Problem 33 — Filtering a Dictionary

Using the scores from Problem 32, construct a new dictionary containing only scores greater than or equal to 0.80.

Use a dictionary comprehension.

7.34 Problem 34 — Key Collision

Predict:

```
1 data = {  
2     "A": 1,  
3     "a": 2,  
4 }  
5  
6 normalized = {  
7     key.lower(): value  
8     for key, value in data.items()  
9 }
```

What is normalized?

Explain why one association disappears.

7.35 Problem 35 — Frequency Counting

Given

```
1 labels = [  
2     "cat",  
3     "dog",  
4     "cat",  
5     "bird",  
6     "dog",  
7     "cat",  
8 ]
```

build a dictionary containing the frequency of each label using

```
1 counts.get(label, 0)
```

The expected result is

```
1 {  
2   "cat": 3,  
3   "dog": 2,  
4   "bird": 1,  
5 }
```

7.36 Problem 36 — Grouping

Given

```
1 records = [  
2   ("A", 10),  
3   ("B", 20),  
4   ("A", 30),  
5   ("C", 40),  
6   ("B", 50),  
7 ]
```

construct

```
1 {  
2   "A": [10, 30],  
3   "B": [20, 50],  
4   "C": [40],  
5 }
```

First solve the problem explicitly using membership tests.

Then rewrite it using `setdefault()`.

7.37 Problem 37 — Lookup Table

Given

```
1 class_names = {  
2     0: "negative",  
3     1: "positive",  
4 }
```

and

```
1 predictions = [1, 0, 1, 1, 0]
```

construct

```
1 [  
2     "positive",  
3     "negative",  
4     "positive",  
5     "positive",  
6     "negative",  
7 ]
```

using the dictionary as a lookup table.

7.38 Problem 38 — Selecting Model Names

Given

```
1 results = {  
2     "linear": 0.82,  
3     "forest": 0.91,  
4     "svm": 0.88,  
5     "tree": 0.74,  
6 }
```

construct a list containing the names of models with scores at least 0.85.

The expected result is

```
1 ["forest", "svm"]
```

7.39 Problem 39 — Structural Mutation during Iteration

Explain why the following code is problematic:

```
1 scores = {  
2     "a": 0.91,  
3     "b": 0.42,  
4     "c": 0.87,  
5 }  
6  
7 for key in scores:  
8     if scores[key] < 0.5:  
9         del scores[key]
```

Rewrite the operation using a dictionary comprehension.

7.40 Problem 40 — Reasoning about References

Without executing the code, determine the exact output:

```
1 a = {  
2     "features": ["age", "income"],  
3     "metrics": {  
4         "accuracy": 0.91,  
5     },  
6 }  
7  
8 b = a  
9 c = a.copy()  
10  
11 b["features"].append("balance")  
12 c["metrics"]["accuracy"] = 0.95  
13 c["name"] = "model_a"  
14  
15 print(a)  
16 print(b)  
17 print(c)  
18  
19 print(a is b)  
20 print(a is c)
```



```
21 print(a["features"] is c["features"])
22 print(a["metrics"] is c["metrics"])
```

Trace the identity relationships at both the outer and nested levels.

8 Summary

A Python dictionary is a mutable mapping between unique, hashable keys and arbitrary values.

Its defining abstraction is

$$k \mapsto v,$$

rather than positional sequence access. Values are retrieved by key, and membership tests on a dictionary examine keys by default.

Dictionary keys must be hashable so that their lookup behaviour remains stable. Values have no corresponding hashability requirement and may themselves be mutable or nested structures.

Direct access with

```
1 dictionary[key]
```

is appropriate when a missing key represents an error, while `get()` provides fallback-based retrieval when absence is expected.

Dictionary views expose the mapping through

```
1 keys()
2 values()
3 items()
```

and remain dynamically connected to the underlying dictionary. Direct iteration traverses keys, while `items()` provides the natural representation when both keys and values are required.

Dictionaries are mutable, so operations such as indexed assignment, `update()`, `pop()`, `clear()`, and `|=` modify existing objects. By contrast, operations such as `copy()` and `a | b` create new outer dictionaries.

Assignment does not copy a dictionary:

```
1 b = a
```

creates an alias. Shallow copies separate the outer mapping but continue to share references to nested mutable values. When full recursive independence is required, `copy.deepcopy()` may be appropriate.

Dictionary comprehensions provide a concise way to construct, transform, and filter mappings:

```
1 {  
2     key_expression: value_expression  
3     for ...  
4     if ...  
5 }
```

They are particularly useful when the resulting data naturally remains a mapping.

Dictionaries also support several recurring computational patterns:

- lookup tables;
- frequency counts;
- grouping;
- named records;
- configuration mappings;
- model metrics;
- mappings between identifiers and structured results.

Because dictionary lookup is hash-based, key retrieval and membership typically have average-case $O(1)$ behaviour. This makes dictionaries particularly suitable when repeated lookup by identifier is a central operation.

The central conceptual distinction is therefore:

```
1 list    -> ordered sequence of values  
2 dictionary -> mapping from keys to values
```

Choosing between them should follow the structure of the data and the operations the program needs to perform, rather than superficial similarities in syntax.

9 Data Structures — Sets

A **set** is one of Python's built-in collection types. It represents a collection of **unique, hashable elements**.

For example:

```
1 labels = {"cat", "dog", "bird"}  
2  
3 labels
```

```
{'bird', 'cat', 'dog'}
```

A set differs fundamentally from a list or dictionary in the way it models data.

A list represents an ordered sequence:

```
1 position -> value
```

A dictionary represents a mapping:

```
1 key -> value
```

A set represents **membership**:

```
1 value -> present or absent
```

This makes sets particularly useful when the central questions are of the form

```
1 Is this value present?  
2 Which values are shared?  
3 Which values appear in either collection?  
4 Which values occur only in one collection?  
5 Is one collection contained in another?
```

These operations correspond closely to the mathematical notion of a set.

Python sets have several important properties:

- their elements are **unique**;
- their elements must be **hashable**;
- sets themselves are **mutable**;

- they are not positional sequences;
- they do not support indexing or slicing;
- their iteration order should not be treated as part of their semantic structure;
- they support efficient membership tests and standard set-theoretic operations.

9.1 Creating Sets

A set can be created by enclosing comma-separated elements in curly braces:

```
1 values = {1, 2, 3, 4, 5}
2
3 values
```

{1, 2, 3, 4, 5}

Its type is set:

```
1 type(values)
```

set

Sets may contain elements of different hashable types:

```
1 data = {
2     1,
3     3.14,
4     "model",
5     (2, 3),
6 }
7
8 data
```

{(2, 3), 1, 3.14, 'model'}

The important restriction is not that all elements have the same type, but that every element must be **hashable**.

9.2 Creating Sets with set()

The set() constructor creates a set from an iterable.

For example, from a list:

```
1 values = set([1, 2, 3, 4, 5])
2
3 values
```

{1, 2, 3, 4, 5}

or from a tuple:

```
1 values = set((1, 2, 3, 4, 5))
2
3 values
```

{1, 2, 3, 4, 5}

The constructor consumes the iterable and inserts its elements into the resulting set.

For example:

```
1 letters = set("python")
2
3 letters
```

{'h', 'n', 'o', 'p', 't', 'y'}

creates a set containing the distinct characters appearing in the string.

This illustrates an important distinction:

```
1 {"python"}
```

creates a set containing one string,

whereas

```
1 set("python")
```

iterates over the string and constructs a set from its characters.

Compare:

```
1 a = {"python"}
2 b = set("python")
3
4 print(a)
5 print(b)
```

```
{'python'}
{'t', 'y', 'n', 'o', 'p', 'h'}
```

The structures represent different data.

9.3 The Empty Set

An important syntactic detail is that

```
1 {}
```

does **not** create an empty set.

It creates an empty dictionary:

```
1 type({})
```

```
dict
```

An empty set must therefore be created with

```
1 empty = set()
2
3 empty
```

```
set()
```

and:

```
1 type(empty)
```

set

returns set.

This asymmetry exists because curly braces are also used for dictionary literals, and `{}` is reserved for the empty dictionary.

9.4 Unique Elements

A defining property of a set is that each distinct element occurs at most once.

Consider:

```
1 values = {1, 2, 2, 3, 3, 3, 4, 4, 4, 4}
2
3 values
```

`{1, 2, 3, 4}`

The repeated values do not create repeated set elements.

Conceptually,

$$\{1, 2, 2, 3, 3, 3\} = \{1, 2, 3\}.$$

A set therefore does not record multiplicity.

This makes sets useful for extracting the distinct values from another iterable:

```
1 labels = [
2     "cat",
3     "dog",
4     "cat",
5     "bird",
6     "dog",
7     "cat",
8 ]
9
10 unique_labels = set(labels)
```

```
11
12 unique_labels
```

```
{'bird', 'cat', 'dog'}
```

However, this conversion also discards information about

- how many times each value occurred;
- the original positional structure.

If multiplicity matters, a set alone is not an adequate representation.

9.5 Sets Are Not Sequences

A set should not be interpreted as a sequence.

For example:

```
1 values = {10, 20, 30}
```

does not establish semantic positions such as

```
1 position 0 -> 10
2 position 1 -> 20
3 position 2 -> 30
```

Consequently, sets do not support positional indexing:

```
1 values[0]
```

raises a `TypeError`.

Likewise, slicing is not defined:

```
1 values[0:2]
```

also fails.

This follows directly from the data model: set elements are identified by **membership**, not position.

9.6 Set Order

Sets should be treated as **unordered with respect to program semantics**.

For example:

```
1 values = {"apple", "pear", "melon", "orange"}
2
3 values
```

```
{'apple', 'melon', 'orange', 'pear'}
```

Python may display or iterate over these elements in some observable order, but that order should not be interpreted as

- insertion order;
- sorted order;
- positional order.

In particular, code should not depend on a particular set element being encountered first or last.

If an explicitly ordered result is required, it must be created deliberately. For example:

```
1 sorted(values)
```

```
['apple', 'melon', 'orange', 'pear']
```

returns a sorted **list**.

Thus:

```
1 set
```

and

```
1 sorted set elements
```

are conceptually different structures.

9.7 Length of a Set

The built-in `len()` function returns the number of distinct elements in a set.

For example:

```
1 values = {1, 2, 2, 3, 3, 3}
2
3 len(values)
```

3

returns

```
1 3
```

because the resulting set contains only

```
1 1, 2, 3
```

This differs from:

```
1 values = [1, 2, 2, 3, 3, 3]
2
3 len(values)
```

6

which returns 6.

The difference follows from the underlying data models:

```
1 list -> occurrences are retained
2 set -> only distinct elements are retained
```

9.8 Membership

Membership is one of the central operations associated with sets.

The operators

```
1 in
2 not in
```

test whether an element belongs to a set.

For example:

```
1 features = {
2     "age",
3     "income",
4     "balance",
5 }
6
7 "age" in features
```

True

returns True, whereas:

```
1 "country" in features
```

False

returns False.

Similarly:

```
1 "country" not in features
```

True

returns True.

In mathematical notation, if

$$A = \{\text{age, income, balance}\},$$

then

$$\text{age} \in A$$

while

country $\notin A$.

The correspondence between mathematical set membership and Python's `in` operator is therefore direct.

9.9 Hashability of Set Elements

Like dictionary keys, set elements must be **hashable**.

For example:

```
1 values = {  
2     42,  
3     3.14,  
4     "model",  
5     (1, 2),  
6 }  
7  
8 values
```

`{(1, 2), 3.14, 42, 'model'}`

These objects can be stored because they are hashable.

A list, however, cannot be inserted into a set:

```
1 {  
2     [1, 2],  
3     [3, 4],  
4 }
```

raises:

```
1 TypeError: unhashable type: 'list'
```

Similarly, an ordinary set cannot be an element of another set:

```
1 {  
2   {1, 2},  
3   {3, 4},  
4 }
```

is invalid because sets are mutable and therefore unhashable.

The restriction can be understood through the same principle used for dictionary keys:

hash-based lookup requires elements whose hash and equality behaviour remain suitable for stable membership testing.

9.10 Sets Are Mutable

Although their **elements must be hashable**, set objects themselves are mutable.

For example:

```
1 features = {  
2   "age",  
3   "income",  
4 }  
5  
6 features.add("balance")  
7  
8 features
```

```
{'age', 'balance', 'income'}
```

The set has been modified after creation.

An element can also be removed:

```
1 features.remove("income")  
2  
3 features
```

```
{'age', 'balance'}
```

This distinction is important:

```
1 set object    -> mutable
2 set elements  -> must be hashable
```

It would therefore be inaccurate to describe Python sets themselves as immutable.

Python does provide a separate immutable set type, `frozenset`, which will be discussed later.

9.11 Why Mutable Objects Cannot Usually Be Set Elements

Consider the following list:

```
1 coordinates = [1, 2]
```

If mutable lists were allowed as set elements, the list could later change:

```
1 coordinates.append(3)
```

potentially changing the information on which equality and hashing would depend.

A hash-based container requires stable element identity for lookup purposes.

This is why objects such as

```
1 integers
2 strings
3 many tuples
4 frozensets
```

can be set elements, while mutable structures such as

```
1 lists
2 dictionaries
3 sets
```

cannot.

Again, the precise rule is **hashability**, not merely a memorised division between particular built-in types.

9.12 Tuples as Set Elements

A tuple may be used as a set element when all the objects relevant to its hash are themselves hashable.

For example:

```
1 points = {  
2     (0, 0),  
3     (1, 2),  
4     (3, 4),  
5 }  
6  
7 points
```

`{(0, 0), (1, 2), (3, 4)}`

This makes sets useful for collections of unique coordinates or other composite immutable identifiers.

However, a tuple containing an unhashable object is itself unhashable:

```
1 (1, [2, 3])
```

cannot be inserted into a set because the nested list prevents the tuple from being hashable.

Thus, being a tuple does not automatically guarantee hashability.

9.13 frozenset

Python provides an immutable counterpart to set called frozenset.

For example:

```
1 values = frozenset({1, 2, 3})  
2  
3 values
```

`frozenset({1, 2, 3})`

Its contents cannot be modified:

```
1 values.add(4)
```

is invalid because a frozenset does not provide mutating methods such as `add()`.

Since a frozenset is immutable and hashable, it can itself be used as a set element:

```
1 groups = {  
2     frozenset({1, 2}),  
3     frozenset({3, 4}),  
4 }  
5  
6 groups
```

```
{frozenset({3, 4}), frozenset({1, 2})}
```

This allows mathematical structures such as a set of sets to be represented when the inner sets do not need to change.

For example, conceptually:

$$\mathcal{S} = \{\{1, 2\}, \{3, 4\}\}$$

can be represented in Python as

```
1 {  
2     frozenset({1, 2}),  
3     frozenset({3, 4}),  
4 }
```

9.14 Iterating over Sets

A set can be traversed using a for loop:

```
1 fruits = {  
2     "apple",  
3     "pear",  
4     "melon",  
5 }  
6  
7 for fruit in fruits:  
8     print(fruit)
```



```
pear  
melon  
apple
```

Every element is visited once.

However, because set iteration should not be treated as having meaningful positional order, code should not depend on which element is visited first.

If deterministic sorted output is required:

```
1 for fruit in sorted(fruits):  
2     print(fruit)
```

```
apple  
melon  
pear
```

the `sorted()` function explicitly constructs an ordered representation for traversal.

9.15 Sets and Duplicate Removal

One practical use of sets is extracting unique values.

Consider:

```
1 predictions = [  
2     "positive",  
3     "negative",  
4     "positive",  
5     "positive",  
6     "negative",  
7 ]  
8  
9 unique_predictions = set(predictions)  
10  
11 unique_predictions
```

```
{'negative', 'positive'}
```

This answers the question:

Which distinct labels occur?

It does not answer:

In what order did they occur?

or:

How often did each label occur?

Those questions require different structures.

For example, if counts are needed, a dictionary or collections.Counter is more appropriate.

9.16 Sets versus Lists and Dictionaries

The three structures developed so far represent different relationships.

Property	List	Dictionary	Set
Main abstraction	sequence	mapping	membership
Elements unique	no	keys only	yes
Positional indexing	yes	no	no
Key-based lookup	no	yes	no
Membership focus	values	keys	elements
Mutable	yes	yes	yes
Hashability required	no	keys	elements
Typical purpose	ordered observations	associations	unique membership

Table 10: Conceptual comparison of lists, dictionaries, and sets.

For example, suppose a machine-learning dataset contains the feature names

```
1 [
2     "age",
3     "income",
4     "balance",
5     "age",
6 ]
```

A list preserves the original sequence, including the repeated "age".

A set:

```

1 {
2   "age",
3   "income",
4   "balance",
5 }

```

represents only which distinct feature names occur.

A dictionary might instead represent metadata associated with those features:

```

1 {
2   "age": "numeric",
3   "income": "numeric",
4   "balance": "numeric",
5 }

```

The appropriate structure therefore depends on the question being represented:

```

1 list    -> what values occur, and in what sequence?
2 dictionary -> what value is associated with each key?
3 set     -> which distinct values belong to the collection?

```

9.17 Computational Characteristics of Membership

Sets are implemented using hashing.

Consequently, membership tests such as

```

1 value in my_set

```

typically have **average-case constant-time complexity**:

$$O(1).$$

By contrast, testing membership in a list generally requires examining elements until the target is found or the list is exhausted:

$$O(n).$$

For example, suppose a program repeatedly needs to determine whether an identifier belongs to a collection of accepted values:

```
1 allowed_ids = {  
2     "A102",  
3     "B317",  
4     "C491",  
5     "D204",  
6 }
```

Then:

```
1 candidate in allowed_ids
```

naturally expresses the problem as a membership query.

The advantage becomes increasingly relevant when the collection is large and membership tests are repeated many times.

However, the choice of a set should not be based on complexity alone. Converting a list to a set discards positional order and duplicate occurrences.

The correct data structure must therefore preserve the semantics required by the problem.

9.18 When a Set Is the Natural Representation

Sets are particularly appropriate for questions involving:

- unique categories;
- membership;
- duplicate elimination;
- overlap between collections;
- exclusion;
- containment;
- comparison of groups;
- combinations of unique identifiers.

For example:

```

1 training_features = {
2     "age",
3     "income",
4     "balance",
5     "country",
6 }
7
8 required_features = {
9     "age",
10    "income",
11    "balance",
12 }

```

The structures naturally support questions such as:

```

1 Are all required features available?
2 Which features are shared?
3 Which training features are additional?
4 Which required features are missing?

```

These questions lead directly to **subset relations and set operations**, which form the next part of the chapter.

9.19 Set Relations

Sets can be compared according to the relationships between their elements.

Consider:

```

1 A = {1, 2, 3}
2 B = {1, 2, 3, 4, 5}

```

Every element of A also belongs to B . In mathematical notation,

$$A \subseteq B.$$

Thus, A is a **subset** of B , while B is a **superset** of A .

Python provides both operators and methods for expressing these relationships.

9.20 Subsets

A set A is a subset of another set B when every element of A also belongs to B :

$$A \subseteq B.$$

In Python, the `<=` operator tests this relationship:

```
1 A = {1, 2, 3}
2 B = {1, 2, 3, 4, 5}
3
4 A <= B
```

True

returns True.

The equivalent method is:

```
1 A.issubset(B)
```

True

The relationship can be expressed logically as

$$A \subseteq B \iff \forall x, x \in A \Rightarrow x \in B.$$

For example:

```
1 required_features = {
2     "age",
3     "income",
4 }
5
6 available_features = {
7     "age",
8     "income",
9     "balance",
10    "country",
11 }
```

```
12
13 required_features <= available_features
```

True

returns True.

This gives a natural interpretation:

every required feature is available.

9.21 A Set Is a Subset of Itself

The subset relation allows equality.

Therefore:

$$A \subseteq A.$$

In Python:

```
1 A = {1, 2, 3}
2
3 A <= A
```

True

returns True.

Likewise:

```
1 A.issubset(A)
```

True

returns True.

This distinction becomes important when introducing **proper subsets**.

9.22 Proper Subsets

A set A is a **proper subset** of B when

1. every element of A belongs to B ;
2. A and B are not equal.

In notation:

$$A \subset B.$$

Python uses the $<$ operator:

```
1 A = {1, 2, 3}
2 B = {1, 2, 3, 4}
3
4 A < B
```

True

returns True.

However:

```
1 A < A
```

False

returns False.

Thus:

```
1 A <= B -> subset, equality allowed
2 A < B  -> proper subset, equality excluded
```

Equivalently,

$$A \subset B \iff A \subseteq B \text{ and } A \neq B.$$

9.23 Supersets

The inverse relationship is a **superset**.

If

$$A \subseteq B,$$

then

$$B \supseteq A.$$

Python uses `>=`:

```
1 A = {1, 2, 3}
2 B = {1, 2, 3, 4, 5}
3
4 B >= A
```

True

The equivalent method is:

```
1 B.issuperset(A)
```

True

A proper superset is tested with `>`:

```
1 B > A
```

True

Thus, the comparison operators form a natural family:

Mathematical relation	Python	Meaning
$A \subseteq B$	<code>A <= B</code>	subset
$A \subset B$	<code>A < B</code>	proper subset
$A \supseteq B$	<code>A >= B</code>	superset

Mathematical relation	Python	Meaning
$A \supset B$	<code>A > B</code>	proper superset

Table 11: Set inclusion relations in Python.

9.24 Set Equality

Two sets are equal when they contain exactly the same elements.

In mathematical notation:

$$A = B \iff \forall x, x \in A \iff x \in B.$$

For example:

```

1 A = {1, 2, 3}
2 B = {3, 1, 2}
3
4 A == B
```

True

returns True.

The order in which the elements were written does not affect equality.

Similarly:

```

1 {1, 2, 2, 3} == {3, 2, 1}
```

True

returns True, because duplicate occurrences do not create additional set elements.

Set equality therefore depends only on **membership**.

This differs from list equality:

```

1 [1, 2, 3] == [3, 2, 1]
```

False

which returns False because lists are sequences and positional order is part of their structure.

9.25 Disjoint Sets

Two sets are **disjoint** when they have no elements in common.

Mathematically:

$$A \cap B = \emptyset.$$

For example:

```
1 A = {1, 2, 3}
2 B = {4, 5, 6}
3
4 A.isdisjoint(B)
```

True

returns True.

By contrast:

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
3
4 A.isdisjoint(B)
```

False

returns False because both sets contain 3.

This operation can be useful when checking whether two groups overlap.

For example:

```
1 training_ids = {"A01", "A02", "A03"}
2 test_ids = {"B01", "B02", "B03"}
3
4 training_ids.isdisjoint(test_ids)
```

True

If the result is True, no identifier occurs in both sets.

This provides a simple conceptual pattern for checking whether two groups that should be separate actually overlap.

9.26 Set Operations

Beyond relationships between sets, Python supports the principal operations of elementary set algebra:

- union;
- intersection;
- difference;
- symmetric difference.

For two sets A and B , these operations construct new sets according to membership conditions.

9.27 Union

The **union** of two sets contains every element belonging to either set.

Mathematically:

$$A \cup B = \{x : x \in A \text{ or } x \in B\}.$$

For example:

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
3
4 A | B
```

$\{1, 2, 3, 4, 5\}$

returns a set equivalent to

```
1 {1, 2, 3, 4, 5}
```

The operator `|` corresponds to set union.

Python also provides the `union()` method:

```
1 A.union(B)
```

$\{1, 2, 3, 4, 5\}$

Thus:

```
1 A | B
```

and

```
1 A.union(B)
```

represent the same basic set operation.

Notice that 3 occurs only once in the result even though it belongs to both input sets.

That follows from the uniqueness of set elements.

9.28 Union of Multiple Sets

The `union()` method can combine more than two sets:

```
1 A = {1, 2}
2 B = {2, 3}
3 C = {3, 4}
4
5 A.union(B, C)
```

$\{1, 2, 3, 4\}$

produces

```
1 {1, 2, 3, 4}
```

The same operation can be expressed with chained operators:

```
1 A | B | C
```

$\{1, 2, 3, 4\}$

Mathematically:

$$A \cup B \cup C.$$

9.29 Intersection

The **intersection** contains only elements shared by both sets.

Mathematically:

$$A \cap B = \{x : x \in A \text{ and } x \in B\}.$$

For example:

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5, 6}
3
4 A & B
```

{3, 4}

returns

```
1 {3, 4}
```

The equivalent method is:

```
1 A.intersection(B)
```

{3, 4}

Thus:

```
1 A & B
```

corresponds to

$$A \cap B.$$

For example, suppose two models use the following features:

```

1 model_a_features = {
2     "age",
3     "income",
4     "balance",
5 }
6
7 model_b_features = {
8     "income",
9     "balance",
10    "country",
11 }
12
13 model_a_features & model_b_features

```

```
{'balance', 'income'}
```

The intersection identifies the features used by both models.

9.30 Intersection of Multiple Sets

Multiple intersections can also be computed:

```

1 A = {1, 2, 3, 4}
2 B = {2, 3, 4, 5}
3 C = {3, 4, 5, 6}
4
5 A.intersection(B, C)

```

```
{3, 4}
```

or:

```
1 A & B & C
```

```
{3, 4}
```

Both produce

```
1 {3, 4}
```

because only those elements belong to all three sets.

9.31 Difference

The **difference**

$$A - B$$

contains the elements that belong to A but not to B :

$$A - B = \{x : x \in A \text{ and } x \notin B\}.$$

For example:

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5}
3
4 A - B
```

{1, 2}

returns

```
1 {1, 2}
```

The equivalent method is:

```
1 A.difference(B)
```

{1, 2}

Unlike union and intersection, difference is not symmetric.

In general:

$$A - B \neq B - A.$$

For example:


```
1 print(A - B)
2 print(B - A)
```

```
{1, 2}
{5}
```

produces sets equivalent to:

```
1 {1, 2}
2 {5}
```

The direction of the operation therefore matters.

9.32 Finding Missing Elements

Set difference provides a particularly useful practical pattern.

Suppose:

```
1 required = {
2     "age",
3     "income",
4     "balance",
5     "country",
6 }
7
8 available = {
9     "age",
10    "income",
11    "balance",
12 }
```

Then:

```
1 required - available
```

```
{'country'}
```

returns

```
1 {'country'}
```

This answers the question:

Which required elements are missing?

Conversely:

```
1 available - required
```

set()

would identify available elements that were not required.

The same operation therefore has different interpretations depending on which set appears on the left.

9.33 Symmetric Difference

The **symmetric difference** contains elements belonging to exactly one of the two sets.

Mathematically:

$$A \triangle B = (A - B) \cup (B - A).$$

Python uses the ^ operator:

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5, 6}
3
4 A ^ B
```

{1, 2, 5, 6}

returns

```
1 {1, 2, 5, 6}
```

The elements 3 and 4 are excluded because they occur in both sets.

The equivalent method is:

```
1 A.symmetric_difference(B)
```

{1, 2, 5, 6}

Thus:

```
1 A ^ B
```

represents the elements for which membership differs between A and B .

9.34 Symmetric Difference versus Difference

Difference and symmetric difference answer different questions.

For:

```
1 A = {1, 2, 3}
```

```
2 B = {3, 4, 5}
```

we have:

```
1 print(A - B)
```

```
2 print(B - A)
```

```
3 print(A ^ B)
```

{1, 2}

{4, 5}

{1, 2, 4, 5}

Conceptually:

```
1 A - B -> elements unique to A
```

```
2 B - A -> elements unique to B
```

```
3 A ^ B -> elements unique to either set
```

Mathematically:

$$A \triangle B = (A - B) \cup (B - A).$$

This identity can be verified directly:

```
1 (A ^ B) == ((A - B) | (B - A))
```

True

which returns True.

9.35 Summary of Set Operations

The principal binary operations can be summarised as follows:

Operation	Mathematics	Operator	Method
Union	$A \cup B$	$A \mid B$	<code>A.union(B)</code>
Intersection	$A \cap B$	$A \& B$	<code>A.intersection(B)</code>
Difference	$A - B$	$A - B$	<code>A.difference(B)</code>
Symmetric difference	$A \triangle B$	$A \wedge B$	<code>A.symmetric_difference(B)</code>

Table 12: Principal set operations in Python.

The operators are particularly expressive when the computation is naturally interpreted as set algebra.

For example:

```
1 required - available
```

communicates the idea of missing elements directly, while:

```
1 training_ids & test_ids
```

naturally represents overlap.

9.36 Set Identities

Because Python set operations closely follow mathematical set algebra, familiar identities can be verified directly.

For example, union and intersection are commutative:

$$A \cup B = B \cup A,$$

$$A \cap B = B \cap A.$$

In Python:

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
3
4 print((A | B) == (B | A))
5 print((A & B) == (B & A))
```

True

True

Both expressions return True.

Difference is not commutative:

$$A - B \neq B - A$$

in general.

```
1 print(A - B)
2 print(B - A)
3 print((A - B) == (B - A))
```

{1, 2}

{4, 5}

False

9.37 Associativity

Union and intersection are associative:

$$(A \cup B) \cup C = A \cup (B \cup C),$$

and

$$(A \cap B) \cap C = A \cap (B \cap C).$$

For example:

```
1 A = {1, 2, 3}
2 B = {2, 3, 4}
3 C = {3, 4, 5}
4
5 print(((A | B) | C) == (A | (B | C)))
6 print(((A & B) & C) == (A & (B & C)))
```

True

True

Both return True.

9.38 Distributive Laws

Intersection distributes over union:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C).$$

Similarly, union distributes over intersection:

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C).$$

These identities can be expressed directly in Python:

```

1 A = {1, 2, 3, 4}
2 B = {3, 4, 5}
3 C = {4, 5, 6}
4
5 left = A & (B | C)
6 right = (A & B) | (A & C)
7
8 left == right

```

True

9.39 Difference and Intersection

Difference can also be interpreted through membership conditions.

The expression

$$A - B$$

keeps elements satisfying

$$x \in A \quad \text{and} \quad x \notin B.$$

This is useful when translating logical requirements into set operations.

For example:

```

1 all_features = {
2     "age",
3     "income",
4     "balance",
5     "country",
6 }
7
8 excluded_features = {
9     "country",
10 }
11
12 selected_features = all_features - excluded_features

```

13

14 `selected_features``{'age', 'balance', 'income'}`

The operation expresses exclusion directly.

9.40 Set Expressions as Logical Conditions

Set algebra often provides a compact representation of logic involving membership.

For example:

1 `A & B`

represents elements satisfying

1 `in A AND in B`

while:

1 `A | B`

represents elements satisfying

1 `in A OR in B`

and:

1 `A - B`

represents elements satisfying

1 `in A AND NOT in B`

Finally:

1 `A ^ B`

represents elements belonging to exactly one set:


```
1 (in A AND NOT in B)
2 OR
3 (in B AND NOT in A)
```

This correspondence makes sets useful when a problem is fundamentally about combinations of membership conditions.

9.41 Practical Example — Comparing Feature Sets

Suppose two experiments use different feature collections:

```
1 model_a = {
2     "age",
3     "income",
4     "balance",
5     "country",
6 }
7
8 model_b = {
9     "age",
10    "income",
11    "transactions",
12    "credit_score",
13 }
```

The common features are:

```
1 model_a & model_b
```

```
{'age', 'income'}
```

The features used only by model A are:

```
1 model_a - model_b
```

```
{'balance', 'country'}
```

The features used only by model B are:

```
1 model_b - model_a
```

```
{'credit_score', 'transactions'}
```

All features used by either model are:

```
1 model_a | model_b
```

```
{'age', 'balance', 'country', 'credit_score', 'income', 'transactions'}
```

The features appearing in exactly one model are:

```
1 model_a ^ model_b
```

```
{'balance', 'country', 'credit_score', 'transactions'}
```

A collection of related comparison questions can therefore be expressed using a small set of well-defined operations.

9.42 Practical Example — Checking Required Features

Suppose a model requires:

```
1 required = {  
2     "age",  
3     "income",  
4     "balance",  
5 }
```

and a dataset provides:

```
1 available = {  
2     "age",  
3     "income",  
4     "balance",  
5     "country",  
6 }
```

The condition:

```
1 required <= available
```

True

tests whether all required features are present.

If the condition is false, the missing features can be identified using:

```
1 required - available
```

set()

This gives a useful two-step validation pattern:

```
1 if required <= available:
2     print("All required features are available")
3 else:
4     missing = required - available
5     print("Missing features:", missing)
```

The subset relation answers whether the requirement is satisfied, while difference explains why it is not.

9.43 Practical Example — Detecting Data Leakage between Splits

Suppose observation identifiers have been divided into training and test sets:

```
1 train_ids = {
2     "C001",
3     "C002",
4     "C003",
5     "C004",
6 }
7
8 test_ids = {
9     "C005",
10    "C006",
11    "C007",
12 }
```

A basic consistency condition is that no observation belongs to both groups:

```
1 train_ids.isdisjoint(test_ids)
```

True

Equivalently, the intersection should be empty:

```
1 train_ids & test_ids
```

set()

If the result is:

```
1 set()
```

the identifier collections do not overlap.

If instead:

```
1 test_ids = {  
2     "C004",  
3     "C005",  
4     "C006",  
5 }
```

then:

```
1 train_ids & test_ids
```

{'C004'}

returns:

```
1 {'C004'}
```

This identifies the overlapping observation directly.

The example illustrates a broader pattern: sets are useful when validating whether collections of identifiers that should be separate actually intersect.

9.44 Operators versus Methods

Most fundamental set operations have both operator and method forms.

For example:

```
1 A | B
2 A.union(B)
```

and:

```
1 A & B
2 A.intersection(B)
```

express equivalent operations when A and B are sets.

The operator forms often make mathematical relationships particularly transparent:

```
1 required - available
2 train_ids & test_ids
3 features_a | features_b
```

The method forms can sometimes read more explicitly:

```
1 required.difference(available)
2 train_ids.intersection(test_ids)
3 features_a.union(features_b)
```

The choice is primarily one of clarity and context.

However, the two interfaces are not completely interchangeable for every possible operand type. Method forms can accept some iterable arguments that the corresponding binary operators require to be set-like.

For introductory reasoning, the most useful principle is:

use the representation that makes the intended set relationship easiest to read, while remaining aware of the operand requirements of the chosen operation.

9.45 Set Algebra as a Data-Processing Tool

The value of set operations extends beyond mathematical examples.

Many practical data-processing questions have the form:

- 1 Which identifiers occur in both datasets?
- 2 Which categories are new?
- 3 Which expected columns are missing?
- 4 Which users appear in either source?
- 5 Do training and test identifiers overlap?
- 6 Is every required field available?

These translate naturally into:

- 1 intersection
- 2 difference
- 3 union
- 4 disjointness
- 5 subset

respectively.

The important skill is therefore not memorising the symbols

- 1 | & - ^

in isolation.

It is recognising when a computational problem is fundamentally a **set-membership problem** and choosing the corresponding operation.

9.46 Modifying Sets

Python sets are mutable. Elements can be added or removed after the set has been created.

The fundamental mutation methods are

```
1 add()
2 update()
3 remove()
4 discard()
5 pop()
6 clear()
```

These operations modify the existing set rather than constructing a new one.

9.47 Adding a Single Element

The `add()` method inserts one element into a set:

```
1 features = {
2     "age",
3     "income",
4 }
5
6 features.add("balance")
7
8 features
```

```
{'age', 'balance', 'income'}
```

If the element is already present, the set does not change:

```
1 features.add("age")
2
3 features
```

```
{'age', 'balance', 'income'}
```

This follows from the uniqueness of set elements.

Thus, `add()` can be interpreted as

```
1 if element is absent -> insert it
2 if element is present -> no structural change
```

Unlike `list.append()`, which can increase a list even when an equal value is already present, a set never stores duplicate occurrences.

9.48 Adding Multiple Elements with update()

The `update()` method inserts all elements produced by one or more iterables.

For example:

```
1 features = {  
2     "age",  
3     "income",  
4 }  
5  
6 features.update([  
7     "balance",  
8     "country",  
9 ])  
10  
11 features
```

```
{'age', 'balance', 'country', 'income'}
```

The existing set is modified in place.

The method can accept other sets:

```
1 A = {1, 2}  
2 B = {2, 3, 4}  
3  
4 A.update(B)  
5  
6 A
```

```
{1, 2, 3, 4}
```

The result is equivalent in value to the union

$$A \cup B,$$

but the important difference is object semantics:


```
1 A | B
```

creates a new set,
whereas

```
1 A.update(B)
```

modifies A.
The operator form

```
1 A |= B
```

also performs an in-place union.

9.49 Removing Elements with `remove()`

The `remove()` method deletes a specified element:

```
1 features = {  
2     "age",  
3     "income",  
4     "balance",  
5 }  
6  
7 features.remove("income")  
8  
9 features
```

```
{'age', 'balance'}
```

If the requested element is absent, `remove()` raises a `KeyError`:

```
1 features.remove("country")
```

This makes `remove()` appropriate when the program expects the element to exist and its absence represents an unexpected state.

9.50 Removing Elements with `discard()`

The `discard()` method also removes an element, but does nothing if that element is absent:

```
1 features = {  
2     "age",  
3     "income",  
4     "balance",  
5 }  
6  
7 features.discard("income")  
8 features.discard("country")  
9  
10 features
```

```
{'age', 'balance'}
```

No exception is raised for "country".

The distinction is therefore:

```
1 remove(x) -> absence is an error  
2 discard(x) -> absence is acceptable
```

This is conceptually similar to the distinction between direct dictionary lookup and fallback-based access.

The appropriate operation depends on the semantics of the program.

9.51 Removing an Arbitrary Element with `pop()`

The `pop()` method removes and returns one element from a set:

```
1 values = {10, 20, 30}  
2  
3 removed = values.pop()  
4  
5 removed, values
```

```
(10, {20, 30})
```

Because sets do not provide meaningful positional ordering, the removed element should be treated as **arbitrary from the programmer's perspective**.

Code should therefore not assume that `pop()` removes

- the first inserted element;
- the smallest element;
- the largest element;
- the element displayed first.

If the set is empty, `pop()` raises a `KeyError`.

`pop()` is appropriate when some element is needed and its particular identity does not matter.

9.52 Clearing a Set

The `clear()` method removes all elements while preserving the set object:

```
1 values = {1, 2, 3}
2
3 values.clear()
4
5 values
```

`set()`

The result is an empty set.

As with other mutable containers, clearing an object is different from rebinding a variable:

```
1 values = set()
```

The difference becomes visible when multiple variables refer to the same set.

9.53 In-Place Set Operations

Each major set-algebra operation also has an in-place form.

For union:

```
1 A.update(B)
```

or:

```
1 A |= B
```

For intersection:

```
1 A.intersection_update(B)
```

or:

```
1 A &= B
```

For difference:

```
1 A.difference_update(B)
```

or:

```
1 A -= B
```

For symmetric difference:

```
1 A.symmetric_difference_update(B)
```

or:

```
1 A ^= B
```

The distinction is systematic:

```
1 ordinary operation -> construct a new set
2 in-place operation -> modify the existing set
```

For example:

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5}
3
4 C = A & B
5
6 print(A)
7 print(C)
```

{1, 2, 3, 4}
{3, 4}

leaves A unchanged.

By contrast:

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5}
3
4 A.intersection_update(B)
5
6 A
```

{3, 4}

modifies A so that only the common elements remain.

9.54 Summary of Mutating Methods

Method	Effect
add(x)	insert one element
update(iterable)	add multiple elements
remove(x)	remove x; raise KeyError if absent
discard(x)	remove x; do nothing if absent
pop()	remove and return an arbitrary element
clear()	remove all elements
intersection_update()	retain only common elements
difference_update()	remove elements appearing in another set
symmetric_difference_update()	retain elements occurring in exactly one set

Table 13: Common methods that modify a set in place.

9.55 Aliasing

Sets are mutable objects, so assignment does not create an independent copy.

Consider:

```

1 A = {1, 2, 3}
2 B = A
3
4 B.add(4)
5
6 A, B

```

$(\{1, 2, 3, 4\}, \{1, 2, 3, 4\})$

Both variables display the same set because only one set object exists.

Conceptually:

```

1 A ┌─┐
2   │─┤ {1, 2, 3, 4}
3 B └─┘

```

Thus:

```

1 A is B

```

True

returns True.

9.56 Equality versus Identity

Two distinct sets may contain exactly the same elements:

```

1 A = {1, 2, 3}
2 B = {3, 2, 1}
3
4 A == B

```

True

returns True.

However:

```
1 A is B
```

False

returns False.

Therefore:

```
1 == -> same members
2 is -> same set object
```

This follows the same general object model already encountered with lists and dictionaries.

9.57 Copying Sets

An independent shallow copy can be created using:

```
1 copied = original.copy()
```

For example:

```
1 original = {1, 2, 3}
2 copied = original.copy()
3
4 copied.add(4)
5
6 original, copied
```

`(({1, 2, 3}, {1, 2, 3, 4}))`

The original remains unchanged.

Because ordinary set elements must themselves be hashable, sets typically do not contain mutable structures such as lists, dictionaries, or other ordinary sets.

This means that the shallow-copy issue is generally less subtle for ordinary sets than it is for lists or dictionaries containing nested mutable values.

However, a set may contain custom hashable objects whose internal behaviour deserves separate consideration. The general distinction between object identity and copying still applies.

9.58 frozenset and Nested Set Structures

A frozenset can be an element of an ordinary set because it is hashable.

For example:

```
1 groups = {  
2     frozenset({"age", "income"}),  
3     frozenset({"balance", "country"}),  
4 }  
5  
6 groups
```

```
{frozenset({'age', 'income'}), frozenset({'balance', 'country'})}
```

This makes structures such as

$$\{\{\text{age, income}\}, \{\text{balance, country}\}\}$$

representable in Python.

Since the inner frozenset objects are immutable, they cannot be modified after creation.

9.59 Set Comprehensions

Python provides **set comprehensions** for constructing sets through iteration.

Their general form is

```
1 {  
2     expression  
3     for variable in iterable  
4 }
```

For example:

```
1 squares = {  
2     x ** 2  
3     for x in range(1, 6)  
4 }  
5  
6 squares
```


{1, 4, 9, 16, 25}

Conceptually, the transformation is

$$x \mapsto x^2.$$

However, unlike a list comprehension, repeated results are retained only once.

For example:

```
1 values = [-3, -2, -1, 0, 1, 2, 3]
2
3 squares = {
4     x ** 2
5     for x in values
6 }
7
8 squares
```

{0, 1, 4, 9}

The resulting set contains only

```
1 0, 1, 4, 9
```

even though several input values produce the same square.

This is an important property of set comprehensions:

the expression may be evaluated many times, but the resulting set stores only distinct values.

9.60 Filtering with Set Comprehensions

A condition can be added:

```
1 {
2     expression
3     for variable in iterable
4     if condition
5 }
```

For example:

```
1 values = range(-5, 6)
2
3 non_negative_squares = {
4     x ** 2
5     for x in values
6     if x >= 0
7 }
8
9 non_negative_squares
```

{0, 1, 4, 9, 16, 25}

The condition controls which inputs contribute to the resulting set.

9.61 Extracting Unique Transformed Values

Set comprehensions are especially useful when both transformation and uniqueness are required.

For example:

```
1 labels = [
2     "Positive",
3     "negative",
4     "POSITIVE",
5     "Negative",
6     "unknown",
7 ]
8
9 normalized = {
10     label.lower()
11     for label in labels
12 }
13
14 normalized
```

{'negative', 'positive', 'unknown'}

The transformation normalises case, while the set automatically removes duplicate resulting labels.

A comparable list comprehension would preserve repeated values:

```
1 [
2     label.lower()
3     for label in labels
4 ]
```

```
['positive', 'negative', 'positive', 'negative', 'unknown']
```

Thus, choosing between a list comprehension and a set comprehension changes the data semantics.

9.62 Conversion between Sets and Other Structures

A list can be converted to a set:

```
1 values = [3, 1, 3, 2, 1]
2
3 unique = set(values)
4
5 unique
```

```
{1, 2, 3}
```

This removes duplicate values but does not preserve the list's sequence semantics.

A set can be converted to a list:

```
1 values = {1, 2, 3}
2
3 list(values)
```

```
[1, 2, 3]
```

but the resulting list should not be interpreted as preserving some meaningful original set order.

If deterministic sorted output is required:

```
1 sorted(values)
```

[1, 2, 3]

is clearer.

9.63 Removing Duplicates while Preserving Order

A common temptation is to use

```
1 list(set(values))
```

to remove duplicates from a list.

This removes duplicate values, but it should not be used when the original order must be preserved.

For example:

```
1 values = ["b", "a", "b", "c", "a"]
```

If the desired result is

```
1 ["b", "a", "c"]
```

with first-occurrence order preserved, plain set conversion does not express that requirement.

One explicit solution is:

```
1 values = ["b", "a", "b", "c", "a"]
2
3 seen = set()
4 unique = []
5
6 for value in values:
7     if value not in seen:
8         seen.add(value)
9         unique.append(value)
10
11 unique
```

```
['b', 'a', 'c']
```

The set provides efficient membership tracking, while the list preserves sequence order.

This illustrates an important pattern: different data structures can be combined when the problem requires several semantics simultaneously.

9.64 Sets as Membership Filters

Sets are particularly effective when repeated membership tests are needed.

Suppose:

```
1 allowed = {  
2     "A",  
3     "B",  
4     "C",  
5 }  
6  
7 labels = [  
8     "A",  
9     "D",  
10    "B",  
11    "E",  
12    "C",  
13 ]
```

Filtering can be expressed as:

```
1 accepted = [  
2     label  
3     for label in labels  
4     if label in allowed  
5 ]  
6  
7 accepted
```

```
['A', 'B', 'C']
```

The output is a list because original sequence order may still matter, while `allowed` is a set because its role is efficient membership testing.

Again, one computational problem can naturally involve more than one data structure.

9.65 Comparing Categorical Values

Suppose two datasets contain categorical values:

```
1 training_categories = {  
2     "A",  
3     "B",  
4     "C",  
5 }  
6  
7 production_categories = {  
8     "A",  
9     "B",  
10    "C",  
11    "D",  
12 }
```

New categories encountered in production are:

```
1 production_categories - training_categories
```

```
{'D'}
```

which returns:

```
1 {'D'}
```

Categories seen during training but absent from production are:

```
1 training_categories - production_categories
```

```
set()
```

This kind of comparison is natural because the problem is about category membership rather than frequency or position.

9.66 Comparing Feature Schemas

Suppose:

```

1 expected_features = {
2     "age",
3     "income",
4     "balance",
5 }
6
7 received_features = {
8     "age",
9     "income",
10    "country",
11 }

```

Missing features are:

```
1 expected_features - received_features
```

```
{'balance'}
```

Unexpected features are:

```
1 received_features - expected_features
```

```
{'country'}
```

All schema differences can be found with:

```
1 expected_features ^ received_features
```

```
{'balance', 'country'}
```

This provides three complementary diagnostics from the same pair of sets.

9.67 Sets and Class Labels

Suppose:

```

1 y_true = [0, 0, 1, 1, 2, 2]
2 y_pred = [0, 1, 1, 2, 2, 3]

```

The observed class labels are:

```
1 true_classes = set(y_true)
2 predicted_classes = set(y_pred)
3
4 print(true_classes)
5 print(predicted_classes)
```

```
{0, 1, 2}
{0, 1, 2, 3}
```

Classes predicted but never observed are:

```
1 predicted_classes - true_classes
```

```
{3}
```

Classes observed but never predicted are:

```
1 true_classes - predicted_classes
```

```
set()
```

These operations do not assess model quality by themselves, but they provide useful structural information about the label sets involved.

9.68 Sets and Computational Complexity

Sets are implemented using hashing, so several common operations have favourable average-case behaviour.

For a set containing n elements, membership testing:

```
1 x in values
```

has average-case complexity approximately

$O(1)$.

Insertion:


```
1 values.add(x)
```

and removal by known value also have average-case constant-time behaviour.

Set algebra operations depend on the sizes of the sets involved, since elements must be examined to construct or modify the result.

For example, intersection between sets of sizes m and n requires work related to the input sizes rather than being constant time.

The main practical point is therefore:

sets are optimised for membership and uniqueness, not for positional traversal.

9.69 Choosing Sets Deliberately

A set is usually appropriate when the problem requires

- unique values;
- repeated membership testing;
- comparison of memberships between collections;
- subset or superset relationships;
- union, intersection, or difference;
- duplicate elimination when sequence order is not part of the required result.

A set is usually not appropriate when the problem requires

- positional indexing;
- duplicate occurrences;
- stable sequence order;
- mapping one identifier to another value.

Those requirements correspond more naturally to lists, dictionaries, or other structures.

10 Problems - Sets

The following problems focus on set semantics, hashability, mutation, algebraic operations, and practical membership-based reasoning.

Where possible, predict the result before executing the code.

10.1 Problem 1 — Creating a Set

Given:

```
1 values = [1, 2, 2, 3, 3, 3, 4]
```

construct a set from values.

What is the length of the resulting set?

Explain why it differs from `len(values)`.

10.2 Problem 2 — Empty Set

Predict:

```
1 type({})  
2 type(set())
```

Why does Python require `set()` to create an empty set?

10.3 Problem 3 — String Conversion

Compare:

```
1 {"python"}
```

with:

```
1 set("python")
```

How many elements does each structure contain?

10.4 Problem 4 — Membership

Given:

```
1 allowed = {"A", "B", "C"}
```

predict:

```
1 "A" in allowed
2 "D" in allowed
3 "D" not in allowed
```

Explain why sets are particularly suitable for this operation.

10.5 Problem 5 — Invalid Set Elements

Which of the following can be inserted into a set?

```
1 42
2 "model"
3 (1, 2)
4 [1, 2]
5 {1, 2}
6 frozenset({1, 2})
```

Explain every answer using hashability.

10.6 Problem 6 — Set Equality

Predict:

```
1 {1, 2, 3} == {3, 2, 1}
```

and:

```
1 {1, 2, 2, 3} == {1, 2, 3}
```

Explain why both results follow from membership semantics.

10.7 Problem 7 — Subset versus Proper Subset

Given:

```
1 A = {1, 2}
2 B = {1, 2, 3}
3 C = {1, 2}
```

predict:

```
1 A <= B
2 A < B
3 A <= C
4 A < C
```

Explain the role of equality.

10.8 Problem 8 — Superset

Using the sets from Problem 7, predict:

```
1 B >= A
2 B > A
3 C >= A
4 C > A
```

Relate each result to the corresponding subset relation.

10.9 Problem 9 — Disjointness

Given:

```
1 A = {1, 2, 3}
2 B = {4, 5, 6}
3 C = {3, 7, 8}
```

predict:

```
1 A.isdisjoint(B)
2 A.isdisjoint(C)
```

Verify each result using intersection.

10.10 Problem 10 — Union

Given:

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
```

calculate the union using

1. the `|` operator;
2. `union()`.

Verify that both results are equal.

10.11 Problem 11 — Intersection

Using the same sets, calculate:

```
1 A & B
```

and:

```
1 A.intersection(B)
```

What does the result represent?

10.12 Problem 12 — Difference Is Directional

Predict:

```
1 A - B
2 B - A
```

for:

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
```

Explain why difference is not commutative.

10.13 Problem 13 — Symmetric Difference

For the same sets, calculate:

```
1 A ^ B
```

Then verify:

```
1 (A ^ B) == ((A - B) | (B - A))
```

Explain the identity.

10.14 Problem 14 — Missing Features

Given:

```
1 required = {
2     "age",
3     "income",
4     "balance",
5     "country",
6 }
7
8 available = {
9     "age",
10    "income",
11    "balance",
12 }
```

determine

1. whether all required features are available;
2. which features are missing.

Use set relations rather than explicit loops.

10.15 Problem 15 — Unexpected Features

Using the sets from Problem 14, suppose:

```
1 available = {  
2     "age",  
3     "income",  
4     "balance",  
5     "country",  
6     "customer_id",  
7 }
```

identify features that are present but not required.

10.16 Problem 16 — Schema Difference

Given two sets of column names, use symmetric difference to identify every column appearing in exactly one schema.

Explain why this differs from checking only missing required columns.

10.17 Problem 17 — Train/Test Overlap

Given:

```
1 train_ids = {  
2     "A01",  
3     "A02",  
4     "A03",  
5     "A04",  
6 }  
7  
8 test_ids = {  
9     "A04",  
10    "A05",  
11    "A06",  
12 }
```

determine

1. whether the sets are disjoint;
2. which identifiers overlap.

10.18 Problem 18 — `add()`

Starting from:

```
1 values = {1, 2, 3}
```

apply:

```
1 values.add(4)
2 values.add(2)
```

Predict the final set and its length.

Why does the second call not increase the length?

10.19 Problem 19 — `update()`

Given:

```
1 A = {1, 2}
2 B = {2, 3, 4}
```

apply:

```
1 A.update(B)
```

Predict A.

Then compare with a fresh:

```
1 C = {1, 2} | B
```

What is the important object-level difference?

10.20 Problem 20 — `remove()` versus `discard()`

Starting independently from:

```
1 values = {1, 2, 3}
```

compare:

```
1 values.remove(4)
```

with:

```
1 values.discard(4)
```

Explain the appropriate use case for each method.

10.21 Problem 21 — `pop()`

Given:

```
1 values = {10, 20, 30}
```

execute:

```
1 x = values.pop()
```

What can be guaranteed about `x`?

What cannot be guaranteed?

10.22 Problem 22 — `clear()` and Aliasing

Predict:

```
1 A = {1, 2, 3}
2 B = A
3
4 A.clear()
5
6 print(A)
7 print(B)
```

Then compare this with rebinding:

```
1 A = {1, 2, 3}
2 B = A
3
4 A = set()
```

Explain the difference.

10.23 Problem 23 — In-Place Intersection

Given:

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5}
```

predict the final value of A after:

```
1 A.intersection_update(B)
```

How would the result differ at the object level from:

```
1 A = A & B
```

when aliases exist?

10.24 Problem 24 — In-Place Difference

Given:

```
1 A = {1, 2, 3, 4}
2 B = {3, 4, 5}
```

apply:

```
1 A.difference_update(B)
```

What remains in A?

10.25 Problem 25 — In-Place Symmetric Difference

Given:

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
```

apply:

```
1 A.symmetric_difference_update(B)
```

Predict the final set.

10.26 Problem 26 — Aliasing

Without executing:

```
1 A = {1, 2, 3}
2 B = A
3
4 B.add(4)
5 A.remove(1)
```

determine the final values of A and B.

What is:

```
1 A is B
```

?

10.27 Problem 27 — Copying

Compare:

```
1 A = {1, 2, 3}
2 B = A
```

with:

```
1 A = {1, 2, 3}
2 B = A.copy()
```

After `B.add(4)`, predict both versions.

10.28 Problem 28 — frozenset

Create a frozenset containing:

```
1 1, 2, 3
```

Attempt to call `add(4)`.

Why does the operation fail?

Then use that frozenset as an element of another set.

10.29 Problem 29 — Basic Set Comprehension

Construct the set of squares of the integers from 1 through 5 using a set comprehension.

10.30 Problem 30 — Duplicate Results in a Set Comprehension

Predict:

```
1 {
2   x ** 2
3   for x in range(-3, 4)
4 }
```

Why does the result contain fewer than seven elements?

10.31 Problem 31 — Filtering

Use a set comprehension to construct all even integers from 0 through 20.

10.32 Problem 32 — Unique Normalised Labels

Given:

```
1 labels = [  
2     "Positive",  
3     "positive",  
4     "NEGATIVE",  
5     "Negative",  
6     "unknown",  
7 ]
```

construct a set of lowercase unique labels.

10.33 Problem 33 — Unique Letters in a Phrase

Given a phrase, construct the set of distinct alphabetic characters appearing in it.

Ignore spaces and treat uppercase and lowercase versions of the same letter as equivalent.

10.34 Problem 34 — Common Letters between Words

Given two words, construct the set of letters appearing in both.

Solve the problem using intersection.

10.35 Problem 35 — Symmetric Difference of Integer Intervals

Given integers a , b , c , and d , construct the sets of integers in the closed intervals

$$[a, b]$$

and

$$[c, d].$$

Calculate their symmetric difference.

10.36 Problem 36 — Maximum without max()

Given a non-empty set of numbers, determine its maximum element without using max().

Use explicit iteration.

Explain why the solution cannot rely on set iteration order.

10.37 Problem 37 — Sieve of Eratosthenes

Given an integer $n \geq 2$, construct the set of all prime numbers between 2 and n using the **Sieve of Eratosthenes**.

A possible strategy is:

1. initialise the candidate set with all integers from 2 through n ;
2. repeatedly remove multiples of each prime candidate beginning at p^2 ;
3. stop eliminating multiples when

$$p^2 > n.$$

Explain why starting elimination at p^2 is sufficient.

10.38 Problem 38 — Removing Duplicates while Preserving Order

Given:

```
1 values = [  
2     "A",  
3     "B",  
4     "A",  
5     "C",  
6     "B",  
7     "D",  
8 ]
```

construct:

```
1 ["A", "B", "C", "D"]
```

using both

- a set for membership tracking;
- a list for preserving first-occurrence order.

Explain why:

```
1 list(set(values))
```

does not express the same requirement.

10.39 Problem 39 — Class-Set Comparison

Given:

```
1 y_true = [0, 0, 1, 1, 2, 2]
2 y_pred = [0, 1, 1, 2, 2, 3]
```

determine

1. the unique observed classes;
2. the unique predicted classes;
3. classes predicted but never observed;
4. classes observed but never predicted;
5. classes appearing in either collection but not both.

10.40 Problem 40 — Reasoning about Mutation and References

Without executing the code, determine the results:

```
1 A = {1, 2, 3}
2 B = A
3 C = A.copy()
4
5 B.add(4)
6 C.remove(1)
7 A |= {5}
8
9 print(A)
10 print(B)
11 print(C)
12
```

```

13 print(A is B)
14 print(A is C)
15 print(A == B)
16 print(A == C)

```

Trace which operations mutate shared objects and which set is independent.

11 Summary

A Python set is a mutable collection of unique, hashable elements.

Its central abstraction is **membership** rather than position or key-value association.

Sets provide a direct computational representation of elementary set relationships:

$$A \subseteq B, \quad A \cup B, \quad A \cap B, \quad A - B, \quad A \triangle B.$$

Python expresses these relationships using both operators and methods:

```

1 <= < >= >
2 | & - ^

```

together with methods such as

```

1 issubset()
2 issuperset()
3 isdisjoint()
4 union()
5 intersection()
6 difference()
7 symmetric_difference()

```

Sets are mutable, while their elements must be hashable. This distinction explains why lists, dictionaries, and ordinary sets cannot normally be elements of a set, while immutable structures such as tuples of hashable elements and frozenset objects can.

Mutation can be performed using methods such as


```
1 add()
2 update()
3 remove()
4 discard()
5 pop()
6 clear()
```

and through the in-place forms of the principal set operations.

As with other mutable Python objects, assignment creates aliases rather than copies. Consequently, distinguishing mutation from rebinding remains important.

Set comprehensions construct unique transformed or filtered values:

```
1 {
2     expression
3     for value in iterable
4     if condition
5 }
```

and are especially useful when uniqueness is part of the desired output semantics.

From a computational perspective, hash-based representation gives set membership tests average-case complexity approximately

$$O(1),$$

making sets particularly effective for repeated membership testing.

In data science and machine-learning workflows, sets are useful for tasks such as

- comparing feature schemas;
- detecting overlapping identifiers;
- identifying unseen categories;
- comparing class-label collections;
- extracting distinct values;
- validating required memberships.

The defining conceptual distinction among the principal mutable containers considered so far is therefore:

```
1 list    -> sequence
2 dictionary -> mapping
3 set     -> membership
```

The appropriate structure is the one whose semantics match the relationships represented by the problem.

12 Data Structures — Tuples

A **tuple** is one of Python's built-in sequence types. It represents an **ordered collection of elements that cannot be replaced, added, or removed after the tuple has been created**.

For example:

```
1 record = ("Bob", 32, "professor", True)
2
3 record
```

('Bob', 32, 'professor', True)

A tuple may contain objects of different types:

```
1 type(record)
```

tuple

In this example, the tuple contains

```
1 str
2 int
3 str
4 bool
```

Like a list, a tuple is a **sequence**. Its elements have positions, so tuples support operations such as

- indexing;
- negative indexing;
- slicing;

- membership testing;
- iteration;
- concatenation;
- repetition.

The fundamental difference is mutability:

```
1 list -> mutable sequence
2 tuple -> immutable sequence
```

This distinction is central to understanding when a tuple is an appropriate representation.

12.1 Tuple Literals

A tuple can be written as comma-separated values:

```
1 coordinates = (10, 20)
2
3 coordinates
```

(10, 20)

Its type is:

```
1 type(coordinates)
```

tuple

Tuples may contain heterogeneous elements:

```
1 observation = (
2     "C001",
3     42,
4     103.7,
5     True,
6 )
7
8 observation
```

('C001', 42, 103.7, True)

However, heterogeneity is not a defining requirement. A tuple may also contain elements of the same type:

```
1 point = (3.5, 7.2)
2
3 point
```

(3.5, 7.2)

What defines the structure is that it is an **ordered, immutable sequence**.

12.2 The Comma Creates the Tuple

Although tuples are commonly written with parentheses, the comma is what defines a tuple expression in many contexts.

For example:

```
1 coordinates = 10, 20
2
3 coordinates
```

(10, 20)

and:

```
1 type(coordinates)
```

tuple

show that coordinates is a tuple even though no parentheses were used.

Thus:

```
1 coordinates = (10, 20)
```

and:

```
1 coordinates = 10, 20
```

create equivalent tuple values.

Parentheses are nevertheless useful because they make grouping explicit and improve readability, particularly in more complex expressions.

12.3 Tuple Packing

The construction

```
1 coordinates = 10, 20
```

is often described as **tuple packing**.

Several values are collected into a single tuple object:

```
1 10, 20
2   ↓
3 (10, 20)
```

For example:

```
1 person = "Alice", 35, "engineer"
2
3 person
```

('Alice', 35, 'engineer')

The three values have been packed into one tuple.

This operation will become particularly useful when discussing **unpacking**, where the process is conceptually reversed.

12.4 Creating Tuples with tuple()

The tuple() constructor creates a tuple from an iterable.

For example, from a list:

```
1 values = tuple([1, 2, 3, 4])
2
3 values
```

(1, 2, 3, 4)

From a string:

```
1 letters = tuple("python")
2
3 letters
```

('p', 'y', 't', 'h', 'o', 'n')

The constructor iterates over the input object and stores its elements in a tuple.

Thus:

```
1 ("python",)
```

and:

```
1 tuple("python")
```

represent different structures.

The first contains one string:

```
1 ("python",)
```

while the second contains the individual characters:

```
1 ("p", "y", "t", "h", "o", "n")
```

12.5 The Empty Tuple

An empty tuple can be written as:

```
1 empty = ()
2
3 empty
```

()

Its type is:

```
1 type(empty)
```

tuple

It can also be created using:

```
1 tuple()
```

()

and its length is zero:

```
1 len(())
```

0

12.6 Single-Element Tuples

A particularly important syntactic case occurs when creating a tuple containing exactly one element.

Consider:

```
1 value = ("apple")
2
3 type(value)
```

str

The result is str, not tuple.

Parentheses alone perform grouping; they do not make this expression a tuple.

A one-element tuple requires a trailing comma:

```
1 value = ("apple",)
2
3 type(value)
```

tuple

The same applies without parentheses:

```
1 value = "apple",
2
3 type(value)
```

tuple

Therefore:

```
1 ("apple") -> string
2 ("apple",) -> tuple
```

This illustrates the more general principle introduced earlier:

in tuple syntax, the comma is structurally more important than the parentheses.

12.7 Length of a Tuple

The built-in len() function returns the number of elements in a tuple.

For example:

```
1 record = (
2     "John",
3     32,
4     "professor",
5     True,
6 )
7
8 len(record)
```


4

returns:

```
1 4
```

As with lists, nested objects count as single elements at the outer level.

For example:

```
1 data = (  
2     [1, 2, 3],  
3     {"a": 1},  
4     (4, 5),  
5 )  
6  
7 len(data)
```

3

returns 3, because the outer tuple contains three objects.

12.8 Indexing

Because tuples are sequences, their elements can be accessed by position.

Python uses zero-based indexing:

```
1 values = (  
2     "a",  
3     "b",  
4     "c",  
5     "d",  
6 )  
7  
8 values[0]
```

'a'

returns:

```
1 'a'
```

while:

```
1 values[2]
```

'c'

returns:

```
1 'c'
```

The positional model is therefore the same basic model used for lists:

```
1 index  0  1  2  3
2       ↓  ↓  ↓  ↓
3 value  "a" "b" "c" "d"
```

Attempting to access an index outside the valid range raises an `IndexError`.

12.9 Negative Indexing

Tuples also support negative indices.

For:

```
1 values = (  
2     "a",  
3     "b",  
4     "c",  
5     "d",  
6 )  
7  
8 values[-1]
```

'd'

returns the final element:

```
1 'd'
```

and:

```
1 values[-2]
```

'c'

returns:

```
1 'c'
```

Conceptually:

```
1 positive index  0  1  2  3
2               ↓  ↓  ↓  ↓
3 value          "a" "b" "c" "d"
4               ↑  ↑  ↑  ↑
5 negative index -4 -3 -2 -1
```

Negative indexing therefore provides positional access relative to the end of the sequence.

12.10 Slicing

Tuples support the same basic slicing syntax as other Python sequences:

```
1 tuple[start:stop]
```

The start index is included, while stop is excluded.

For example:

```
1 values = (
2     0,
3     1,
4     2,
5     3,
6     4,
7     5,
8 )
9
10 values[2:5]
```

(2, 3, 4)

returns:

```
1 (2, 3, 4)
```

The element at index 5 is not included.

12.11 Omitting Slice Bounds

The starting index may be omitted:

```
1 values[:3]
```

(0, 1, 2)

which means:

```
1 values[0:3]
```

Similarly:

```
1 values[3:]
```

(3, 4, 5)

means from index 3 through the end of the tuple.

Both bounds may be omitted:

```
1 values[:]
```

(0, 1, 2, 3, 4, 5)

which produces a tuple containing all the elements.

12.12 Slice Steps

A third slicing component specifies the step:

```
1 tuple[start:stop:step]
```

For example:

```
1 values = (  
2     0,  
3     1,  
4     2,  
5     3,  
6     4,  
7     5,  
8     6,  
9     7,  
10    )  
11  
12 values[::2]
```

(0, 2, 4, 6)

returns every second element:

```
1 (0, 2, 4, 6)
```

A negative step can reverse traversal:

```
1 values[::-1]
```

(7, 6, 5, 4, 3, 2, 1, 0)

returns:

```
1 (7, 6, 5, 4, 3, 2, 1, 0)
```

The result of slicing a tuple is another tuple.

12.13 Membership

The operators

```
1 in
2 not in
```

test whether a value occurs in a tuple.

For example:

```
1 features = (
2     "age",
3     "income",
4     "balance",
5 )
6
7 "income" in features
```

True

returns True.

Whereas:

```
1 "country" in features
```

False

returns False.

Unlike membership in a set, tuple membership generally requires a sequential search through the elements:

$$O(n)$$

in the general case.

This reflects the difference between the underlying abstractions:

```
1 tuple -> sequence
2 set  -> hash-based membership collection
```

If repeated membership testing is the central requirement and order or duplicate occurrences are not needed, a set may be a more appropriate structure.

12.14 Tuples Preserve Duplicates

Unlike sets, tuples preserve repeated values:

```
1 values = (  
2     1,  
3     2,  
4     2,  
5     3,  
6     3,  
7     3,  
8 )  
9  
10 values
```

(1, 2, 2, 3, 3, 3)

The length is:

```
1 len(values)
```

6

which returns 6.

Thus:

```
1 tuple -> preserves occurrences  
2 set  -> preserves distinct membership
```

This distinction follows directly from their respective data models.

12.15 Tuples Are Immutable

The defining distinction between tuples and lists is that tuples are immutable.

Consider:

```
1 fruits = (  
2     "cherry",  
3     "apple",  
4     "pear",  
5 )
```

Attempting:

```
1 fruits[1] = "kiwi"
```

raises:

```
1 TypeError: 'tuple' object does not support item assignment
```

Similarly, tuples do not provide mutating methods corresponding to list operations such as

```
1 append()  
2 extend()  
3 insert()  
4 remove()  
5 pop()  
6 clear()
```

Once a tuple has been created, its sequence of references cannot be modified.

12.16 Immutability Does Not Mean That Every Contained Object Is Immutable

This distinction is important.

A tuple itself may contain mutable objects:

```
1 data = (  
2     "model",  
3     [0.81, 0.84, 0.86],  
4 )  
5  
6 data
```

```
('model', [0.81, 0.84, 0.86])
```


The tuple cannot be changed so that its second element refers to a different object:

```
1 data[1] = [0.90]
```

is invalid.

However, the list already stored inside the tuple remains mutable:

```
1 data[1].append(0.89)
2
3 data
```

```
('model', [0.81, 0.84, 0.86, 0.89])
```

The tuple still contains the same two object references, but the internal state of the list has changed.

Therefore, tuple immutability means:

- the tuple's sequence of references cannot be changed after creation.

It does **not** mean:

- every object reachable through a tuple is necessarily immutable.

12.17 Tuple Immutability and Hashability

Tuple immutability is related to, but not identical with, hashability.

A tuple whose elements are hashable can itself be hashable:

```
1 point = (3, 5)
2
3 hash(point)
```

```
7586885779985432798
```

Such a tuple can be used as a dictionary key:

```
1 locations = {  
2     (0, 0): "origin",  
3     (3, 5): "point A",  
4 }  
5  
6 locations[(3, 5)]
```

'point A'

It can also be an element of a set:

```
1 points = {  
2     (0, 0),  
3     (1, 2),  
4     (3, 5),  
5 }  
6  
7 points
```

{(0, 0), (1, 2), (3, 5)}

This is particularly useful for composite identifiers such as coordinates.

12.18 Not Every Tuple Is Hashable

A tuple containing an unhashable object is itself unhashable.

For example:

```
1 data = (  
2     "model",  
3     [0.81, 0.84],  
4 )
```

cannot be hashed because it contains a list.

Therefore:

```
1 hash(data)
```

raises a `TypeError`.

Likewise, it cannot be used as a set element or dictionary key.

The precise principle is:

a tuple is hashable only when its contents satisfy the requirements for the tuple's hash to be computed.

This connects directly with the hashability rules used by dictionaries and sets.

12.19 Immutability versus Rebinding

Tuple immutability does not prevent a variable from being rebound.

For example:

```
1 point = (1, 2)
2
3 point = (3, 4)
4
5 point
```

`(3, 4)`

This is valid.

The original tuple was not modified. Instead, the name `point` was made to refer to a different tuple.

Conceptually:

```
1 before:
2
3 point —> (1, 2)
4
5 after:
6
7     (1, 2)
8
9 point —> (3, 4)
```

This distinction between **mutating an object** and **rebinding a name** is the same object-model distinction encountered with lists, dictionaries, and sets.

12.20 “Modifying” a Tuple by Reconstruction

Because tuples cannot be modified in place, changing their contents requires constructing another tuple.

For example:

```
1 fruits = (  
2     "cherry",  
3     "apple",  
4     "pear",  
5 )  
6  
7 fruits = (  
8     fruits[0],  
9     "kiwi",  
10    fruits[2],  
11 )  
12  
13 fruits
```

('cherry', 'kiwi', 'pear')

The variable now refers to a new tuple.

Another possible approach is to convert the tuple to a list, modify the list, and convert it back:

```
1 fruits = (  
2     "cherry",  
3     "apple",  
4     "pear",  
5 )  
6  
7 temporary = list(fruits)  
8 temporary[1] = "kiwi"  
9 fruits = tuple(temporary)  
10  
11 fruits
```

```
('cherry', 'kiwi', 'pear')
```

This works, but it is important to interpret the operation correctly: the original tuple was never mutated.

A new tuple was constructed and the variable was rebound.

12.21 When Immutability Is Useful

Immutability can express an important semantic property:

 this collection represents a fixed structure rather than a sequence intended to grow or change.

For example, a two-dimensional point can naturally be represented as:

```
1 point = (3.5, 7.2)
```

A model evaluation result might be represented as:

```
1 result = ("random_forest", 0.91)
```

and a fixed database-style record might be represented as:

```
1 customer = ("C001", 42, "Argentina")
```

Whether a tuple is preferable to another representation depends on context, but its immutability communicates that the **structural collection itself is fixed**.

12.22 Lists versus Tuples

Lists and tuples share sequence semantics but differ primarily in mutability.

Property	List	Tuple
Ordered sequence	yes	yes
Positional indexing	yes	yes
Slicing	yes	yes
Duplicate values	allowed	allowed
Heterogeneous elements	allowed	allowed

Property	List	Tuple
Mutable	yes	no
Can grow or shrink in place	yes	no
Potentially hashable	no	yes, depending on contents
Common notation	[...]	(...)

Table 14: Conceptual comparison of lists and tuples.

The decision should not be reduced to syntax.

A list naturally represents a sequence whose membership or size may change:

```
1 measurements = [10.2, 10.7, 11.1]
```

A tuple naturally represents a fixed grouping:

```
1 coordinate = (10.2, 10.7)
```

The underlying question is therefore:

Is this collection conceptually a mutable sequence, or a fixed sequence of values?

12.23 Tuples as Records

One common use of tuples is to group several related values into a single record.

For example:

```
1 prediction = (
2     "C001",
3     0.87,
4     1,
5 )
6
7 prediction
```

('C001', 0.87, 1)

The tuple might represent:

```
1 customer identifier
2 predicted probability
3 predicted class
```

The positions therefore have different semantic meanings:

```
1 index 0 -> customer identifier
2 index 1 -> probability
3 index 2 -> class
```

This differs from a tuple used as a homogeneous coordinate:

```
1 point = (3.5, 7.2)
```

Both are tuples, but their conceptual roles differ.

For small, local structures, positional records can be concise. As the number of fields grows or the meaning of positions becomes difficult to remember, named structures such as dictionaries, data classes, or other record-oriented abstractions may provide clearer semantics.

12.24 Tuples as Composite Keys

Because suitable tuples are hashable, they are useful for representing composite dictionary keys.

For example:

```
1 temperatures = {
2     ("Auckland", 2025): 15.8,
3     ("Wellington", 2025): 14.6,
4     ("Christchurch", 2025): 12.9,
5 }
6
7 temperatures[("Wellington", 2025)]
```

14.6

The key represents a combination of two dimensions:

```
1 (city, year) -> temperature
```

Similarly:

```
1 matrix_entries = {  
2     (0, 0): 10,  
3     (0, 1): 20,  
4     (1, 0): 30,  
5 }  
6  
7 matrix_entries[(1, 0)]
```

30

uses coordinate tuples as keys.

This is an important practical consequence of tuple hashability: several values can be combined into one immutable composite identifier.

12.25 Choosing a Tuple Deliberately

A tuple is particularly natural when

- sequence order matters;
- the collection has a fixed structural size;
- values form a logical record or coordinate;
- unpacking will make the code clearer;
- a hashable composite identifier is required and all components are hashable;
- mutation of the outer sequence is not part of the intended semantics.

A tuple is less appropriate when

- elements must frequently be inserted or removed;
- the sequence is expected to grow dynamically;
- positional field meanings become difficult to understand;
- repeated membership testing is the principal operation.

Choosing a tuple should therefore communicate something about the structure of the data, rather than merely replacing square brackets with parentheses.

12.26 Tuple Unpacking

One of the most characteristic uses of tuples in Python is **unpacking**.

Suppose:

```
1 point = (3.5, 7.2)
```

The two elements can be assigned directly to separate variables:

```
1 x, y = point
2
3 print(x)
4 print(y)
```

3.5

7.2

Conceptually:

```
1 point = (3.5, 7.2)
2     ↓  ↓
3     x  y
```

The tuple is unpacked according to position.

This provides a concise alternative to:

```
1 x = point[0]
2 y = point[1]
```

and often communicates the structure of the data more clearly.

12.27 Packing and Unpacking

Tuple packing and unpacking are complementary operations.

Packing:

```

1 x = 3.5
2 y = 7.2
3
4 point = x, y
5
6 point

```

(3.5, 7.2)

collects several values into one tuple.

Unpacking:

```

1 x, y = point

```

assigns those values to separate variables.

Conceptually:

```

1 packing:
2
3 x = 3.5
4 y = 7.2
5
6 x, y
7   ↓
8 (3.5, 7.2)
9
10
11 unpacking:
12
13 (3.5, 7.2)
14   ↓   ↓
15  x   y

```

This pattern appears frequently throughout Python.

12.28 Unpacking Requires Compatible Structure

Ordinary unpacking requires the number of target variables to match the number of elements being unpacked.

For example:

```
1 record = (  
2     "C001",  
3     42,  
4     0.87,  
5 )  
6  
7 customer_id, age, probability = record  
8  
9 print(customer_id)  
10 print(age)  
11 print(probability)
```

```
C001  
42  
0.87
```

The three tuple elements match the three target variables.

However:

```
1 customer_id, age = record
```

raises:

```
1 ValueError: too many values to unpack
```

because the tuple contains three elements but only two target variables are provided.

Similarly:

```
1 a, b, c, d = record
```

raises:

```
1 ValueError: not enough values to unpack
```

The assignment structure must therefore be compatible with the structure of the iterable being unpacked.

12.29 Extended Unpacking with *

Python provides **extended iterable unpacking** when one target should collect multiple elements.

For example:

```
1 record = (  
2     "C001",  
3     42,  
4     103.7,  
5     "Argentina",  
6 )  
7  
8 customer_id, *details = record  
9  
10 print(customer_id)  
11 print(details)
```

```
C001  
[42, 103.7, 'Argentina']
```

The first element is assigned to `customer_id`, while the remaining elements are collected into `details`.

An important detail is that the starred target receives a **list**, not a tuple:

```
1 type(details)
```

```
list
```

returns:

```
1 list
```

Thus:

```
1 ("C001", 42, 103.7, "Argentina")
2   ↓       ↓
3 customer_id  *details
4
5 customer_id -> "C001"
6 details    -> [42, 103.7, "Argentina"]
```

12.30 Starred Targets in Different Positions

The starred target does not need to appear at the end.

For example:

```
1 values = (
2     1,
3     2,
4     3,
5     4,
6     5,
7 )
8
9 first, *middle, last = values
10
11 print(first)
12 print(middle)
13 print(last)
```

```
1
[2, 3, 4]
5
```

produces conceptually:

```
1 first -> 1
2 middle -> [2, 3, 4]
3 last -> 5
```

Python assigns enough elements to the starred target so that the remaining non-starred targets can still receive one value each.

Another example is:

```
1 first, second, *rest = values
2
3 print(first)
4 print(second)
5 print(rest)
```

```
1
2
[3, 4, 5]
```

which produces:

```
1 first -> 1
2 second -> 2
3 rest -> [3, 4, 5]
```

Only one starred target is allowed at a given unpacking level.

12.31 Ignoring Values with _

Sometimes an unpacked value is intentionally not needed.

By convention, the variable name `_` is often used for such values.

For example:

```
1 record = (
2     "C001",
3     42,
4     0.87,
5 )
6
7 customer_id, _, probability = record
8
9 print(customer_id)
10 print(probability)
```

```
C001
0.87
```

The age is unpacked but intentionally ignored.

The underscore has no special assignment semantics here: it is still a valid Python variable name.

Its significance is conventional:

```
    this value is intentionally not used.
```

This convention can improve readability when only some positions are relevant.

12.32 Combining `_` and Starred Unpacking

Starred unpacking can also be used when several intermediate values are irrelevant.

For example:

```
1 record = (
2     "C001",
3     42,
4     103.7,
5     "Argentina",
6     0.87,
7 )
8
9 customer_id, *_ , probability = record
10
11 print(customer_id)
12 print(probability)
```

```
C001
0.87
```

The first and final values are retained, while the intermediate values are collected into `_`.

This pattern should be used when it makes the intended structure clearer rather than simply to shorten code.

12.33 Swapping Variables

Python unpacking provides a concise way to exchange the values of two variables.

Suppose:

```
1 a = 10
2 b = 20
```

The values can be swapped using:

```
1 a, b = b, a
2
3 print(a)
4 print(b)
```

```
20
10
```

Conceptually, the values on the right-hand side are evaluated before the assignments on the left-hand side are completed.

This avoids the explicit temporary-variable pattern:

```
1 temporary = a
2 a = b
3 b = temporary
```

Tuple-like packing and unpacking therefore support a common Python idiom:

```
1 a, b = b, a
```

12.34 Multiple Assignment

Unpacking also allows several variables to be assigned in one statement:

```
1 name, age, country = (
2     "Alice",
3     35,
4     "New Zealand",
5 )
```



```
6
7 print(name)
8 print(age)
9 print(country)
```

```
Alice
35
New Zealand
```

The important idea is not merely syntactic compactness.

The left-hand side mirrors the structure of the right-hand side:

```
1 (name, age, country)
2   ↓
3 ("Alice", 35, "New Zealand")
```

This structural correspondence is one of the reasons unpacking is widely used in Python.

12.35 Nested Unpacking

Unpacking can follow nested structure.

For example:

```
1 record = (
2     "C001",
3     (3.5, 7.2),
4 )
5
6 customer_id, (x, y) = record
7
8 print(customer_id)
9 print(x)
10 print(y)
```

```
C001
3.5
7.2
```

The assignment pattern mirrors the nested tuple:

```
1 (
2     customer_id,
3     (x, y),
4 )
```

corresponding to:

```
1 (
2     "C001",
3     (3.5, 7.2),
4 )
```

Nested unpacking can be useful when the data structure is simple and well understood.

For deeply nested structures, however, excessive positional unpacking can reduce readability.

12.36 Returning Multiple Values from Functions

Python functions can appear to return multiple values:

```
1 def min_max(values):
2     return min(values), max(values)
3
4
5 result = min_max([4, 8, 1, 7, 3])
6
7 result
```

(1, 8)

The returned object is a tuple:

```
1 type(result)
```

tuple

The expression:

```
1 return min(values), max(values)
```

packs the two values into a tuple.

The caller can then unpack them directly:

```
1 minimum, maximum = min_max([4, 8, 1, 7, 3])
2
3 print(minimum)
4 print(maximum)
```

```
1
8
```

Thus, what is commonly described as “returning multiple values” is more precisely:

```
1 function
2   ↓
3 returns one tuple
4   ↓
5 caller unpacks tuple
```

This pattern is extremely common in Python APIs.

12.37 Tuple Concatenation

Tuples support concatenation with the + operator.

For example:

```
1 A = (
2     1,
3     2,
4     3,
5 )
6
7 B = (
8     4,
9     5,
10 )
```

```
11
12 C = A + B
13
14 C
```

(1, 2, 3, 4, 5)

The result is:

```
1 (1, 2, 3, 4, 5)
```

Because tuples are immutable, concatenation does not modify either operand.

Conceptually:

```
1 A —> (1, 2, 3)
2
3 B —> (4, 5)
4
5 C —> (1, 2, 3, 4, 5)
```

C refers to a newly constructed tuple.

12.38 Concatenation Does Not Mutate a Tuple

Consider:

```
1 A = (
2     1,
3     2,
4     3,
5 )
6
7 original_id = id(A)
8
9 A = A + (4, 5)
10
11 print(A)
12 print(id(A) == original_id)
```

```
(1, 2, 3, 4, 5)
False
```

The comparison returns False under ordinary execution because the variable has been rebound to a newly constructed tuple.

The expression:

```
1 A = A + (4, 5)
```

should therefore not be interpreted as modifying the original tuple.

It performs two conceptual steps:

```
1 1. construct a new tuple from A and (4, 5)
2 2. rebind A to the new tuple
```

This is another example of the distinction between mutation and rebinding.

12.39 Tuple Repetition

The * operator repeats a tuple a specified number of times.

For example:

```
1 values = (
2     "A",
3     "B",
4 )
5
6 repeated = values * 3
7
8 repeated
```

```
('A', 'B', 'A', 'B', 'A', 'B')
```

produces:

```
1 ("A", "B", "A", "B", "A", "B")
```

The original tuple remains unchanged.

Similarly:

```
1 (0,) * 5
```

(0, 0, 0, 0, 0)

produces:

```
1 (0, 0, 0, 0, 0)
```

Notice again the comma in (0,).

Without it:

```
1 (0) * 5
```

would simply perform integer multiplication.

12.40 Repetition and Object References

Tuple repetition repeats references to the contained objects.

This matters when the tuple contains a mutable object.

For example:

```
1 values = ([],) * 3
```

```
2
```

```
3 values
```

([], [], [])

The three positions refer to the same list object.

Therefore:

```
1 values[0].append(10)
```

```
2
```

```
3 values
```

```
([10], [10], [10])
```

produces a structure equivalent to:

```
1 ([10], [10], [10])
```

This does not violate tuple immutability.

The tuple's three references have not changed. The mutable list referenced by all three positions has changed internally.

The distinction is:

```
1 tuple repetition -> repeats references  
2 not necessarily independent copies
```

This is the same aliasing principle encountered with nested lists and other mutable objects.

12.41 Tuple Methods

Tuples provide relatively few methods because they are immutable.

Two particularly useful methods are:

```
1 count()  
2 index()
```

12.42 count()

The count() method returns the number of occurrences of a value.

For example:

```
1 values = (  
2     1,  
3     2,  
4     3,  
5     2,  
6     4,  
7     2,  
8 )
```

```
9
10 values.count(2)
```

3

returns:

```
1 3
```

Unlike a set, a tuple preserves duplicate occurrences, so counting repeated values is meaningful.

12.43 index()

The `index()` method returns the index of the first occurrence of a value.

For example:

```
1 values = (  
2     "A",  
3     "B",  
4     "C",  
5     "B",  
6 )  
7  
8 values.index("B")
```

1

returns:

```
1 1
```

Although "B" also occurs at index 3, `index()` returns the first matching position.

If the value is absent:

```
1 values.index("D")
```

raises a `ValueError`.

12.44 Iterating over Tuples

A tuple can be traversed directly with a for loop:

```
1 features = (  
2     "age",  
3     "income",  
4     "balance",  
5 )  
6  
7 for feature in features:  
8     print(feature)
```

```
age  
income  
balance
```

The elements are visited in sequence order.

Because tuples preserve order, iteration follows their positional structure.

12.45 Iterating with enumerate()

When both the position and the value are needed, `enumerate()` can be used:

```
1 features = (  
2     "age",  
3     "income",  
4     "balance",  
5 )  
6  
7 for index, feature in enumerate(features):  
8     print(index, feature)
```

```
0 age  
1 income  
2 balance
```

Each iteration produces a pair:

```
1 (index, value)
```

which is unpacked directly into the two loop variables.

This combines iteration and unpacking naturally.

12.46 Unpacking Tuples in Loops

Suppose a list contains tuples:

```
1 observations = [  
2     ("C001", 0.82),  
3     ("C002", 0.64),  
4     ("C003", 0.91),  
5 ]
```

A loop could access each tuple first:

```
1 for observation in observations:  
2     customer_id = observation[0]  
3     probability = observation[1]  
4  
5     print(customer_id, probability)
```

```
C001 0.82  
C002 0.64  
C003 0.91
```

However, the tuple can be unpacked directly in the loop header:

```
1 for customer_id, probability in observations:  
2     print(customer_id, probability)
```

```
C001 0.82  
C002 0.64  
C003 0.91
```

Conceptually, each iteration performs:

```

1 ("C001", 0.82)
2   ↓   ↓
3 customer_id probability

```

This is one of the most common practical uses of tuple unpacking.

12.47 Unpacking More Complex Records in Loops

The same principle extends to larger records:

```

1 results = [
2     ("model_a", 0.87, 0.81),
3     ("model_b", 0.91, 0.84),
4     ("model_c", 0.89, 0.83),
5 ]
6
7 for model, accuracy, recall in results:
8     print(model, accuracy, recall)

```

```

model_a 0.87 0.81
model_b 0.91 0.84
model_c 0.89 0.83

```

The structure of the loop variables documents the expected structure of every record.

If an element does not contain the expected number of values, unpacking raises an error.

This can be useful because structural inconsistencies become visible rather than being silently ignored.

12.48 Tuples and Other Data Structures

Tuples can contain other Python objects, including lists, dictionaries, sets, and other tuples.

For example:

```

1 data = (
2     [1, 2, 3],
3     {"model": "linear_regression"},
4     {"train", "test"},
5     (10, 20),

```

```
6 )
7
8 data
```

```
([1, 2, 3], {'model': 'linear_regression'}, {'test', 'train'}, (10, 20))
```

The tuple itself remains immutable, but some of the objects it contains may be mutable.

For example:

```
1 data[0].append(4)
2
3 data
```

```
([1, 2, 3, 4], {'model': 'linear_regression'}, {'test', 'train'}, (10, 20))
```

is valid because the list stored at index 0 is being modified.

By contrast:

```
1 data[0] = [4, 5, 6]
```

is invalid because it attempts to replace one of the tuple's references.

12.49 Tuples inside Lists

Lists can contain tuples:

```
1 points = [
2     (1, 2),
3     (3, 4),
4     (5, 6),
5 ]
6
7 points
```

```
[(1, 2), (3, 4), (5, 6)]
```

This is a common pattern when

- the outer collection may grow or shrink;
- each individual record has a fixed structure.

For example:

```
1 points.append((7, 8))
2
3 points
```

```
[(1, 2), (3, 4), (5, 6), (7, 8)]
```

The list is mutable, while each coordinate pair is represented by an immutable tuple.

12.50 Tuples inside Dictionaries

Tuples may appear as dictionary values:

```
1 locations = {
2     "A": (10, 20),
3     "B": (30, 40),
4 }
5
6 locations["A"]
```

```
(10, 20)
```

Suitable tuples can also appear as dictionary keys:

```
1 grid = {
2     (0, 0): "origin",
3     (1, 0): "east",
4     (0, 1): "north",
5 }
6
7 grid[(0, 1)]
```

```
'north'
```

The second pattern requires the tuple to be hashable.

12.51 Tuples inside Sets

A hashable tuple can be stored in a set:

```
1 points = {  
2     (0, 0),  
3     (1, 2),  
4     (3, 4),  
5 }  
6  
7 points
```

`{(0, 0), (1, 2), (3, 4)}`

However, a tuple containing an unhashable object cannot:

```
1 {  
2     (1, [2, 3]),  
3 }
```

raises a `TypeError`.

Therefore, the rule is not:

```
1 sets cannot contain tuples
```

but:

```
1 sets can contain hashable tuples
```

This follows directly from the hashability requirement developed in the previous chapter.

12.52 Converting Iterables to Tuples

The `tuple()` constructor can consume iterable objects.

For example:

```
1 tuple([1, 2, 3])
```

(1, 2, 3)

converts a list to a tuple.

Similarly:

```
1 tuple({1, 2, 3})
```

(1, 2, 3)

constructs a tuple from the elements produced by iterating over the set.

However, because a set does not provide sequence-order semantics, the resulting tuple should not be interpreted as recovering some meaningful original order.

A dictionary behaves differently:

```
1 tuple({  
2     "age": 42,  
3     "income": 100000,  
4 })
```

('age', 'income')

iterates over the dictionary's keys.

Thus, the result is conceptually equivalent to:

```
1 tuple(dictionary.keys())
```

The general principle is:

`tuple(iterable)` constructs a tuple from the sequence of values produced by iterating over the object.

12.53 The `zip()` Function

The built-in `zip()` function combines elements from multiple iterables according to their positions.

Consider:

```
1 names = [  
2     "Alice",  
3     "Bob",  
4     "Charlie",  
5 ]  
6  
7 scores = [  
8     0.87,  
9     0.91,  
10    0.84,  
11 ]
```

Calling:

```
1 paired = zip(names, scores)  
2  
3 paired
```

<zip at 0x1828dc50d00>

does not directly produce a list or tuple.

zip() returns a **zip object**, which is an iterator.

Its values can be materialised using:

```
1 list(paired)
```

[('Alice', 0.87), ('Bob', 0.91), ('Charlie', 0.84)]

producing:

```
1 [  
2     ("Alice", 0.87),  
3     ("Bob", 0.91),  
4     ("Charlie", 0.84),  
5 ]
```

Each generated element is a tuple containing values from corresponding positions.

12.54 Understanding zip() Positionally

Conceptually:

```
1 names      scores
2
3 "Alice"     0.87
4   ↓         ↓
5 ("Alice", 0.87)
6
7 "Bob"       0.91
8   ↓         ↓
9 ("Bob", 0.91)
10
11 "Charlie"   0.84
12   ↓         ↓
13 ("Charlie", 0.84)
```

Thus, `zip()` performs a positional combination of iterables.

It does not match elements by value or search for relationships between them.

12.55 Iterating Directly over zip()

It is often unnecessary to convert a zip object to a list.

For example:

```
1 names = [
2     "Alice",
3     "Bob",
4     "Charlie",
5 ]
6
7 scores = [
8     0.87,
9     0.91,
10    0.84,
11 ]
12
13 for name, score in zip(names, scores):
14     print(name, score)
```

Alice 0.87
Bob 0.91
Charlie 0.84

Each tuple generated by `zip()` is unpacked directly in the loop.

This combines

```
1 iteration
2 +
3 positional pairing
4 +
5 unpacking
```

in one expression.

12.56 `zip()` with More Than Two Iterables

`zip()` can combine more than two iterables.

For example:

```
1 models = [
2     "linear",
3     "tree",
4     "forest",
5 ]
6
7 accuracy = [
8     0.81,
9     0.84,
10    0.89,
11 ]
12
13 recall = [
14     0.78,
15     0.80,
16     0.87,
17 ]
18
19 list(zip(models, accuracy, recall))
```

```
[('linear', 0.81, 0.78), ('tree', 0.84, 0.8), ('forest', 0.89, 0.87)]
```

produces tuples of size three:

```
1 [
2     ("linear", 0.81, 0.78),
3     ("tree", 0.84, 0.80),
4     ("forest", 0.89, 0.87),
5 ]
```

The number of elements in each generated tuple corresponds to the number of input iterables.

12.57 zip() Stops at the Shortest Iterable

By default, `zip()` stops when the shortest input iterable is exhausted.

For example:

```
1 names = [
2     "Alice",
3     "Bob",
4     "Charlie",
5 ]
6
7 scores = [
8     0.87,
9     0.91,
10 ]
11
12 list(zip(names, scores))
```

```
[('Alice', 0.87), ('Bob', 0.91)]
```

produces only two pairs.

The unmatched "Charlie" is not included.

Conceptually:

```

1 names      scores
2
3 Alice      0.87 -> pair
4 Bob        0.91 -> pair
5 Charlie    ---- -> stop

```

This behaviour is important when the iterables are expected to have equal lengths.

12.58 Strict zip()

In modern Python, `zip()` provides the optional argument:

```

1 strict=True

```

when unequal iterable lengths should be considered an error.

For example:

```

1 names = [
2     "Alice",
3     "Bob",
4     "Charlie",
5 ]
6
7 scores = [
8     0.87,
9     0.91,
10 ]
11
12 list(zip(names, scores, strict=True))

```

raises a `ValueError`.

This can be useful when positional correspondence is part of a data invariant.

The distinction is:

```

1 zip(a, b)
2     -> stop at the shortest iterable
3
4 zip(a, b, strict=True)
5     -> require compatible lengths

```

When unequal lengths indicate malformed data rather than an intended truncation, the strict form makes that expectation explicit.

12.59 zip() Objects Are Iterators

A zip object is consumed as it is iterated.

For example:

```
1 names = [  
2     "Alice",  
3     "Bob",  
4 ]  
5  
6 scores = [  
7     0.87,  
8     0.91,  
9 ]  
10  
11 paired = zip(names, scores)  
12  
13 first_pass = list(paired)  
14 second_pass = list(paired)  
15  
16 print(first_pass)  
17 print(second_pass)
```

```
[('Alice', 0.87), ('Bob', 0.91)]  
[]
```

The first conversion consumes the iterator.

The second produces:

```
1 []
```

because no elements remain.

This explains why code such as:

```
1 paired = zip(names, scores)
2
3 list(paired)
4 tuple(paired)
```

does not produce the same data twice.

If the pairs need to be reused, they should be materialised once:

```
1 pairs = list(zip(names, scores))
2
3 print(pairs)
4 print(pairs)
```

```
[('Alice', 0.87), ('Bob', 0.91)]
[('Alice', 0.87), ('Bob', 0.91)]
```

or a new `zip()` object should be constructed.

12.60 Converting `zip()` to a Tuple

The generated pairs can be collected into an outer tuple:

```
1 names = [
2     "Alice",
3     "Bob",
4     "Charlie",
5 ]
6
7 scores = [
8     0.87,
9     0.91,
10    0.84,
11 ]
12
13 pairs = tuple(zip(names, scores))
14
15 pairs
```

```
(('Alice', 0.87), ('Bob', 0.91), ('Charlie', 0.84))
```

The result has the structure:

```
1 (
2   ("Alice", 0.87),
3   ("Bob", 0.91),
4   ("Charlie", 0.84),
5 )
```

Thus, this is a **tuple of tuples**.

12.61 Converting zip() to a Dictionary

When each generated tuple contains two elements, the result can be passed directly to dict():

```
1 names = [
2     "Alice",
3     "Bob",
4     "Charlie",
5 ]
6
7 scores = [
8     0.87,
9     0.91,
10    0.84,
11 ]
12
13 score_by_name = dict(
14     zip(names, scores)
15 )
16
17 score_by_name
```

```
{'Alice': 0.87, 'Bob': 0.91, 'Charlie': 0.84}
```

Conceptually:

```
1 ("Alice", 0.87) -> key-value pair
2 ("Bob", 0.91)   -> key-value pair
3 ("Charlie", 0.84) -> key-value pair
```

This works because `dict()` can construct a mapping from an iterable of two-element key-value pairs.

12.62 Unzipping with `zip(*)`

A collection of tuples can also be separated back into positional groups.

Consider:

```
1 pairs = [  
2     ("Alice", 0.87),  
3     ("Bob", 0.91),  
4     ("Charlie", 0.84),  
5 ]
```

Using:

```
1 names, scores = zip(*pairs)  
2  
3 print(names)  
4 print(scores)
```

```
('Alice', 'Bob', 'Charlie')  
(0.87, 0.91, 0.84)
```

produces:

```
1 ("Alice", "Bob", "Charlie")  
2 (0.87, 0.91, 0.84)
```

The `*` operator expands the collection so that the tuples become separate positional arguments to `zip()`.

Conceptually:

```
1 [  
2     ("Alice", 0.87),  
3     ("Bob", 0.91),  
4     ("Charlie", 0.84),  
5 ]  
6
```



```

7      ↓ *
8
9  zip(
10     ("Alice", 0.87),
11     ("Bob", 0.91),
12     ("Charlie", 0.84),
13 )
14
15     ↓
16
17 ("Alice", "Bob", "Charlie")
18 (0.87, 0.91, 0.84)

```

This operation is commonly described as **unzipping**.

12.63 zip() as a Structural Operation

The most useful way to understand zip() is not simply as a function that “creates tuples.”

Its role is to establish **positional correspondence between iterables**.

For example:

```

1 feature_names = [
2     "age",
3     "income",
4     "balance",
5 ]
6
7 feature_values = [
8     42,
9     85000,
10    1200,
11 ]

```

can be combined as:

```

1 feature_names = [
2     "age",
3     "income",

```

```

4     "balance",
5 ]
6
7 feature_values = [
8     42,
9     85000,
10    1200,
11 ]
12
13 list(zip(feature_names, feature_values))

```

```
[('age', 42), ('income', 85000), ('balance', 1200)]
```

producing:

```

1 [
2     ("age", 42),
3     ("income", 85000),
4     ("balance", 1200),
5 ]

```

The important relationship is:

```

1 position i in feature_names
2     ↓
3 position i in feature_values

```

The tuple is the representation of each resulting positional pairing.

12.64 When Unpacking Improves Readability

Tuple unpacking is particularly effective when the structure of each tuple is small and stable.

For example:

```

1 for customer_id, probability in predictions:
2     ...

```

is generally clearer than:

```
1 for prediction in predictions:
2     customer_id = prediction[0]
3     probability = prediction[1]
4     ...
```

because the loop header documents the expected record structure directly.

However, positional unpacking becomes less readable when records contain many fields:

```
1 for a, b, c, d, e, f, g in records:
2     ...
```

At that point, the issue is not a limitation of tuple unpacking itself but a possible sign that a named record structure would communicate the data more clearly.

12.65 Tuples as Structural Sequences

The operations developed in this section illustrate an important aspect of tuples.

Their usefulness is not limited to being “lists that cannot be changed.”

Tuples work particularly well when **position has structural meaning**.

Examples include:

```
1 (x, y)
2 (name, score)
3 (customer_id, probability)
4 (key, value)
5 (model, accuracy, recall)
```

This structural interpretation explains why tuples appear naturally in

- multiple assignment;
- function return values;
- dictionary iteration;
- `enumerate()`;
- `zip()`;
- coordinate systems;
- composite keys;
- fixed records.

Their immutability complements this role by indicating that the outer positional structure is not intended to change.

12.66 Practical Tuple Patterns

Tuples are particularly useful when a small number of values belong together and their **positions have stable meanings**.

This occurs frequently in Python because many operations naturally produce fixed-size records.

For example:

```
1 result = (  
2     "random_forest",  
3     0.91,  
4     0.84,  
5 )  
6  
7 result
```

('random_forest', 0.91, 0.84)

may represent

```
1 (model name, accuracy, recall)
```

The usefulness of the tuple comes from the relationship between the positions:

```
1 index 0 -> model name  
2 index 1 -> accuracy  
3 index 2 -> recall
```

This structure can then be unpacked:

```
1 model, accuracy, recall = result  
2  
3 print(model)  
4 print(accuracy)  
5 print(recall)
```

```
random_forest  
0.91  
0.84
```

The tuple therefore serves not merely as a container, but as a compact **structural record**.

12.67 Fixed-Size Records

Suppose several model evaluations have the same structure:

```
1 results = [  
2     ("linear", 0.84, 0.79),  
3     ("forest", 0.91, 0.87),  
4     ("svm", 0.88, 0.82),  
5 ]
```

Each tuple represents:

```
1 (model, accuracy, recall)
```

and can be unpacked directly:

```
1 for model, accuracy, recall in results:  
2     print(model, accuracy, recall)
```

```
linear 0.84 0.79  
forest 0.91 0.87  
svm 0.88 0.82
```

The outer list is mutable and may contain any number of records, while every record has a fixed positional structure.

This combination is common:

```
1 list of tuples
```

when

- the collection may grow or shrink;
- each individual record has a stable shape.

12.68 Tuples versus Dictionaries as Records

A tuple record:

```
1 result = (  
2     "forest",  
3     0.91,  
4     0.84,  
5 )
```

uses position to encode meaning.

A dictionary:

```
1 result = {  
2     "model": "forest",  
3     "accuracy": 0.91,  
4     "recall": 0.84,  
5 }
```

uses explicit keys.

The tuple is compact and convenient when the structure is small and already understood.

The dictionary is generally more self-describing.

Compare:

```
1 result[1]
```

with:

```
1 result["accuracy"]
```

The dictionary access communicates the field meaning directly.

Thus, tuples are particularly appropriate when

- the record is small;
- field order is stable;
- positional meaning is obvious in context;
- unpacking makes the structure clear.

As records become larger or more complex, named structures are often easier to maintain.

12.69 Tuples versus Lists

A list and tuple may contain identical values:

```
1 list_value = [1, 2, 3]
2 tuple_value = (1, 2, 3)
3
4 print(list_value)
5 print(tuple_value)
```

```
[1, 2, 3]
(1, 2, 3)
```

but they communicate different intentions.

A list suggests:

```
1 this sequence may be modified
```

whereas a tuple suggests:

```
1 this positional structure is fixed
```

This distinction is semantic rather than merely syntactic.

For example:

```
1 feature_names = [
2     "age",
3     "income",
4     "balance",
5 ]
```

is naturally a list if features may be inserted or removed.

By contrast:

```
1 coordinate = (3.5, 7.2)
```

is naturally a tuple because a two-dimensional coordinate has a fixed structure.

12.70 Tuples versus Sets

Both tuples and sets can contain several values, but they model fundamentally different relationships.

A tuple:

```
1 ("age", "income", "balance")
```

means:

```
1 first value
2 second value
3 third value
```

A set:

```
1 {"age", "income", "balance"}
```

means:

```
1 these distinct values are members
```

The distinction is therefore:

```
1 tuple -> position and order
2 set  -> membership and uniqueness
```

A tuple preserves duplicates:

```
1 (1, 2, 2, 3)
```

(1, 2, 2, 3)

whereas:

```
1 {1, 2, 2, 3}
```

{1, 2, 3}

represents the same set as:


```
1 {1, 2, 3}
```

12.71 Tuples versus Dictionaries and Sets as Hashable Objects

A sufficiently simple tuple can be hashable:

```
1 key = (  
2     "forest",  
3     42,  
4 )  
5  
6 hash(key)
```

```
6332411540823100017
```

This allows it to serve as a dictionary key:

```
1 scores = {  
2     ("forest", 42): 0.91,  
3     ("forest", 43): 0.89,  
4 }  
5  
6 scores[("forest", 42)]
```

```
0.91
```

or as an element of a set:

```
1 experiments = {  
2     ("forest", 42),  
3     ("forest", 43),  
4     ("linear", 42),  
5 }  
6  
7 experiments
```

```
{('forest', 42), ('forest', 43), ('linear', 42)}
```

This is an important difference from lists.

A list containing the same values:

```
1 ["forest", 42]
```

is not hashable and therefore cannot serve as a dictionary key or set element.

12.72 Composite Identifiers

Tuples are useful when an identifier naturally has several components.

For example:

```
1 measurement = {  
2     ("station_a", "temperature"): 21.4,  
3     ("station_a", "humidity"): 0.63,  
4     ("station_b", "temperature"): 19.8,  
5 }  
6  
7 measurement[("station_a", "humidity")]
```

0.63

The key:

```
1 ("station_a", "humidity")
```

represents a composite identifier with two dimensions.

Similarly:

```
1 results = {  
2     ("forest", 42): 0.91,  
3     ("forest", 123): 0.90,  
4     ("linear", 42): 0.84,  
5 }
```

can represent:

```
1 (model, random_state) -> score
```

This is often clearer than manually constructing encoded strings such as:

```
1 "forest_42"
```

because the components remain explicit and independently accessible.

12.73 Structural Validation through Unpacking

Unpacking can provide a simple form of structural validation.

Suppose every record is expected to have exactly three values:

```
1 (model, accuracy, recall)
```

Then:

```
1 record = (  
2     "forest",  
3     0.91,  
4     0.84,  
5 )  
6  
7 model, accuracy, recall = record
```

succeeds.

However:

```
1 record = (  
2     "forest",  
3     0.91,  
4 )  
5  
6 model, accuracy, recall = record
```

raises a `ValueError`.

Thus, unpacking makes an assumption about the expected structure explicit.

This should not be confused with full data validation, but it can expose malformed positional records immediately.

12.74 Swapping and State Updates

Tuple-style multiple assignment can be useful when several variables must be updated simultaneously.

For example:

```
1 a = 3
2 b = 7
3
4 a, b = b, a
5
6 print(a, b)
```

7 3

A more algorithmic example appears in sequences such as Fibonacci numbers:

```
1 a, b = 0, 1
2
3 for _ in range(8):
4     print(a)
5     a, b = b, a + b
```

0
1
1
2
3
5
8
13

The right-hand expressions are evaluated before the new bindings are assigned.

Conceptually:

$$(a, b) \mapsto (b, a + b).$$

This pattern provides a concise representation of a state transition involving several variables.

12.75 Functions Returning Structured Results

Functions frequently return tuples when several closely related outputs are produced.

For example:

```
1 def evaluate(values):
2     return min(values), max(values), sum(values) / len(values)
3
4
5 minimum, maximum, mean = evaluate(
6     [4, 7, 1, 9, 3]
7 )
8
9 print(minimum)
10 print(maximum)
11 print(mean)
```

```
1
9
4.8
```

The tuple creates one return object containing several results, while unpacking makes those components immediately available to the caller.

For a small, stable result structure this pattern is convenient.

For a large number of outputs, however, positional meaning may become difficult to remember and a named structure can provide clearer semantics.

12.76 Tuples and Function Arguments

Unpacking also works in the opposite direction when passing arguments to a function.

Consider:

```
1 def distance_squared(x, y):
2     return x ** 2 + y ** 2
3
4
5 point = (
6     3,
```

```
7  4,  
8  )
```

The tuple can be expanded into positional arguments:

```
1  distance_squared(*point)
```

25

This is equivalent to:

```
1  distance_squared(  
2      point[0],  
3      point[1],  
4  )
```

The * operator therefore has complementary roles:

```
1  assignment context -> collect multiple values  
2  function call      -> expand an iterable into positional arguments
```

The meaning depends on context.

12.77 Combining enumerate() and Tuples

The built-in enumerate() function produces pairs consisting of an index and a value.

For example:

```
1  models = (  
2      "linear",  
3      "tree",  
4      "forest",  
5  )  
6  
7  for index, model in enumerate(models):  
8      print(index, model)
```

```
0 linear
1 tree
2 forest
```

Each iteration effectively produces a tuple:

```
1 (0, "linear")
2 (1, "tree")
3 (2, "forest")
```

which is unpacked directly.

Thus, tuple-like structures appear naturally even when the programmer does not explicitly write tuple literals.

12.78 Dictionary items() and Tuples

Dictionary iteration using items() follows a similar pattern:

```
1 scores = {
2     "linear": 0.82,
3     "tree": 0.78,
4     "forest": 0.91,
5 }
6
7 for model, score in scores.items():
8     print(model, score)
```

```
linear 0.82
tree 0.78
forest 0.91
```

Conceptually, each iteration provides a two-component structure:

```
1 (key, value)
```

that can be unpacked.

This reinforces an important observation:

unpacking is a general iterable operation, although tuples are one of its most natural representations.

12.79 Comprehensions Producing Tuples

List comprehensions can construct collections of tuple records.

For example:

```
1 values = [  
2     -2,  
3     0,  
4     5,  
5 ]  
6  
7 classified = [  
8     (  
9         value,  
10        "positive"  
11        if value > 0  
12        else "negative"  
13        if value < 0  
14        else "zero",  
15     )  
16     for value in values  
17 ]  
18  
19 classified
```

```
[(-2, 'negative'), (0, 'zero'), (5, 'positive')]
```

The outer structure is a mutable list, while each individual classification is represented by a fixed two-element tuple.

Similarly:

```
1 squares = [  
2     (x, x ** 2)  
3     for x in range(1, 6)  
4 ]  
5  
6 squares
```

```
[(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```


produces:

```
1  [  
2    (1, 1),  
3    (2, 4),  
4    (3, 9),  
5    (4, 16),  
6    (5, 25),  
7  ]
```

This pattern is useful when each input produces a small structured result.

12.80 Sorting Collections of Tuples

Tuples support lexicographic comparison when their elements support the required comparisons.

For example:

```
1  records = [  
2    ("forest", 0.91),  
3    ("linear", 0.84),  
4    ("svm", 0.88),  
5  ]  
6  
7  sorted(records)
```

```
[('forest', 0.91), ('linear', 0.84), ('svm', 0.88)]
```

sorts first according to the first tuple element.

If another criterion is required:

```
1  sorted(  
2    records,  
3    key=lambda record: record[1],  
4    reverse=True,  
5  )
```

```
[('forest', 0.91), ('svm', 0.88), ('linear', 0.84)]
```

sorts by score.

The tuple itself remains immutable; `sorted()` produces a new list containing the tuple objects in the requested order.

12.81 Lexicographic Comparison

Tuple comparison proceeds position by position.

For example:

```
1 (1, 5) < (2, 0)
```

True

is True because the first differing components satisfy:

$$1 < 2.$$

Similarly:

```
1 (1, 5) < (1, 7)
```

True

is True because the first elements are equal and the second comparison determines the result:

$$5 < 7.$$

This is known as **lexicographic ordering**.

Conceptually, for tuples

$$(a_1, a_2, \dots)$$

and

$$(b_1, b_2, \dots),$$

Python compares the first position at which the corresponding elements differ.

This behaviour can be useful for naturally ordered composite values such as:

```
1 (year, month, day)
```

or:

```
1 (primary criterion, secondary criterion)
```

provided the ordering semantics match the problem.

12.82 Computational Characteristics

Tuples are sequence structures and share several broad computational characteristics with lists.

Direct positional access:

```
1 values[i]
```

has constant-time behaviour:

$$O(1).$$

Membership testing:

```
1 x in values
```

generally requires a sequential search:

$$O(n).$$

Similarly:

```
1 values.index(x)
```

may inspect multiple elements before locating the first match.

Tuple concatenation:

1 A + B

constructs a new tuple and therefore requires work proportional to the amount of data copied into the result.

Because tuples are immutable, repeated concatenation is generally not an appropriate mechanism for dynamically accumulating large collections.

If a sequence needs to grow repeatedly during a computation, a list is usually a more natural structure.

12.83 Choosing among the Main Data Structures

The four structures developed so far represent different core semantics.

Data structure	Main abstraction	Mutable	Positional	Unique elements	Key-value association
list	mutable sequence	yes	yes	no	no
tuple	fixed sequence	no	yes	no	no
dict	mapping	yes	no	keys unique	yes
set	membership	yes	no	yes	no

Table 15: Conceptual comparison of the principal Python collection types.

A useful first question when choosing among them is:

1 What relationship does the data represent?

If the answer is:

1 an ordered sequence that may change

a list is a natural choice.

If it is:

1 a fixed positional structure

a tuple may be appropriate.

If it is:

1 an identifier associated with a value

a dictionary represents that mapping directly.

If it is:

1 unique membership

a set is the natural abstraction.

The appropriate choice follows the semantics of the data rather than the superficial similarity of the container syntax.

13 Problems - Tuples

The following problems focus on tuple semantics, immutability, packing and unpacking, hashability, iteration, `zip()`, and interaction with other Python data structures.

Where possible, predict the result before executing the code.

13.1 Problem 1 — Basic Tuple Creation

Given:

```
1 values = (  
2     10,  
3     20,  
4     30,  
5 )
```

determine:

```
1 type(values)
2 len(values)
3 values[0]
4 values[-1]
```

Explain the positional relationship between the positive and negative indices.

13.2 Problem 2 — The Single-Element Tuple

Predict:

```
1 a = ("python")
2 b = ("python",)
3
4 type(a)
5 type(b)
6 len(b)
```

Explain why the comma is required.

13.3 Problem 3 — Parentheses versus Commas

Predict the types of:

```
1 a = 1, 2, 3
2 b = (1, 2, 3)
3 c = (1)
4 d = 1,
```

Which syntactic element is essential to tuple construction?

13.4 Problem 4 — Converting an Iterable

Predict:

```
1 tuple("data")
```

and:

```
1 ("data",)
```

Explain the structural difference.

13.5 Problem 5 — Indexing

Given:

```
1 values = (  
2     "a",  
3     "b",  
4     "c",  
5     "d",  
6     "e",  
7 )
```

determine:

```
1 values[2]  
2 values[-2]  
3 values[-5]
```

State the corresponding non-negative index for each negative index.

13.6 Problem 6 — Slicing

Given:

```
1 values = tuple(range(10))
```

predict:

```
1 values[2:6]  
2 values[:4]  
3 values[6:]  
4 values[::2]  
5 values[::-1]
```

For each expression, identify the selected indices.

13.7 Problem 7 — Membership

Given:

```
1 features = (  
2     "age",  
3     "income",  
4     "balance",  
5 )
```

predict:

```
1 "income" in features  
2 "country" in features  
3 "country" not in features
```

Why does repeated tuple membership testing have different computational characteristics from set membership?

13.8 Problem 8 — Duplicate Values

Given:

```
1 values = (  
2     1,  
3     2,  
4     2,  
5     3,  
6     2,  
7 )
```

determine:

```
1 len(values)  
2 values.count(2)  
3 values.index(2)
```

Explain why duplicate occurrences are meaningful in tuples.

13.9 Problem 9 — Immutability

Predict what happens:

```
1 values = (  
2     1,  
3     2,  
4     3,  
5 )  
6  
7 values[1] = 10
```

What property of tuples causes the result?

13.10 Problem 10 — Rebinding Is Not Mutation

Consider:

```
1 values = (  
2     1,  
3     2,  
4     3,  
5 )  
6  
7 values = (  
8     4,  
9     5,  
10    6,  
11 )
```

Why is this valid even though tuples are immutable?

13.11 Problem 11 — Mutable Object inside a Tuple

Predict:

```

1 data = (
2     "model",
3     [0.81, 0.84],
4 )
5
6 data[1].append(0.89)
7
8 print(data)

```

Why does this not violate tuple immutability?

13.12 Problem 12 — Replacing a Mutable Element

Using the same initial tuple:

```

1 data = (
2     "model",
3     [0.81, 0.84],
4 )

```

predict what happens with:

```

1 data[1] = [0.90]

```

Explain the distinction between mutating the contained list and replacing the tuple element.

13.13 Problem 13 — Tuple Hashability

Determine whether each tuple can be hashed:

```

1 (1, 2, 3)
2 ("model", 42)
3 ((1, 2), (3, 4))
4 (1, [2, 3])

```

Explain each result.

13.14 Problem 14 — Tuple as a Dictionary Key

Construct a dictionary mapping:

```
1 ("forest", 42) -> 0.91
2 ("forest", 123) -> 0.89
3 ("linear", 42) -> 0.84
```

Retrieve the score associated with ("forest", 123).

Why is a tuple appropriate for this key?

13.15 Problem 15 — Tuple as a Set Element

Predict whether the following is valid:

```
1 points = {
2     (1, 2),
3     (3, 4),
4 }
```

Then compare with:

```
1 points = {
2     (1, [2, 3]),
3 }
```

Explain the difference using hashability.

13.16 Problem 16 — Basic Packing

Starting from:

```
1 x = 3
2 y = 7
```

pack the two values into a tuple called point without explicitly calling tuple().

13.17 Problem 17 — Basic Unpacking

Given:

```
1 record = (  
2     "C001",  
3     42,  
4     0.87,  
5 )
```

assign the three elements to:

```
1 customer_id  
2 age  
3 probability
```

using one assignment statement.

13.18 Problem 18 — Unpacking Error

Predict what happens:

```
1 record = (  
2     "C001",  
3     42,  
4     0.87,  
5 )  
6  
7 customer_id, probability = record
```

Why does Python raise an exception rather than silently discarding one value?

13.19 Problem 19 — Starred Unpacking

Given:

```
1 values = (  
2     1,  
3     2,  
4     3,  
5     4,  
6     5,  
7 )
```

predict:

```
1 first, *middle, last = values
```

Determine the values and types of first, middle, and last.

13.20 Problem 20 — Ignoring Values

Given:

```
1 record = (  
2     "C001",  
3     42,  
4     "Argentina",  
5     0.87,  
6 )
```

retrieve only the customer identifier and probability using unpacking.

Use `_` or a starred ignored target for the intermediate information.

13.21 Problem 21 — Swapping Variables

Starting from:

```
1 a = 10  
2 b = 20
```

swap the values without using a temporary variable.

Explain conceptually why the operation works.

13.22 Problem 22 — Nested Unpacking

Given:

```
1 record = (  
2     "C001",  
3     (3.5, 7.2),  
4 )
```

unpack the values into:

```
1 customer_id  
2 x  
3 y
```

using one assignment statement.

13.23 Problem 23 — Function Returning a Tuple

Write a function that receives a sequence of numbers and returns:

```
1 minimum  
2 maximum  
3 mean
```

as one tuple.

Then unpack the returned value directly into three variables.

13.24 Problem 24 — Expanding Function Arguments

Given:

```
1 def rectangle_area(width, height):  
2     return width * height  
3  
4 dimensions = (  
5     4,  
6     7,  
7 )
```

call `rectangle_area()` using `*dimensions`.

Explain the role of `*` in this context.

13.25 Problem 25 — Concatenation

Given:

```
1 A = (  
2     1,  
3     2,  
4 )  
5  
6 B = (  
7     3,  
8     4,  
9 )
```

construct:

```
1 (1, 2, 3, 4)
```

using tuple concatenation.

Are A or B modified?

13.26 Problem 26 — Repetition

Predict:

```
1 ("A", "B") * 3
```

Then explain the difference between:

```
1 (0,) * 4
```

and:

```
1 (0) * 4
```

13.27 Problem 27 — Repeated Mutable Reference

Predict:

```
1 values = ([],) * 3
2
3 values[0].append("x")
4
5 print(values)
```

Why do all three positions appear to change?

13.28 Problem 28 — Iterating with Unpacking

Given:

```
1 results = [
2     ("linear", 0.82),
3     ("tree", 0.78),
4     ("forest", 0.91),
5 ]
```

use one for loop to print each model and score without indexing the tuple explicitly.

13.29 Problem 29 — enumerate()

Given:

```
1 models = (
2     "linear",
3     "tree",
4     "forest",
5 )
```

use enumerate() to print:


```
1 0 linear
2 1 tree
3 2 forest
```

Explain where tuple unpacking occurs.

13.30 Problem 30 — zip()

Given:

```
1 models = [
2     "linear",
3     "tree",
4     "forest",
5 ]
6
7 scores = [
8     0.82,
9     0.78,
10    0.91,
11 ]
```

construct a list of tuples:

```
1 [
2     ("linear", 0.82),
3     ("tree", 0.78),
4     ("forest", 0.91),
5 ]
```

using zip().

13.31 Problem 31 — zip() Iterator Consumption

Predict:

```

1 a = [1, 2, 3]
2 b = ["A", "B", "C"]
3
4 pairs = zip(a, b)
5
6 first = list(pairs)
7 second = list(pairs)

```

What are first and second?

Explain the result.

13.32 Problem 32 — Unequal Lengths with zip()

Predict:

```

1 a = [1, 2, 3]
2 b = ["A", "B"]
3
4 list(zip(a, b))

```

Which value is discarded?

Then explain how:

```

1 zip(a, b, strict=True)

```

changes the behaviour.

13.33 Problem 33 — Unzipping

Given:

```

1 pairs = [
2     ("A", 10),
3     ("B", 20),
4     ("C", 30),
5 ]

```

use:

```
1 zip(*pairs)
```

to recover the keys and values into separate tuples.

13.34 Problem 34 — Classification by Sign

Given a sequence of integers, construct a list of tuples of the form:

```
1 (number, classification)
```

where the classification is:

```
1 "positive"
2 "negative"
3 "zero"
```

according to the sign of each integer.

For example:

```
1 values = [
2     -3,
3     0,
4     5,
5 ]
```

should produce:

```
1 [
2     (-3, "negative"),
3     (0, "zero"),
4     (5, "positive"),
5 ]
```

Solve the problem first using an explicit loop.

13.35 Problem 35 — Grade Classification

Given integer grades from 1 through 10, classify each grade using:

$$[1, 5) \rightarrow \text{fail},$$
$$[5, 7) \rightarrow \text{pass},$$
$$[7, 9) \rightarrow \text{notable},$$

and:

$$[9, 10] \rightarrow \text{excellent}.$$

Create tuples of the form:

```
1 (grade, classification)
```

for a supplied sequence of grades.

13.36 Problem 36 — Character Counts as Tuple Records

Given a phrase, calculate:

- number of vowels;
- number of consonants;
- number of spaces.

Store the results as:

```
1 [  
2   ("Vowels", ...),  
3   ("Consonants", ...),  
4   ("Spaces", ...),  
5 ]
```

Treat uppercase and lowercase letters equivalently.

13.37 Problem 37 — Average by Subject

Given records of the form:

```
1 (subject, grade)
```

for subjects such as:

```
1 Math
2 Language
3 History
4 Computer Science
5 Music
```

calculate the average grade for every subject that has at least one observation.

Use tuples to represent the input observations.

Choose an appropriate additional data structure for grouping the grades and explain why that structure is useful.

13.38 Problem 38 — Points and Quadrants

Given a collection of two-dimensional points represented by:

```
1 (x, y)
```

construct tuples of the form:

```
1 (x, y, region)
```

where region identifies the quadrant.

Treat points lying on an axis separately rather than assigning them arbitrarily to two possible quadrants.

For example, distinguish at least:

```
1 quadrant I
2 quadrant II
3 quadrant III
4 quadrant IV
5 x-axis
6 y-axis
7 origin
```

13.39 Problem 39 — Words and Initial Letters

Given:

```
1 words = [
2     "wave",
3     "snail",
4     "pineapple",
5     "beach",
6     "barbecue",
7     "window",
8     "fly",
9 ]
```

create an iterable of tuples:

```
1 (word, first_letter)
```

using `zip()`.

Then convert the result to a tuple of tuples.

13.40 Problem 40 — Reasoning about Tuples and References

Without executing the code, determine the exact results:

```
1 A = (
2     [1, 2],
3     "model",
4 )
5
```

```

6 B = A
7 C = A[:]
8
9 A[0].append(3)
10
11 print(A)
12 print(B)
13 print(C)
14
15 print(A is B)
16 print(A is C)
17 print(A == C)
18 print(A[0] is C[0])

```

Explain carefully:

1. why mutating the inner list is allowed;
2. whether slicing creates a distinct tuple object in this case;
3. why object identity and tuple equality must be considered separately.

14 Summary

A Python tuple is an ordered, immutable sequence.

Like a list, it supports:

```

1 indexing
2 negative indexing
3 slicing
4 membership
5 iteration
6 concatenation
7 repetition

```

but unlike a list, its sequence of references cannot be modified after creation.

Tuple syntax has an important characteristic:

```

1 the comma creates the tuple

```

which explains why:

```
1 ("value")
```

is not a one-element tuple, while:

```
1 ("value",)
```

is.

Tuples are closely associated with **packing and unpacking**:

```
1 point = x, y
2 x, y = point
```

and Python extends this model through:

```
1 starred unpacking
2 nested unpacking
3 multiple assignment
4 swapping
5 loop unpacking
6 function return values
7 argument expansion
```

Tuple immutability should not be confused with recursive immutability. A tuple may contain mutable objects, and those objects can still change:

```
1 data = (
2     [1, 2],
3     "model",
4 )
5
6 data[0].append(3)
```

The tuple's outer sequence of references remains unchanged.

Similarly, tuple immutability does not automatically imply hashability. A tuple is hashable only when its contents allow the tuple itself to be hashed. Suitable tuples can therefore serve as:


```
1 dictionary keys
2 set elements
3 composite identifiers
```

Tuple unpacking is particularly useful when position has structural meaning:

```
1 (x, y)
2 (model, score)
3 (customer_id, probability)
4 (key, value)
```

and appears naturally throughout Python in operations such as:

```
1 enumerate()
2 dict.items()
3 zip()
4 function returns
```

The built-in `zip()` function establishes positional correspondence between multiple iterables and produces tuple-like records during iteration. Its default behaviour stops at the shortest input, while `strict=True` can enforce equal lengths when positional alignment is an invariant.

Among the principal Python collection types considered so far, the central distinctions are:

```
1 list    -> mutable sequence
2 tuple   -> fixed sequence
3 dictionary -> mapping
4 set     -> membership
```

The appropriate data structure is therefore determined by the relationship the data represents rather than merely by which syntax is shortest.

For tuples specifically, the central question is:

Does this collection represent a fixed positional structure whose positions have stable meaning?

When the answer is yes, a tuple is often a natural representation.